BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 1: Giới thiệu chung

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường ĐH Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung

EBOOKBKMT.COM



Nội dung chương 1

- Hệ điều hành (Operating System) làm việc gì?
- Tổ chức của hệ thống máy tính
- Cấu trúc của HĐH
- Hoạt động của HĐH
- Sự quản lý tiến trình
- Sự quản lý bộ nhớ chính
- Sự quản lý bộ nhớ lưu trữ
- Protection và Security
- Các môi trường sử dụng máy tính





Mục tiêu

- Cung cấp một chuyến du ngoạn lớn qua các thành phần chính của hệ điều hành.
- Cung cấp sự tổng quát về tổ chức hệ thống máy tính cơ bản.





1.1. Hệ điều hành làm việc gì

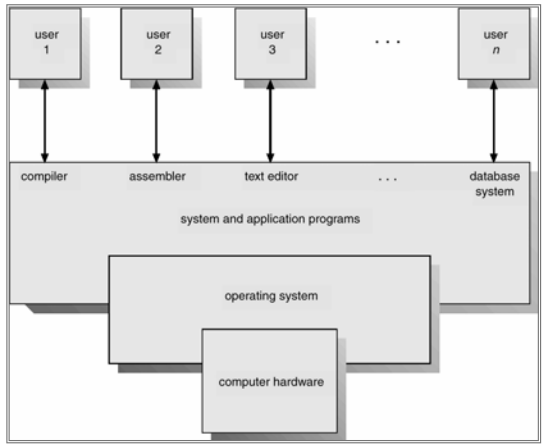
- Hệ điều hành (Operating System): Là một chương trình hoạt động như một lớp trung gian giữa người sử dụng máy tính và phần cứng máy tính.
- Các mục đích của HĐH:
  - Thực hiện các chương trình của người sử dụng và giúp việc giải các bài toán của người sử dụng dễ dàng hơn.
  - Giúp cho việc sử dụng hệ thống máy tính thuận tiện hơn.
  - Sử dụng phần cứng máy tính theo một cách hiệu quả.



Cấu trúc của hệ thống máy tính

1. Phần cứng (Hardware) – cung cấp các tài nguyên tính toán cơ bản (CPU, memory, I/O devices)
2. Hệ điều hành (Operating system) – điều khiển và sắp xếp việc sử dụng phần cứng trong các chương trình ứng dụng khác nhau đối với những người sử dụng khác nhau.
3. Các chương trình ứng dụng (Applications programs) – định cách sử dụng tài nguyên hệ thống để giải quyết các bài toán của người sử dụng (word processors, compilers, database systems, video games)
4. Users (people, machines, other computers)

Bốn thành phần hệ thống máy tính



Các cách nhìn đối với hệ điều hành

- Là trình phân phối tài nguyên (Resource allocator) – quản lý và quyết định phân phối các tài nguyên (CPU, không gian bộ nhớ, các thiết bị vào/ra...) cho các yêu cầu nhằm đạt hiệu quả và công bằng.
- Là một chương trình điều khiển (Control program) – điều khiển sự thực hiện các chương trình của người sử dụng và sự hoạt động của các thiết bị vào/ra để ngăn các lỗi và sự sử dụng sai.
- Không có định nghĩa hoàn toán đúng về HĐH.
- **Kernel** (nhân) – là một chương trình chạy liên tục không ngừng trên máy tính (tất cả các chương trình khác là chương trình hệ thống hoặc chương trình ứng dụng).

Những gì dễ nhầm với hệ điều hành?

- Các lệnh/ứng dụng truy vấn trạng thái hệ thống: *ls* (UNIX), Task Manager (Windows). Đó là các công cụ và không chạy liên tục.
- Các trình điều khiển thiết bị ngoại vi (drivers): khiến cho HĐH có thể sử dụng phần cứng mới, chúng là OS extensions chứ không phải là bản thân OS. Cũng giống như browser plugin và browser.
- Các phần mềm có thể đơn phương truy nhập phần cứng:
  - VMWare là một virtual PC (không phải hardware). Có thể cài Windows "trên" nó.
  - Java VM là một *virtual* machine (không phải hardware), API (application program interface) của nó là một HĐH suy rộng. (Careful: JavaOS là một project riêng)

Tại sao phải nghiên cứu HĐH?

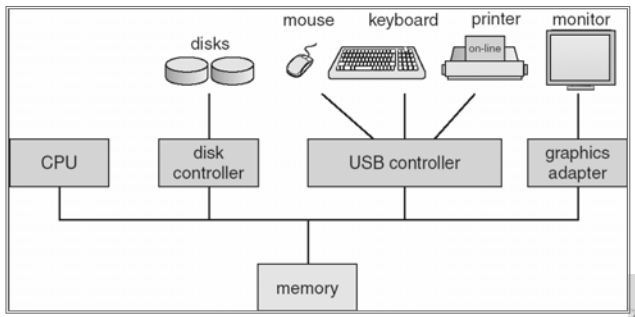
- Thiết kế HĐH là một nghiên cứu riêng (case study) rất tốt về thiết kế kỹ nghệ phần mềm.
- The better you know the OS, the better apps you write, the better you understand its bugs and work around them.
- HĐH sử dụng các thuật giải phức tạp, rất nhiều trong đó có thể được sử dụng lại trong các phần mềm khác, vd: phát hiện bế tắc (deadlock detection).
- HĐH cần có sự phát triển nhanh của phần cứng.

Sự khởi động máy tính

- **bootstrap program** (chương trình mồi) được nạp khi bật máy hoặc khi khởi động lại.
  - Thường được chứa trong ROM hoặc EPROM, thường được gọi là **firmware**.
  - Khởi tạo tất cả các khía cạnh của hệ thống.
  - Nạp nhân (kernel) của HĐH và bắt đầu sự thực hiện.

1.2. Tổ chức của hệ thống máy tính

- Một hoặc nhiều CPU, device controler kết nối với nhau bằng bus chung cho phép truy nhập bộ nhớ chia sẻ.
- Sự thực hiện đồng thời của CPU và các thiết bị cạnh tranh các chu kỳ bộ nhớ.



Hoạt động của hệ thống máy tính

- Các thiết bị vào-ra và CPU có thể thực hiện đồng thời.
- Mỗi mạch điều khiển thiết bị (device controller) phụ trách một loại thiết bị riêng và có một bộ nhớ đệm riêng (local buffer).
- CPU chuyển dữ liệu từ/đến bộ nhớ chính đến/từ các buffer.
- Vào-ra từ thiết bị đến local buffer của mạch điều khiển.
- Mạch điều khiển thiết bị thông báo cho CPU biết nó đã hoàn tất công việc của nó bằng cách gây ra một *ngắt* (*interrupt)*.

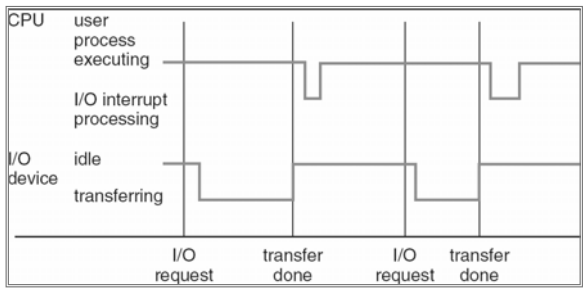
Các chức năng chính của ngắt

- HĐH được điều khiển bằng ngắt *(interrupt driven*), nghĩa là mã lệnh của nó chỉ được gọi đến khi ngắt xuất hiện.
- Thông thường ngắt chuyển điều khiển cho thường trình dịch vụ ngắt thông qua *vector ngắt* (*interrupt vector)*, có chứa địa chỉ của tất cả các thường trình dịch vụ ngắt (interrupt service routine).
- Kiến trúc ngắt (Interrupt architecture) phải lưu địa chỉ của lệnh bị ngắt.
- Các ngắt đến *bị vô hiệu* (*disabled)* trong khi một ngắt đang được thực hiện để tránh bị mất ngắt (*lost interrupt)*.
- Một *bẫy (trap)* là phần mềm tạo ngắt gây ra bởi một lỗi hoặc yêu cầu của người sử dụng.

Xử lý ngắt

- HĐH duy trì trạng thái của CPU bằng cách lưu giữ nội dung các thanh ghi, bộ đếm chương trình (program counter - PC) và địa chỉ của lệnh bị ngắt.
- HĐH xác định loại ngắt nào đã xuất hiện và có những hành động thực hiện tương ứng:
  - *polling*
  - *vectored*
- Thường trình dịch vụ ngắt (interrupt service routine) chịu trách nhiệm thực hiện các ngắt, CPU được dành cho xử lý ngắt.
- Sau khi phục vụ ngắt, HĐH khôi phục lại ngữ cảnh trước ngắt

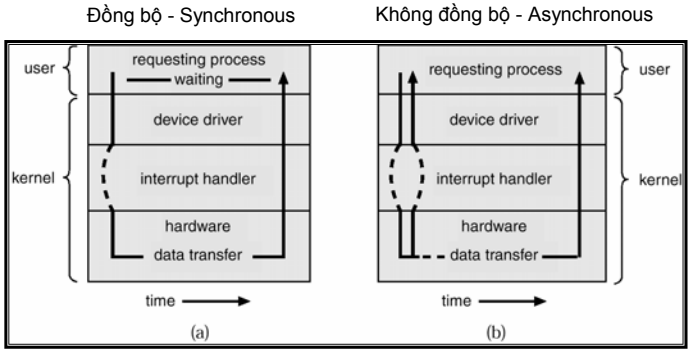
Interrupt Timeline



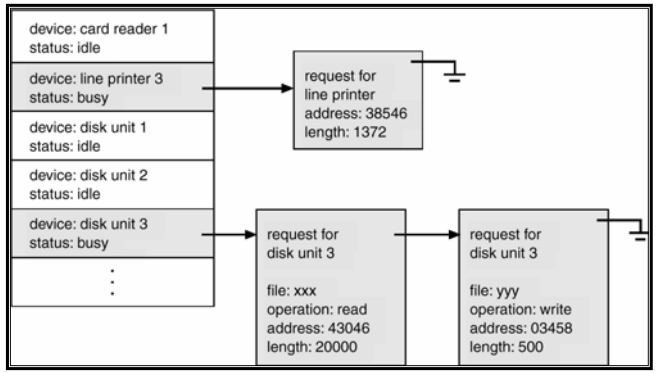
Cấu trúc vào-ra

- *Synchronous I/O:* Sau khi bắt đầu vào-ra, quyền kiểm soát chỉ quay lại chương trình của người sử dụng khi vào-ra đó kết thúc.
  - Chờ lệnh làm rỗi CPU cho đến khi có lệnh ngắt kế tiếp.
  - Chờ theo vòng lặp kiểm tra CPU rỗi → tranh chấp truy nhập bộ nhớ.
  - Tại một thời điểm, chỉ có 1 yêu cầu vào-ra được thực hiện, không có sự xử lý I/O đồng thời.
- *Asynchronous I/O:* Sau khi I/O bắt đầu, quyền kiểm soát quay lại chương trình của người sử dụng mà không cần chờ I/O kết thúc.
  - *System call* – gửi yêu cầu tới HĐH cho phép người sử dụng đợi I/O kết thúc (nếu muốn).
  - *Bảng trạng thái thiết bị (Device-status table)* chứa thông tin (entry) của mỗi thiết bị I/O cho biết trạng thái, địa chỉ và loại của thiết bị.
  - HĐH tra cứu vào bảng I/O device để xác định trạng thái thiết bị và sửa đổi thông tin để thêm thông tin ngắt.

Mô tả 2 phương pháp vào-ra



Bảng trạng thái thiết bị



Cấu trúc DMA (Direct Memory Access)

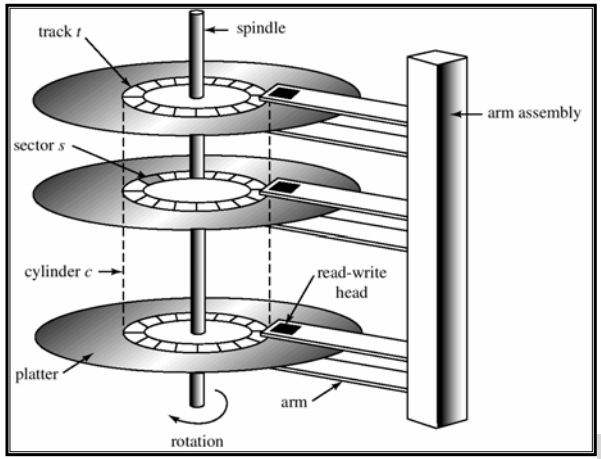
- Được sử dụng cho các thiết bị tốc độ cao (disk, communications network) để có thể tăng tốc độ trao đổi thông tin gần tới tốc độ bộ nhớ.
- Mạch điều khiển thiết bị chuyển toàn bộ các khối dữ liệu (block of data) từ bộ nhớ buffer trực tiếp tới bộ nhớ chính không qua sự can thiệp của CPU.
- Chỉ có 01 ngắt được sinh ra đối với mỗi block, tối ưu hơn là mỗi ngắt đối với 01 byte (hoặc word) đối với các thiết bị tốc độ chậm.

Cấu trúc hệ thống nhớ

- Bộ nhớ chính (Main memory: RAM) – phương tiện lưu trữ lớn duy nhất mà CPU có thể truy nhập trực tiếp.
  - Quá nhỏ để có thể lưu trữ ổn định tất cả các chương trình và dữ liệu cần thiết.
  - Có tính không ổn định ⇒ mất dữ liệu khi tắt nguồn.
- Bộ nhớ thứ cấp (Secondary storage) – là sự mở rộng của bộ nhớ chính, để cung cấp dung lượng bộ nhớ lớn và ổn định.

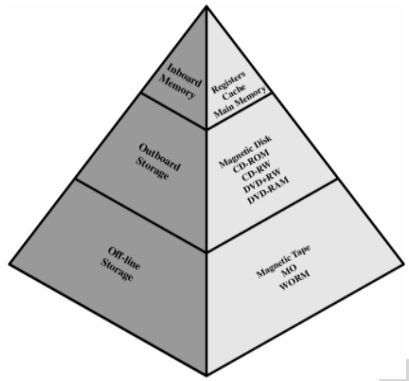
  Vd: Đĩa từ (Magnetic disks: đĩa cứng, đĩa mềm)
  - Cấu tạo bằng kim loại cứng hoặc các miếng kính được bao bọc bởi vật liệu nhiễm từ.
  - Bề mặt đĩa được chia (vật lý) thành các *tracks*, mỗi track được chia thành các *sectors*.
  - Mạch điều khiển đĩa (*disk controller)* xác định sự tương tác vật lý giữa thiết bị và máy tính.

Cơ cấu đĩa có đầu từ chuyển động

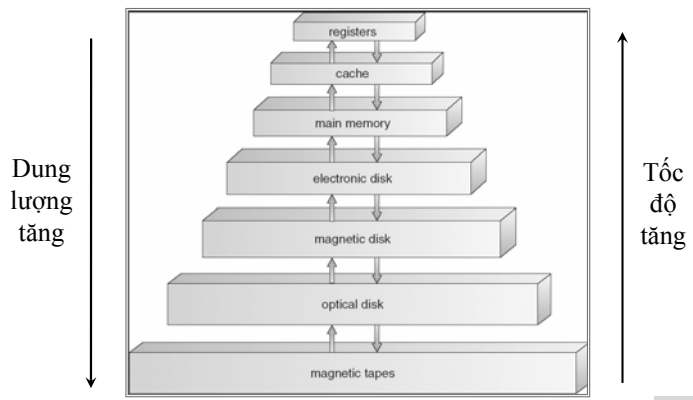


Sơ đồ phân cấp hệ thống nhớ

- Hệ thống lưu trữ được tổ chức dạng sơ đồ phân cấp dựa vào:
  - Tốc độ
  - Giá thành
  - Tính không ổn định



Sơ đồ phân cấp thiết bị nhớ



Caching

- *Caching* – nguyên lý quan trọng, được thực hiện tại nhiều mức trong 1 máy tính (trong phần cứng, HĐH, phần mềm)
- Là kỹ thuật làm tăng tốc độ xử lý của hệ thống bằng cách:
  - thực hiện copy thông tin đang sử dụng tới thiết bị nhớ nhanh hơn để tăng tốc độ xử lý của hệ thống.
  - Giữ lại các dữ liệu mới được truy nhập trong thiết bị tốc độ cao đó.
- Bộ nhớ nhanh hơn (cache) được kiểm tra trước tiên xem thông tin có ở đó không:
  - Nếu có, thông tin được sử dụng trực tiếp từ cache (nhanh)
  - Nếu không, dữ liệu được copy vào cache rồi được sử dụng ở đó
- Yêu cầu: dữ liệu phải được lưu trữ đồng bộ trong nhiều mức hệ thống nhớ để đảm bảo tính nhất quán (*consistent)*.
- Vì dung lượng cache có hạn, yêu cầu có sự quản lý cache (*cache management)* để tăng hiệu năng.

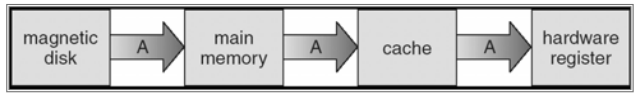
Sự thực thi tại nhiều mức bộ nhớ

- Sự di chuyển giữa các mức phân cấp bộ nhớ có thể là rõ ràng hoặc không.

Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape

Sự di trú của số nguyên A từ đĩa tới thanh ghi

- Các môi trường đa nhiệm phải cẩn thận để sử dụng được giá trị mới nhất, dù nó được chứa ở đâu trong phân cấp bộ nhớ



- Môi trường đa bộ vi xử lý phải cung cấp tính gắn kết cache trong phần cứng để tất cả các CPU có được giá trị mới nhất trong cache của nó.
- Môi trường phân tán còn phức tạp hơn
  - Có thể tồn tại nhiều bản copy của dữ liệu

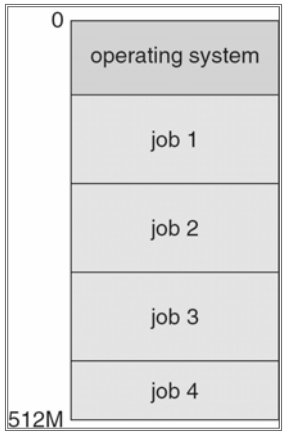
1.3. Cấu trúc hệ điều hành

- **Multiprogramming** (kỹ thuật đa chương trình) cần có để đạt hiệu quả
  - User đơn không thể giữ CPU và các thiết bị vào-ra hoạt động tại mọi thời điểm.
  - Multiprogramming tổ chức các công việc (job, gồm code và data) để CPU luôn có việc để thực hiện
  - Một tập con của tất cả các công việc trong hệ thống được giữ trong bộ nhớ
  - Một công việc được chọn và chạy thông qua **job scheduling**
  - Khi nó phải đợi (vd đợi vào-ra), HĐH chuyển sang một công việc khác.

Cấu trúc hệ điều hành (tiếp)

- **Timesharing (multitasking)** (kỹ thuật chia sẻ thời gian, đa nhiệm) là sự mở rộng logic mà trong đó CPU chuyển giữa các công việc rất thường xuyên để những user có thể tương tác với mỗi công việc trong khi nó đang chạy, tạo thành sự tính toán **tương tác**
  - **Thời gian đáp ứng** (Response time) nên < 1 giây
  - Mỗi user có ít nhất 1 chương trình đang thực hiện trong bộ nhớ ⇨ **process** (tiến trình)
  - Nếu một số công việc sẵn sàng chạy tại cùng thời điểm ⇨ **CPU scheduling** (lập lịch CPU)
  - Nếu các tiến trình không chứa vừa trong bộ nhớ, **swapping** (hoán đổi) chuyển chúng vào và ra để chạy
  - **Virtual memory** (bộ nhớ ảo) cho phép sự thực hiện các tiến trình không hoàn toàn trong bộ nhớ.

Bố trí bộ nhớ của HĐH đa chương trình

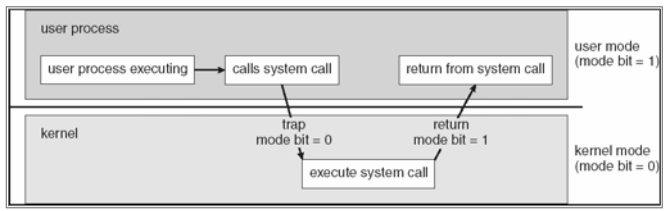


1.4. Hoạt động của hệ điều hành

- Ngắt do phần cứng
- Lỗi phần mềm hoặc yêu cầu tạo ra **exception** hay **trap**
  - Chia cho 0, yêu cầu dịch vụ của HĐH
- Các vấn đề tiến trình khác gồm: lặp vô hạn, các tiến trình thay đổi lẫn nhau hoặc thay đổi HĐH.
- Hoạt động **chế độ kép** (Dual-mode) cho phép HĐH bảo vệ chính nó và các thành phần hệ thống khác
  - **User mode** và **kernel mode**
  - **Mode bit** được cung cấp bởi phần cứng
    - Cung cấp khả năng phân biệt khi nào hệ thống chạy user code hay kernel code
    - Một số lệnh được thiết kế là **đặc quyền** (privileged), chỉ có thể thực hiện được trong kernel mode
    - **System call** thay đổi chế độ thành kernel mode, **return from call** thiết lập nó thành user mode.

Chuyển từ User sang Kernel Mode

- Định thời để ngăn lặp vô hạn / tiến trình lấy quá các tài nguyên
  - Thiết lập ngắt sau khoảng thời gian xác định
  - HĐH giảm bộ đếm
  - Khi bộ đếm bằng 0 thì sinh ra một ngắt
  - Thiết lập tiến trình lập lịch trước đó để giành lại được sự điều khiển hoặc chấm dứt chương trình vượt quá thời gian được cấp.



1.5. Sự quản lý tiến trình

- Tiến trình (process) là một chương trình đang được thực hiện. Nó là một đơn vị công việc trong hệ thống. Chương trình là một ***thực thể bị động***, tiến trình là một ***thực thể chủ động***.
- Tiến trình cần các tài nguyên để hoàn tất công việc:
  - CPU, bộ nhớ, các thiết bị vào-ra, các tệp (files)
  - Dữ liệu khởi tạo
- Sự chấm dứt tiến trình đòi hỏi sự giành lại bất kỳ tài nguyên nào có thể tái sử dụng.
- Tiến trình đơn luồng (thread) có một **program counter** xác định vị trí của lệnh kế tiếp để thực hiện
  - Tiến trình thực hiện các lệnh tuần tự, mỗi thời điểm một lệnh cho đến khi kết thúc.
- Tiến trình đa luồng: mỗi luồng có một program counter.
- Hệ thống thông thường có nhiều tiến trình, một số user, một số HĐH chạy đồng thời trên một hay nhiều CPU.
  - Đồng thời bằng sự đa nhiệm các CPU giữa các tiến trình / các luồng.



Các hoạt động quản lý tiến trình

HĐH chịu trách nhiệm đối với các hoạt động sau trong quản lý tiến trình

- Tạo và xoá các tiến trình của cả user và hệ thống.
- Tạm ngừng và tiếp tục lại các tiến trình.
- Cung cấp các cơ chế cho:
  - sự đồng bộ hoá tiến trình
  - sự giao tiếp tiến trình
  - sự xử lý bế tắc (deadlock)





1.6. Sự quản lý bộ nhớ (chính)

- Tất cả dữ liệu ở trong bộ nhớ trước và sau xử lý
- Tất cả các lệnh ở trong bộ nhớ để thực hiện
- Sự quản lý bộ nhớ xác định cái gì được ở trong bộ nhớ khi
  - Tối ưu hóa sự sử dụng CPU và sự đáp ứng máy tính với các user
- Các hoạt động quản lý bộ nhớ
  - Lưu lại dấu vết của các phần bộ nhớ đang được sử dụng và được sử dụng bởi tiến trình nào.
  - Quyết định xem những tiến trình (hoặc những phần của chúng) và dữ liệu nào được đưa vào và đưa ra khỏi bộ nhớ.
  - Phân phối và thu hồi bộ nhớ khi cần đến.





1.7. Sự quản lý bộ nhớ lưu trữ

- HĐH cung cấp cái nhìn logic, giống nhau đối với lưu trữ thông tin
  - Trừu tượng hóa các thuộc tính vật lý thành đơn vị lưu trữ logic - **file**
  - Mỗi phương tiện được điều khiển bởi thiết bị (nghĩa là: disk drive, tape drive)
    - Các thuộc tính khác nhau gồm: tốc độ truy nhập, dung lượng, tốc độ truyền dữ liệu, phương pháp truy nhập (tuần tự hoặc ngẫu nhiên)
- **A) Sự quản lý Hệ thống file**
  - Các file thường được tổ chức trong các thư mục
  - Kiểm soát truy nhập trên hầu hết các hệ thống để xác định ai có thể truy nhập cái gì
  - Các hoạt động của HĐH gồm:
    - Tạo và xóa các file và thư mục
    - Hỗ trợ từ gốc (primitive) đối với việc thao tác với các file và thư mục (read/write).
    - Ánh xạ các file vào bộ nhớ thứ cấp.
    - Sao dự phòng (Backup) file trên các phương tiện lưu trữ ổn định.





Sự quản lý bộ nhớ lưu trữ (tiếp)

B) Sự quản lý bộ nhớ lưu trữ lớn

- Thường sử dụng các đĩa để chứa dữ liệu không chứa vừa trong bộ nhớ chính hoặc dữ liệu cần được giữ lâu dài.
- Quản lý đúng cách đóng vai trò quan trọng trung tâm.
- Toàn bộ tốc độ thực hiện của máy tính xoay quanh hệ thống con đĩa và các giải thuật của nó.
- Các hoạt động của HĐH
  - Quản lý các vùng nhớ tự do
  - Phân phối bộ nhớ
  - Lập lịch đĩa (Disk scheduling)
- Một số bộ nhớ lưu trữ (storage) không cần phải nhanh
  - Bộ nhớ cấp ba gồm: bộ nhớ quang, băng từ
  - Vẫn cần được quản lý
  - Khác nhau giữa WORM (write-once, read-many-times) và RW (read-write)



C) Hệ thống con vào-ra (I/O Subsystem)

- Một trong những mục đích của HĐH là ẩn các tính chất khác thường của các thiết bị phần cứng không cho user thấy. Chức năng đó do hệ thống vào-ra đảm nhận.
- Hệ thống con vào-ra chịu trách nhiệm đối với:
  - Quản lý bộ nhớ của vào-ra gồm:
    - buffering (chứa dữ liệu tạm thời trong khi nó đang được truyền)
    - caching (chứa các phần của dữ liệu trong bộ nhớ nhanh hơn để tăng hiệu năng)
    - spooling (gối chồng output của 1 công việc với input của các công việc khác)
  - Giao diện device-driver chung
  - Các driver cho các thiết bị phần cứng riêng biệt.
- Chỉ device driver biết các tính chất đặc biệt của thiết bị mà nó điều khiển.

1.8. Protection và Security

- **Protection** – mọi cơ chế để kiểm soát sự truy nhập của các tiến trình hoặc user tới các tài nguyên được xác định bởi HĐH.
- **Security** – sự bảo vệ của hệ thống chống lại những sự tấn công từ bên trong và bên ngoài
  - Rất nhiều dạng, bao gồm denial-of-service, worms, viruses, identity theft, theft of service
- Các hệ thống đầu tiên thường phân biệt giữa các user để xác định ai có thể làm cái gì
  - User identifiers (**user IDs**, security IDs) gồm tên và số kèm theo, mỗi user một số.
  - User ID sau đó được liên kết với tất cả các file, các tiến trình của user đó để xác định kiểm soát truy nhập.
  - Group identifier (**group ID**) cho phép tập các user để được xác định và các kiểm soát được quản lý, sau đó cũng được liên kết với mỗi tiến trình, file.
  - **Privilege escalation** (sự leo thang đặc quyền) cho phép user thay đổi thành ID có nhiều quyền hơn.

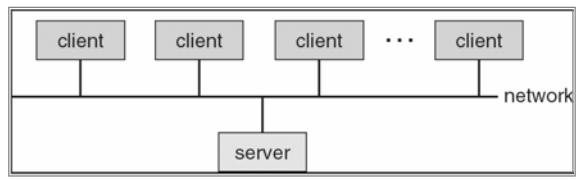
1.9. Các môi trường sử dụng máy tính

A) Máy tính truyền thống

- Đang mờ nhạt dần theo thời gian
- Môi trường văn phòng
  - Các PC được nối vào một mạng, các máy đầu cuối (terminal) được gắn vào mainframe hoặc minicomputers cung cấp xử lý theo lô và chia sẻ thời gian.
  - Ngày nay các cổng cho phép nối mạng và các hệ thống từ xa truy nhập tới cùng các tài nguyên.
- Các mạng gia đình
  - Đã thường là các hệ thống đơn, sử dụng các modem
  - Ngày nay được nối mạng, bảo vệ bằng firewall.

Các môi trường sử dụng máy tính (tiếp)

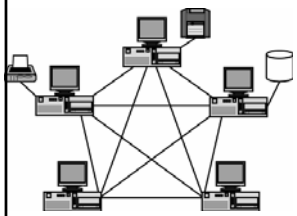
- **B) Client-Server Computing**
  - Các máy đầu cuối câm được thay thế bởi các PC thông minh
  - Giờ đây nhiều hệ thống là **server**, đáp ứng các yêu cầu từ các **client.**
    - **Compute-server** cung cấp một giao diện cho client yêu cầu các dịch vụ (nghĩa là. database)
    - **File-server** cung cấp các giao diện cho các client lưu trữ và lấy ra các file.



Các môi trường sử dụng máy tính (tiếp)

C) Peer-to-Peer Computing

- Một mô hình khác của hệ thống phân tán (distributed system)
- P2P không phân biệt các client và server
  - Tất cả các nút được coi là ngang nhau
  - Mỗi nút có thể hoạt động như client, server hoặc cả hai
  - Nút phải gia nhập mạng P2P



» Đăng ký dịch vụ của nó với dịch vụ tìm kiếm trung tâm trên mạng, hoặc

» Truyền quảng bá (broadcast) yêu cầu dịch vụ và đáp ứng các yêu cầu thông qua *giao thức khám phá* (*discovery protocol*)

❖ Các ví dụ gồm các mạng *Napster* và *Gnutella*

Các môi trường sử dụng máy tính (tiếp)

D) Web-Based Computing

- Web đã trở nên phổ biến khắp nơi
- Các PC đã là thiết bị phổ dụng
- Càng nhiều thiết bị có thể được nối mạng để cho phép truy nhập web.
- Loại thiết bị mới để quản lý truyền web trong các server tương tự nhau: **load balancers** (thiết bị cân bằng tải)
- Sự sử dụng các HĐH như Windows 95, client-side, đã tiến triển thành Linux và Windows XP, vừa có thể là client, vừa có thể là server.

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 2: Cấu trúc hệ điều hành

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung



Nội dung chương 2

- Các dịch vụ của HĐH
- Giao diện người sử dụng của HĐH
- Lời gọi HĐH (System Calls)
- Các chương trình hệ thống
- Thiết kế và thực thi HĐH
- Cấu trúc HĐH
- Virtual Machines
- Tạo ra HĐH (Operating System Generation)





Mục tiêu

- Mô tả các dịch vụ mà một HĐH cung cấp cho các user, tiến trình và các hệ thống khác.
- Thảo luận các cách xây dựng HĐH khác nhau.
- Giải thích các HĐH được cài đặt và khởi động như thế nào.





2.1. Các dịch vụ của hệ điều hành

Một tập các dịch vụ của HĐH cung cấp các chức năng hữu dụng với các user:

- **Giao diện người sử dụng** (user interface, UI) – hầu như tất cả các HĐH có một giao diện người sử dụng
  - Khác nhau giữa giao diện dòng lệnh (Command-Line, CLI), giao diện đồ họa (Graphics User Interface, GUI), Batch
- **Thực hiện chương trình** (Program execution) – khả năng của hệ thống để nạp một chương trình vào bộ nhớ và chạy nó, dừng chương trình (bình thường hoặc bất thường).
- **Thực hiện vào-ra** (I/O operations) – vì chương trình của người sử dụng không thể thực hiện trực tiếp các hoạt động vào/ra, HĐH phải cung cấp một số phương pháp để thực hiện vào/ra (1 file hoặc 1 thiết bị vào/ra).
- **Thao tác với hệ thống file** (File-system manipulation) – vì các chương trình cần đọc, ghi, tạo, xoá, tìm kiếm, liệt kê thông tin, quản lý quyền với các file và thư mục.





Các dịch vụ của hệ điều hành (tiếp)

- **Giao tiếp** (Communications) – trao đổi thông tin giữa các tiến trình đang thực hiện trên cùng 1 máy tính hoặc trên các máy tính khác nhau được nối mạng.
  - Giao tiếp có thể thông qua bộ nhớ chia sẻ (*shared memory)* hoặc chuyển thông điệp (*message passing)*: các gói tin được chuyển bởi HĐH.
- **Phát hiện lỗi** (Error detection) – HĐH cần phải thường xuyên quan tâm đến các lỗi có thể xảy ra
  - Có thể lỗi trong CPU và bộ nhớ, trong các thiết bị vào-ra, hoặc trong chương trình của người sử dụng.
  - Với mỗi loại lỗi, HĐH cần có hành động thích hợp để đảm bảo sự tính toán phù hợp và đúng đắn.
  - Tính năng sửa lỗi có thể tăng đáng kể khả năng sử dụng hiệu quả HĐH của người sử dụng và các lập trình viên.

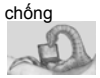




Các dịch vụ của hệ điều hành (tiếp)

Có một tập chức năng khác của HĐH để đảm bảo sự hoạt động hiệu quả của chính nó thông qua chia sẻ tài nguyên:

- **Resource allocation** – các tài nguyên phải được phân phối cho mỗi user/tiến trình khi chúng chạy đồng thời
  - Có nhiều loại tài nguyên: một số (như các chu kỳ CPU, bộ nhớ chính, file) có thể có mã phân phối đặc biệt, số khác (như các thiết bị vào/ra) có thể có mã yêu cầu và giải phóng chung.
- **Accounting** – theo dõi và ghi lại loại tài nguyên và lượng sử dụng (tài nguyên) của user nhằm mục đích thống kê.
- **Protection & Security** – người chủ thông tin trên một hệ thống máy tính nhiều người sử dụng hoặc nối mạng có thể muốn kiểm soát sự sử dụng thông tin đó, các tiến trình đồng thời không nên can thiệp lẫn nhau
  - Protection gồm sự đảm bảo rằng tất cả sự truy nhập đến các tài nguyên hệ thống được kiểm soát.
  - Security của hệ thống từ bên ngoài yêu cầu thẩm định người sử dụng, chống các thiết bị vào/ra bên ngoài (modem, NIC) cố truy nhập không hợp lệ.





2.2. Giao diện người sử dụng của HĐH

a) Command-Line Interpreter (CLI)

CLI cho phép nhập lệnh trực tiếp để HĐH thực hiện

- Đôi khi CLI được thực thi trong kernel, đôi khi bởi chương trình hệ thống
- Đôi khi hệ thống có nhiều CLI – **shells**, vd: UNIX, LINUX
- Chức năng chính là tìm nạp 1 lệnh từ người sử dụng rồi thực hiện nó
  - Đôi khi các lệnh là built-in,
  - Đôi khi chỉ là tên của các chương trình (vd UNIX):
    - » Vd lệnh **rm file.txt**
    - » Nếu sau cần thêm lệnh mới thì không cần sửa đổi shell





Giao diện người sử dụng của HĐH (tiếp)

b) Graphical User Interface (GUI)

- Giao diện **desktop** thân thiện
  - Thường dùng chuột, bàn phím, màn hình
  - **Icons** đại diện cho các file, chương trình, hành động,…
  - Các nút chuột khác nhau trên các đối tượng gây các hành động khác nhau (cung cấp thông tin, lựa chọn, thực hiện chức năng, mở thư mục)
  - Được phát minh tại Xerox PARC những năm 1970
- Hiện nay nhiều HĐH bao gồm cả giao diện CLI và GUI
  - Microsoft Windows có GUI với CLI "command" shell
  - Apple Mac OS X có giao diện GUI là "Aqua" với UNIX kernel bên dưới và có các shell.
  - Solaris là CLI với các giao diện GUI tùy chọn (Java Desktop, KDE)



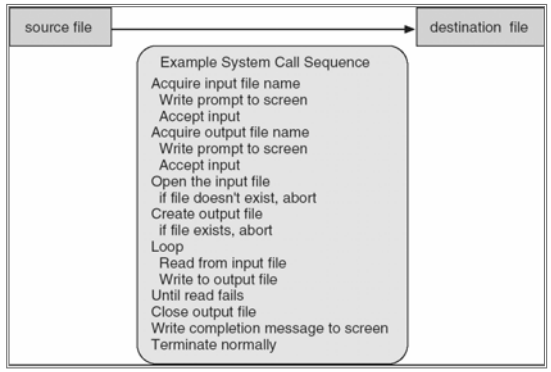
2.3. System Calls

- Cung cấp giao diện lập trình cho các dịch vụ của HĐH.
- Thường được viết bằng một ngôn ngữ bậc cao (C, C++)
- Hầu hết được truy nhập bởi các chương trình thông qua một giao diện lập trình ứng dụng (**Application Program Interface - API)** bậc cao, ít khi sử dụng trực tiếp system call.
- Ba API phổ biến nhất là Win32 API cho Windows, POSIX API cho các hệ thống trên nền POSIX (gồm hầu hết các phiên bản của UNIX, Linux, Mac OS X), và Java API cho Java virtual machine (JVM)
- Tại sao lại sử dụng các API thay vì các system call?

  (Chú ý: tên của các system-call được sử dụng ở đây là tổng quát)

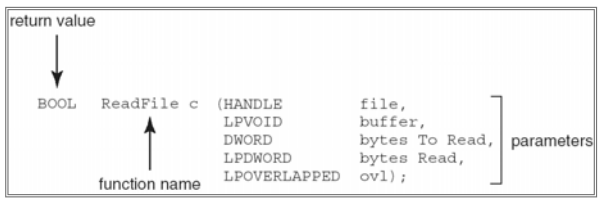
Ví dụ các System Call

- Chuỗi system call để copy nội dung của 1 file tới file khác



Ví dụ API chuẩn

- Xét hàm ReadFile() trong Win32 API – hàm đọc 1 file

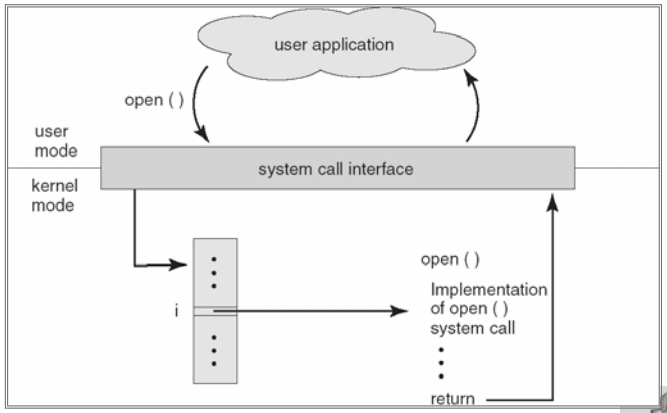


- Các tham số được truyền cho ReadFile()
  - HANDLE file – file cần đọc
  - LPVOID buffer – buffer để dữ liệu được đọc vào và ghi ra
  - DWORD bytesToRead – số byte được đọc vào buffer
  - LPDWORD bytesRead – số byte đọc được trong lần đọc trước
  - LPOVERLAPPED ovl – chỉ ra nếu sử dụng vào/ra kiểu gối chồng.

Thực thi system call

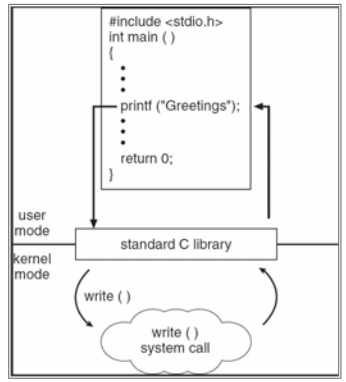
- Thường thì một số được gắn với mỗi system call
  - Giao diện system-call duy trì một bảng được đánh chỉ số theo những số này.
- Giao diện system call gọi system call mong muốn trong kernel HĐH và trả về trạng thái của nó và các giá trị trả về nào đó.
- Người gọi không cần biết gì về system call được thực thi như thế nào
  - Chỉ cần tuân thủ API và hiểu HĐH sẽ làm ra kết quả gì
  - Hầu hết giao diện HĐH ẩn đối với lập trình viên bởi API
    - Được quản lý bởi thư viện hỗ trợ tại giai đoạn chạy (tập các hàm được xây dựng vào các thư viện cùng với trình biên dịch)

Mối quan hệ API – System Call – HĐH



Ví dụ thư viện C chuẩn

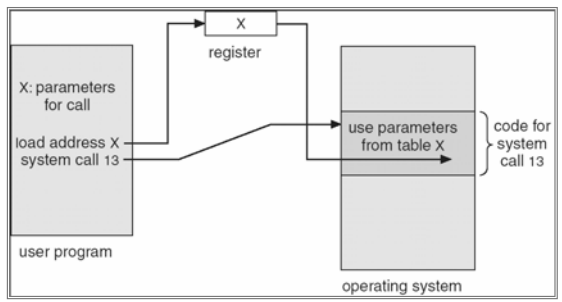
- Chương trình C gọi lời gọi thư viện printf(), mà gọi tới system call write()



Truyền tham số cho System Call

- Thường cần nhiều thông tin hơn là đơn giản chỉ xác định system call mong muốn
  - Kiểu và lượng thông tin chính xác thay đổi theo HĐH và theo lời gọi
- Ba phương thức tổng quát được sử dụng để *truyền tham số* cho HĐH.
  1. Đơn giản nhất: Truyền tham số trong các thanh ghi.
     - Trong một số trường hợp: số tham số nhiều hơn số thanh ghi
  2. Tham số được chứa trong một bảng trong bộ nhớ, và địa chỉ của bảng được truyền như một tham số trong một thanh ghi.
     - Phương pháp này được sử dụng bởi Linux và Solaris
  3. Đẩy *(push,* store) các tham số vào *stack* bằng chương trình, và lấy ra khỏi stack (*pop)* bởi HĐH.
  - Các phương pháp dùng bảng và stack không giới hạn số lượng hay độ dài của các tham số được truyền.

Truyền tham số thông qua Bảng



Các loại System Calls

- Điều khiển tiến trình (Process control)
  - kết thúc, bỏ dở (abort)
  - nạp, thực hiện
  - tạo, chấm dứt tiến trình
  - lấy, thiết lập các thuộc tính của tiến trình
  - chờ đợi
  - đợi sự kiện, báo hiệu sự kiện
  - phân phối và giải phóng bộ nhớ

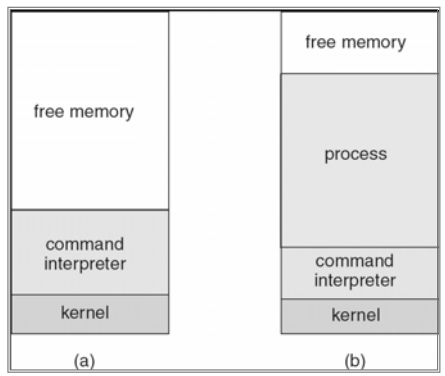
Các loại System Calls (tiếp)

- Quản lý file (File management)
  - tạo file, xóa file
  - mở, đóng
  - đọc, ghi, định vị
  - lấy/ thiết lập thuộc tính file
- Quản lý thiết bị (Device management)
  - yêu cầu thiết bị, giải phóng thiết bị
  - đọc, ghi, định vị
  - lấy/ thiết lập các thuộc tính thiết bị
  - gắn kết (attack), tháo gỡ (detach) logic các thiết bị

Các loại System Calls (tiếp)

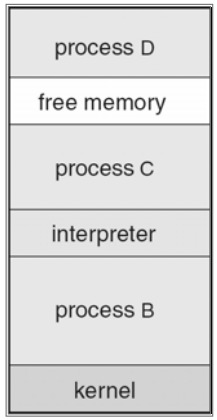
- Duy trì thông tin (Information maintenance)
  - lấy/ thiết lập giờ hoặc ngày
  - lấy/ thiết lập dữ liệu hệ thống
  - lấy/ thiết lập thuộc tính của tiến trình, file, thiết bị
- Giao tiếp (Communications)
  - tạo, xóa kết nối giao tiếp
  - gửi, nhận thông điệp
  - truyền thông tin trạng thái
  - gắn kết, tháo gỡ logic các thiết bị ở xa (remote device)

MS-DOS



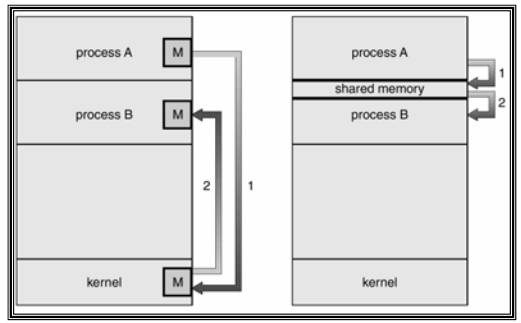
At System Start-up Running a Program

UNIX



Các phương thức giao tiếp

- Sự giao tiếp có thể thực hiện bằng cách sử dụng phương thức message passing hoặc shared memory.



Message Passing Shared Memory

2.4. Các chương trình hệ thống

- Các chương trình hệ thống cung cấp một môi trường thuận tiện cho việc thực hiện và phát triển chương trình. Chúng có thể được phân loại thành:
  - Thao tác với file: tạo, xóa, copy, đổi tên… các file và thư mục
  - Thông tin trạng thái: ngày giờ, dung lượng bộ nhớ trống, số user…
  - Sửa đổi file: trình soạn thảo văn bản có thể tạo, sửa nội dung file trên đĩa
  - Hỗ trợ ngôn ngữ lập trình: trình biên dịch, trình thông dịch, trình gỡ lỗi…
  - Nạp và thực hiện chương trình: nạp CT đã được biên dịch vào bộ nhớ để thực hiện
  - Giao tiếp: cung cấp cơ chế tạo kết nối ảo giữa các tiến trình, các user, các máy tính để gửi message, duyệt web, gửi email, truyền file…
- Hầu hết cách nhìn nhận của người sử dụng về HĐH được xác định bởi các chương trình hệ thống, không thực sự bởi các system call.

2.5. Thiết kế và thực thi HĐH

- Thiết kế và thực thi HĐH không có giải pháp hoàn hảo, nhưng một số phương pháp đã chứng minh thành công
- Cấu trúc bên trong của các HĐH khác nhau có thể rất khác nhau
- Bắt đầu từ việc xác định các mục tiêu và đặc điểm
- Bị tác động bởi sự lựa chọn phần cứng, loại HĐH: chia sẻ thời gian, đơn người dùng, đa người dùng, phân tán, thời gian thực…
- *User* goals và *System* goals
  - User goals – HĐH cần dễ sử dụng, dễ học, đáng tin cậy, an toàn, nhanh.
  - System goals – HĐH cần dễ thiết kế, thực thi và duy trì, cũng như linh hoạt, đáng tin cậy, không có lỗi, hiệu quả.

Thiết kế và thực thi HĐH (tiếp)

- Nguyên lý quan trọng là sự tách biệt:

  Policy (chính sách): *Cái gì* sẽ được làm? - What
  Mechanism (cơ chế): Làm nó *như thế nào*? - How

  - Sự tách biệt chính sách với cơ chế cho phép sự linh hoạt tối đa nếu sau này các quyết định chính sách được thay đổi
  - VD: Các HĐH vi nhân (như UNIX, Solaris) tách biệt cơ chế và chính sách bằng cách thực thi một tập cơ bản các khối tạo dựng ban đầu, hầu như độc lập với chính sách; cho phép các cơ chế và chính sách tiên tiến hơn có thể được thêm vào thông qua các môđun kernel do người sử dụng tạo hoặc do chính chương trình của người sử dụng. Trong phiên bản mới nhất của Solaris, tùy vào bảng nào được nạp, hệ thống có thể là chia sẻ thời gian, xử lý theo lô, thời gian thực, chia sẻ công bằng, hay dạng kết hợp bất kỳ.

Thực thi hệ thống

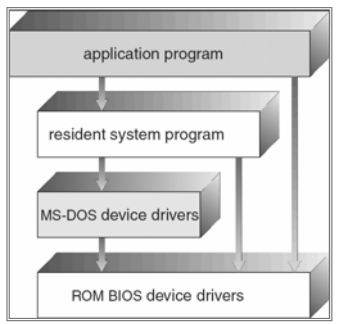
- Trước đây, HĐH được viết bằng ngôn ngữ assembly, hiện nay có thể viết bằng những ngôn ngữ bậc cao (UNIX, PS/2, Windows NT chủ yếu viết bằng C).
- Mã được viết bằng ngôn ngữ bậc cao:
  - có thể viết nhanh hơn.
  - cô đọng hơn.
  - dễ hiểu và dễ gỡ rối.
- Một HĐH được viết bằng một ngôn ngữ bậc cao sẽ dễ dàng hơn khi chuyển sang phần cứng mới.

2.6. Cấu trúc hệ điều hành

a) Cấu trúc đơn giản

Hệ điều hành MS-DOS

- MS-DOS – được viết để cung cấp hầu hết các chức năng trong một không gian nhỏ nhất
  - Không chia thành các module
  - Dù MS-DOS có một vài cấu trúc, giao diện của nó và các mức chức năng không được phân định rõ ràng.

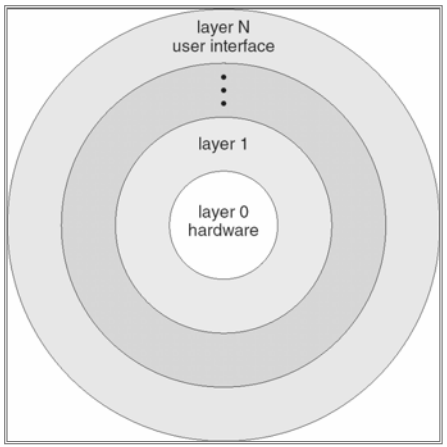


Cấu trúc lớp của MS-DOS

b) Phương pháp phân lớp

- HĐH được chia thành các lớp (layer, level), mỗi lớp được xây dựng trên đỉnh của các lớp thấp hơn. Lớp ở đáy (layer 0) là phần cứng; lớp cao nhất (layer N) là user interface.
- Bằng cách chia thành các module như trên, mỗi lớp chỉ sử dụng các chức năng và dịch vụ của các lớp dưới.

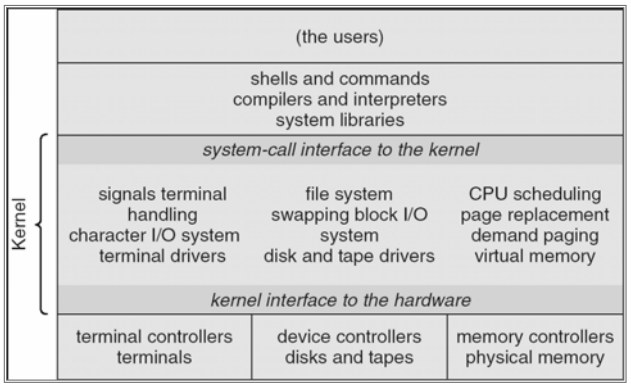
HĐH được phân lớp



Cấu trúc HĐH UNIX

- UNIX – là HĐH khác mà ban đầu đã bị hạn chế bởi chức năng phần cứng.
- HĐH UNIX bao gồm 2 phần riêng biệt:
  - Systems programs – các chương trình hệ thống
  - The kernel - nhân
    - Bao gồm tất cả các lớp nằm dưới giao diện system-call và nằm trên physical hardware
    - Cung cấp hệ thống file, lập lịch CPU, quản lý bộ nhớ và các chức năng HĐH khác; rất nhiều chức năng cho 1 mức.

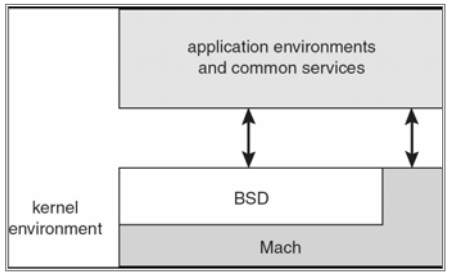
Cấu trúc lớp HĐH UNIX (tiếp)



c) Cấu trúc vi nhân (Microkernel)

- Vd: Windows NT
- Chuyển rất nhiều thành phần không thiết yếu từ kernel vào trong user space ⇒ microkernel
- Sự giao tiếp diễn ra giữa các module của người sử dụng bằng phương thức message passing.
- Các lợi điểm:
  - dễ dàng mở rộng hệ điều hành mà không phải thay đổi kernel
  - dễ dàng mang một HĐH đặt vào những kiến trúc khác
  - đáng tin cậy hơn (ít mã lệnh chạy trong kernel mode)
  - an toàn hơn (ít thứ phải bảo vệ hơn)
- Nhược điểm: có thể làm giảm hiệu năng vì quá tải giao tiếp từ user space tới kernel space.

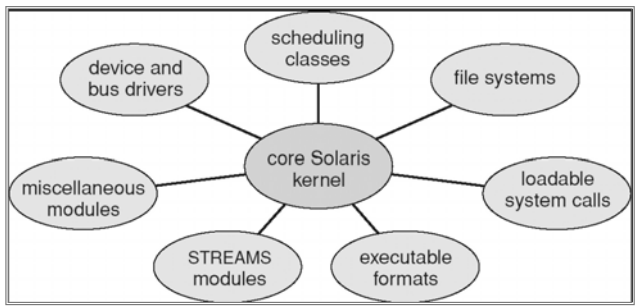
Cấu trúc vi nhân lai của Mac OS X



d) Modules

- Hầu hết các HĐH hiện đại thực thi các kernel module:
  - Sử dụng phương pháp hướng đối tượng
  - Mỗi thành phần hạt nhân là tách biệt
  - Mỗi thành phần giao tiếp với các thành phần khác qua giao diện đã định trước
  - Mỗi thành phần là có thể nạp vào trong kernel khi cần thiết
- Tổng quát: tương tự như các lớp nhưng phức tạp hơn.

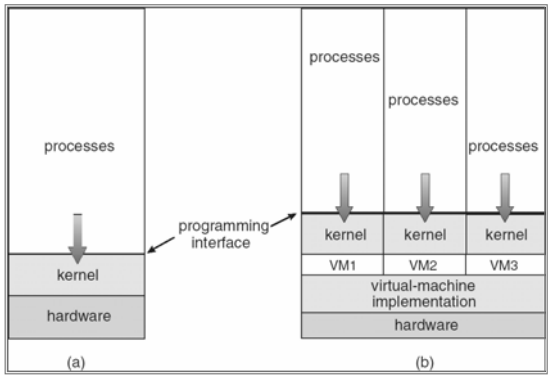
Cấu trúc môđun của HĐH Solaris



2.7. Virtual Machines

- Một *máy ảo* (*virtual machine)* là một chương trình giả lập phần cứng (hardware simulator). Chạy N bản copy của chương trình giả lập này, một máy vật lý trở thành N máy ảo.
- Mỗi máy ảo có thể chạy:
  - một tiến trình đơn dưới một HĐH đơn
  - tất cả tiến trình của một user dưới một HĐH
  - một HĐH chia sẻ thời gian phức tạp (vd: để gỡ rối)
- "HĐH" (Virtual Machine) có 3 phần:
  - Trình giả lập phần cứng - hardware simulator,
  - Tài nguyên (processor, memory) chia sẻ giữa các trình giả lập,
  - HĐH chạy trong mỗi trình giả lập.

Các mô hình hệ thống VM và non-VM



Non-virtual Machine

Virtual Machine

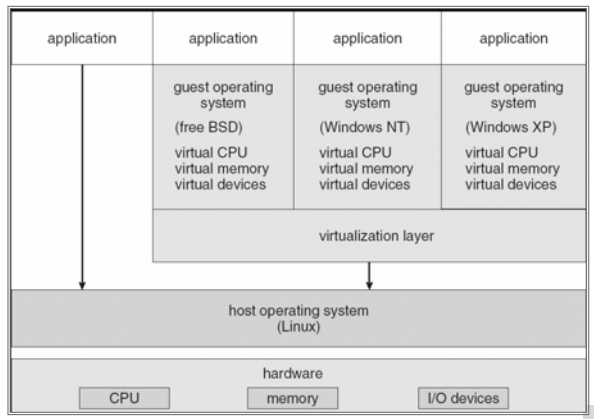
Các lợi điểm của Virtual Machine

- Virtual-machine cung cấp sự bảo vệ hoàn toàn các tài nguyên hệ thống vì mỗi máy ảo được tách biệt với các máy ảo khác.
- Rất lý tưởng cho việc nghiên cứu và phát triển các HĐH. Sự phát triển hệ thống ảo không phá vỡ sự hoạt động của hệ thống thật.

Các bất lợi của Virtual Machine

- Không chia sẻ trực tiếp các tài nguyên.
- Khó thực hiện. Để tăng hiệu quả, chúng ta chạy lệnh không đặc quyền trên phần cứng. Nhưng điều gì xảy ra nếu…
  - Chương trình người sử dụng trong trình giả lập tạo system call trong chế độ thực (*real user* mode), gây ra ngắt thực
  - HĐH thực ở trong chế độ *real kernel*, thiết lập trình giả lập thành chế độ *simulated kernel*, khởi động lại sự thực hiện trình giả lập trong chế độ *real user*
  - Trình giả lập chạy lệnh đặc quyền (vd: I/O) trong chế độ *real user*, gây ra *real* trap
  - HĐH thực giả lập I/O trong chế độ *real kernel*, khởi động lại trình giả lập trong chế độ *simulated kernel* và *real user*.
- Nếu điều này quá khó hiểu, hãy tưởng tượng xem việc viết code xử lý nó đúng thì khó khăn như thế nào…

Kiến trúc VMware





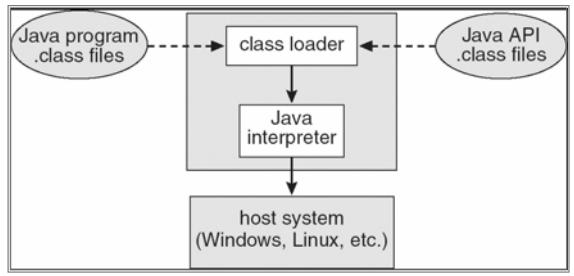
Java Virtual Machine

- Các chương trình Java sau khi biên dịch thành các tệp bytecode có tính trung lập nền (platform-neutral bytecode, có tên mở rộng .class), và được thực hiện bởi Java Virtual Machine (JVM).
- JVM bao gồm:
  - trình nạp lớp (class loader)
  - trình xác định lớp (class verifier)
  - trình thông dịch thời gian chạy (runtime interpreter)
- Trình thông dịch Java có thể là:
  - môđun phần mềm thông dịch các bytecode chỉ 1 lần.
  - Just-In-Time (JIT) compiler chuyển các bytecode thành ngôn ngữ máy tự nhiên → làm tăng hiệu năng.





Java Virtual Machine (tiếp)







2.8. System Generation (SYSGEN)

- Các HĐH được thiết kế để chạy trên bất kỳ loại máy nào; sau đó hệ thống phải được cấu hình cho mỗi máy tính cụ thể. Tiến trình đó được gọi là System generation.
- HĐH thường được phân phối trên các đĩa CD. Để tạo ra 1 HĐH, chúng ta sử dụng 1 chương trình đặc biệt - SYSGEN.
- Chương trình SYSGEN xác định thông tin liên quan đến cấu hình riêng của hệ thống phần cứng từ 1 file hoặc yêu cầu người sử dụng cung cấp:
  - Sử dụng CPU nào? Dung lượng bộ nhớ khả dụng?
  - Thông tin về các thiết bị khả dụng?
  - Các lựa chọn HĐH nào được yêu cầu? Những giá trị tham số nào được sử dụng?





SYSGEN (tiếp)

- Các thông tin trên sau đó thường được System generation sử dụng để tạo các bảng thích hợp mô tả hệ thống và để sinh ra hệ thống.
- Sau khi hệ thống được sinh ra, nó phải được phần cứng sử dụng → làm sao để phần cứng biết nơi chứa nhân HĐH (kernel), nạp như thế nào?
- *Booting* – quá trình khởi động máy tính bằng cách nạp nhân .
- *Bootstrap program* – đoạn mã được chứa trong ROM của hầu hết các hệ thống máy tính để có thể xác định vị trí của nhân, nạp nó vào bộ nhớ, và bắt đầu sự thực hiện của nó.



BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 3: Tiến trình (Processes)

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp HN
Website: fita.hua.edu.vn/pqdung

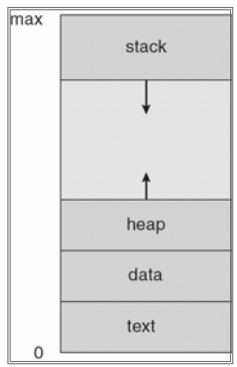
Nội dung chương 3

- Khái niệm tiến trình
- Lập lịch tiến trình
- Các hoạt động trên tiến trình
- Các tiến trình hợp tác (Cooperating Processes)
- Giao tiếp liên tiến trình (Interprocess Communication)

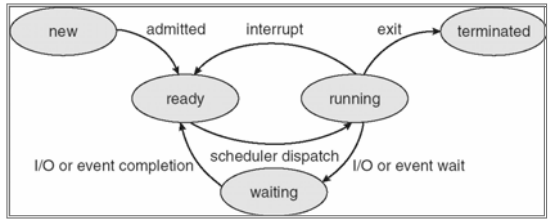
3.1. Khái niệm tiến trình (process)

- Một HĐH thực hiện nhiều loại chương trình khác nhau:
  - Batch system: thực hiện các *job*
  - Time-shared system: thực hiện *user programs* hoặc *tasks*
- Các thuật ngữ *job* và *process* là tương tự nhau.
- Process – một chương trình đang được thực hiện; sự thực hiện tiến trình phải tiến triển theo kiểu tuần tự.
- Một tiến trình (process) bao gồm:
  - program counter - bộ đếm chương trình
  - stack - ngăn xếp
  - data section - đoạn dữ liệu

Tiến trình trong bộ nhớ



Các trạng thái tiến trình

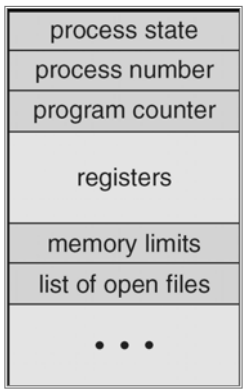


- Khi một tiến trình thực hiện, nó có thể thay đổi *trạng thái (state)*
  - **new**: Tiến trình đang được khởi tạo.
  - **running**: Tiến trình ở trong CPU. Các lệnh đang được thực hiện.
  - **waiting**: Tiến trình đang chờ sự kiện nào đó xuất hiện.
  - **ready**: Tiến trình đang chờ đến lượt được thực hiện bởi CPU.
  - **terminated**: Tiến trình kết thúc. Nó không biến mất cho đến khi một tiến trình khác đọc được trạng thái thoát của nó.

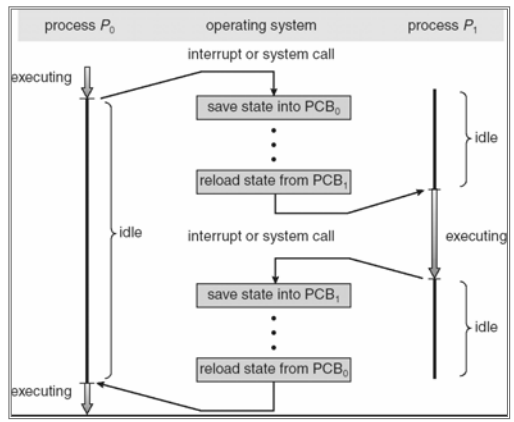
Khối điều khiển tiến trình Process Control Block (PCB)

- Mỗi tiến trình được biểu diễn trong HĐH bởi một PCB. Mỗi PCB chứa các thông tin được gắn với mỗi tiến trình:
  - Trạng thái tiến trình
  - Bộ đếm chương trình
  - Các thanh ghi của CPU
  - Thông tin lịch trình CPU
  - Thông tin quản lý bộ nhớ
  - Thông tin sử dụng CPU, thời gian, các số hiệu tiến trình…
  - Thông tin trạng thái vào/ra

Process Control Block (PCB)



CPU chuyển giữa các tiến trình



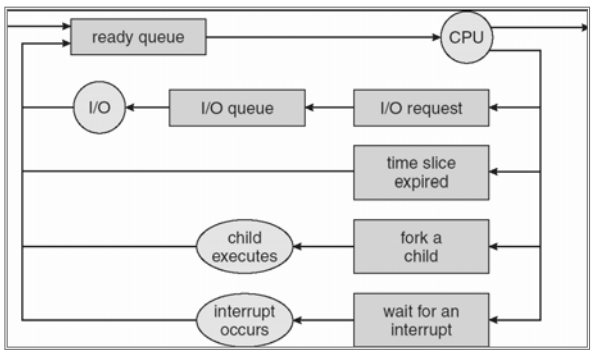
3.2. Lập lịch tiến trình (process scheduling)

- Mục tiêu của multiprogramming là có nhiều tiến trình cùng chạy tại mọi thời điểm để tối đa hóa sử dụng CPU.
- Mục tiêu của time-sharing là chuyển CPU giữa các tiến trình càng thường xuyên càng tốt để người sử dụng có thể tương tác với mỗi chương trình khi nó đang chạy.
- Một HĐH đơn processor chỉ có thể chạy 1 tiến trình.
- Nếu có nhiều tiến trình tồn tại, chúng phải đợi đến khi CPU rỗi và được lập lịch lại.

Các queue lập lịch tiến trình

- Job queue – tập hợp tất cả các tiến trình trong hệ thống.
- Ready queue – tập hợp tất cả các tiến trình cư trú trong bộ nhớ chính, đã sẵn sàng và chờ được thực hiện.
  - FIFO queue
  - Priority queue
  - Tree
  - Danh sách liên kết
- Device queues – tập hợp các tiến trình đang chờ một thiết bị vào/ra.
- Tiến trình có thể di trú giữa các queue khác nhau.

Sơ đồ lập lịch tiến trình

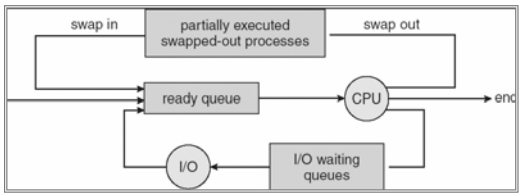


Các trình lập lịch - Schedulers

- Long-term scheduler (trình lập lịch dài kỳ)
  - còn được gọi là job scheduler.
  - lựa chọn những tiến trình nào nên được đưa từ đĩa vào trong ready queue.
  - được sử dụng đến rất không thường xuyên (seconds, minutes) ⇒ may be slow.
  - kiểm soát *mức đa chương trình* (*degree of multiprogramming),* vd: số tiến trình.
- Short-term scheduler (trình lập lịch ngắn kỳ)
  - còn được gọi là CPU scheduler.
  - lựa chọn tiến trình nào nên được thực hiện kế tiếp và phân phối CPU cho nó.
  - được sử dụng đến rất thường xuyên (milliseconds) ⇒ must be fast.

Các trình lập lịch (tiếp)

- Một số HĐH, như HĐH chia sẻ thời gian, có thể có thêm trình lập lịch trung kỳ (medium-term scheduler).
- Tư tưởng là thực hiện swapping (kỹ thuật hoán chuyển):
  - Đưa tiến trình ra khỏi bộ nhớ khi cần thiết
    - Khi cân đối giữa các tiến trình tính toán nhiều và tiến trình vào/ra nhiều
    - Khi cần giải phóng bộ nhớ cho việc khác
  - Sau đó đưa tiến trình trở lại bộ nhớ để thực hiện tiếp



Chuyển ngữ cảnh - Context Switch

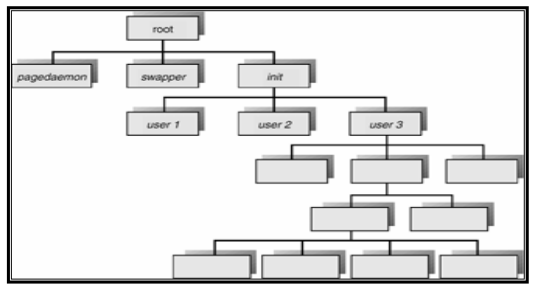
- Các tiến trình có thể được mô tả là:
  - *I/O-bound process* – sử dụng nhiều thời gian thực hiện vào/ra hơn việc tính toán, nhiều lần chiếm dụng CPU ngắn. Cần chuyển ngữ cảnh thường xuyên tại thời điểm bắt đầu và kết thúc I/O.
  - *CPU-bound process* – sử dụng nhiều thời gian cho việc tính toán hơn; ít lần chiếm dụng CPU dài. Cũng cần chuyển ngữ cảnh thường xuyên để tránh tr.hợp một tiến trình ngăn chặn các tiến trình khác sử dụng CPU.
- Khi CPU chuyển tới một tiến trình khác, hệ thống phải lưu trạng thái của tiến trình trước và nạp trạng thái đã lưu cho tiến trình mới.
- Thời gian chuyển ngữ cảnh phụ thuộc vào sự hỗ trợ phần cứng.
- Hệ thống thực hiện công việc vô ích trong khi chuyển (ngữ cảnh).

3.3. Các hoạt động trên tiến trình

- Các tiến trình trong hệ thống có thể thực hiện đồng thời, và chúng phải được tạo (create) và xóa (delete) một cách tự động.
- Do đó HĐH phải cung cấp kỹ thuật tạo và xóa tiến trình.

a) Sự tạo tiến trình - Process Creation

- Tiến trình cha (parent process) tạo ra các tiến trình con (children processes), chúng lần lượt tạo ra các tiến trình con khác tạo thành cây tiến trình (tree of processes).



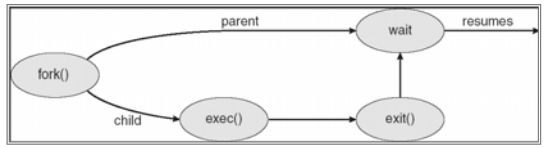
- Tạo tiến trình là một công việc "nặng nhọc" vì phải phân phối bộ nhớ và tài nguyên.

Sự tạo tiến trình (tiếp)

- Các lựa chọn chia sẻ tài nguyên (resource sharing)
  - Tiến trình cha và con chia sẻ tất cả tất cả các tài nguyên.
  - Tiến trình con chia sẻ tập con các tài nguyên của tiến trình cha.
  - Tiến trình cha và con không có sự chia sẻ tài nguyên.
- Không gian địa chỉ (Address space)
  - Tiến trình con sao chép tiến trình cha.
  - Tiến trình con có một chương trình được nạp vào trong nó.
- Sự thực hiện (execution)
  - Tiến trình cha và con thực hiện đồng thời.
  - Tiến trình cha đợi cho đến khi tiến trình con kết thúc.
- UNIX examples

Các tiến trình trong UNIX

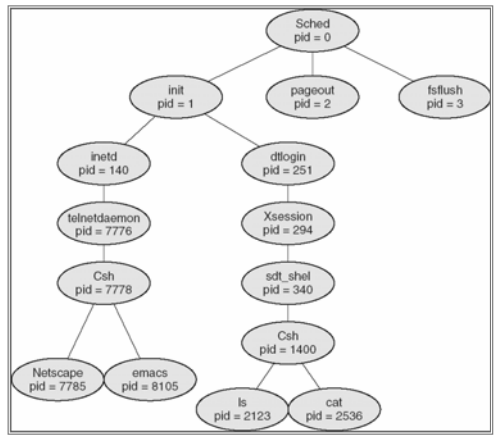
- **fork** - lệnh hệ thống tạo một tiến trình mới.
- **exec** - lệnh hệ thống được sử dụng sau lệnh fork để thay thế không gian bộ nhớ của tiến trình bởi một chương trình mới.



C Program Forking Separate Process

```
int main()
{
pid_t  pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait (NULL);
        printf ("Child Complete");
        exit(0);
    }
}
```


A tree of processes on a typical Solaris





b) Sự kết thúc tiến trình

- Tiến trình thực hiện câu lệnh cuối cùng và yêu cầu HĐH tự kết thúc (**exit**).
  - Dữ liệu ra từ tiến trình con đến tiến trình cha (qua lệnh **wait**).
  - Các tài nguyên của tiến trình được HĐH phân phối lại.
- Tiến trình cha có thể chấm dứt việc thực hiện tiến trình con (**abort**).
  - Tiến trình con dùng quá tài nguyên được phân phối.
  - Nhiệm vụ mà tiến trình con thực hiện không còn cần thiết.
  - Tiến trình cha đang kết thúc. HĐH có thể lựa chọn:
    - Dừng tiến trình con. Xếp tầng sự chấm dứt (Cascading termination): tiến trình cháu cũng bị dừng,…
    - Tiến trình cháu tồn tại do được tiến trình ông chấp nhận.





3.4. Các tiến trình hợp tác

- Tiến trình *độc lập (Independent* process):
  - không thể tác động hay chịu tác động bởi sự thực hiện của tiến trình khác.
- Tiến trình *hợp tác (Cooperating* process):
  - có thể tác động hoặc chịu tác động bởi sự thực hiện của tiến trình khác.
  - vd: tiến trình này chia sẻ dữ liệu với tiến trình khác.





Các tiến trình hợp tác (tiếp)

- Các lợi điểm của tiến trình hợp tác
  - Chia sẻ thông tin - Information sharing
  - Tăng tốc độ tính toán - Computation speed-up
  - Mô-đun hóa - Modularity
  - Sự tiện lợi - Convenience (vd người sử dụng cùng thực hiện soạn thảo, in ấn, biên dịch song song)
- Mô hình các tiến trình hợp tác: tiến trình sản xuất (*producer* process) tạo ra các thông tin để tiến trình tiêu thụ (*consumer* process) sử dụng.
  - *unbounded-buffer* : giả thiết kích thước buffer vô hạn.
  - *bounded-buffer* : thừa nhận có một kích thước buffer cố định.



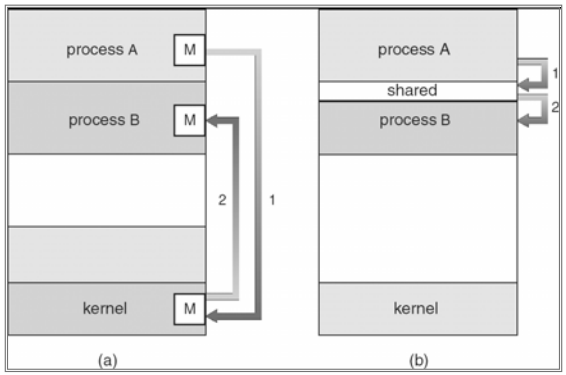


3.5. Giao tiếp liên tiến trình Interprocess Communication (IPC)

- Là cơ chế để các tiến trình giao tiếp và để đồng bộ các hành động của chúng mà không phải chia sẻ không gian địa chỉ chung.
- Khả năng IPC cung cấp 2 hoạt động:
  - **send** (*message*)– kích thước của message cố định hoặc biến đổi
  - **receive** (*message*)
- Nếu các tiến trình *P* và *Q* muốn giao tiếp, chúng cần phải:
  - thiết lập một *liên kết giao tiếp* (*communication link)* giữa chúng.
  - trao đổi các message qua các hoạt động send/receive.
- Sự thực hiện của communication link
  - physical (vd: shared memory, hardware bus)
  - logical (vd: logical properties)



Các mô hình giao tiếp



Giao tiếp trực tiếp

- Các tiến trình phải xác định rõ tên của nhau:
  - **send** (*P, message*) – gửi một message tới tiến trình P
  - **receive** (*Q, message*) – nhận message từ tiến trình Q
- Các đặc tính của communication link:
  - Các liên kết được thiết lập tự động.
  - Mỗi liên kết được gắn với duy nhất một cặp tiến trình giao tiếp với nhau.
  - Giữa mỗi cặp tiến trình tồn tại duy nhất 1 liên kết.
  - Liên kết thường là 2 chiều (bi-directional), hoặc có thể có 2 liên kết một chiều (unidirectional) P-to-Q, Q-to-P.

Giao tiếp gián tiếp

- Các message được gửi và nhận từ các mailbox (còn được gọi là port).
  - Mỗi mailbox có duy nhất một ***id***.
  - Các tiến trình chỉ có thể giao tiếp nếu chúng chia sẻ một mailbox.
- Các đặc tính của communication link:
  - Các liên kết được thiết lập chỉ khi các tiến trình chia sẻ một mailbox chung.
  - Một liên kết có thể được gắn với nhiều tiến trình.
  - Mỗi cặp tiến trình có thể chia sẻ một số communication link.
  - Liên kết có thể là 2 chiều hoặc 1 chiều.

Giao tiếp gián tiếp (tiếp)

- Các hoạt động:
  - tạo/xoá một mailbox
  - **send**(*A, message*) – gửi một message tới mailbox A
  - **receive**(*A, message*) – nhận một message từ mailbox A
- Vấn đề Mailbox sharing:
  - P_1, P_2, và P_3 chia sẻ mailbox A.
  - P_1, gửi; P_2 và P_3 nhận.
  - Tiến trình nào nhận được message?



Giao tiếp gián tiếp (tiếp)

- Giải pháp:
  - Một liên kết được gắn với tối đa 2 tiến trình.
  - Cho phép tại một thời điểm chỉ 1 tiến trình thực hiện nhận message.
  - Cho phép hệ thống tuỳ chọn tiến trình nhận. Tiến trình gửi được thông báo tiến trình nào nhận message.
  - Các tiến trình khác nhận được một bản copy.





Sự đồng bộ hóa - Synchronization

- Message passing có thể là có khóa (blocking) hoặc không khóa (non-blocking)
- **Blocking** được coi là *đồng bộ* (synchronous).
  - Blocking send: tiến trình gửi bị khóa đến khi message được nhận bởi tiến trình nhận hoặc bởi mailbox.
  - Blocking receive: tiến trình nhận/mailbox khóa đến khi nhận xong message.
- **Non-blocking** được coi là *không đồng bộ* (asynchronous).
  - Nonblocking send: tiến trình gửi gửi message rồi lại tiếp tục.
  - Nonblocking receive: tiến trình nhận/mailbox nhận được một message đúng hoặc vô dụng.





Buffering

- Hàng đợi của các message được gắn vào liên kết và được thực hiện bằng một trong ba cách:

1. Zero capacity: queue có độ dài lớn nhất bằng 0. Do đó, liên kết không thể có message đang chờ trong nó. Tiến trình gửi phải đợi tiến trình nhận.
2. Bounded capacity: queue có độ dài giới hạn bằng n. Khi 1 message được gửi, nó sẽ được đưa vào queue nếu queue chưa đầy, tiến trình gửi không phải đợi, và ngược lại.
3. Unbounded capacity: queue có độ dài vô hạn. Tiến trình gửi không bao giờ phải đợi.





VD: Windows 2000/XP

- Cung cấp sự hỗ trợ cho các môi trường đa xử lý (*subsystems*).
- Các chương trình ứng dụng có thể được coi là client của Windows 2000 subsystem server. Chúng giao tiếp sử dụng kỹ thuật message-passing.
- Khả năng message-passing trong Windows 2000 được gọi là LPC (local procedure call). LPC trong Windows 2000 cho phép giao tiếp giữa 2 tiến trình trên cùng máy.
- Mọi client gọi 1 subsystem cần có 1 kênh giao tiếp (connection port hay communication port) được cung cấp bởi port object.
- Sử dụng 3 kỹ thuật chuyển message qua cổng:
  - dùng message queue của cổng: message ≤ 256 byte
  - dùng bộ nhớ chia sẻ: dung lượng message lớn hơn
  - dùng kỹ thuật callback: khi client/server không thể đáp ứng lập tức



BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 4: Luồng (Threads)

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung



Nội dung chương 4

- Giới thiệu chung
- Các mô hình đa luồng
- Các vấn đề về luồng
- Một số loại luồng





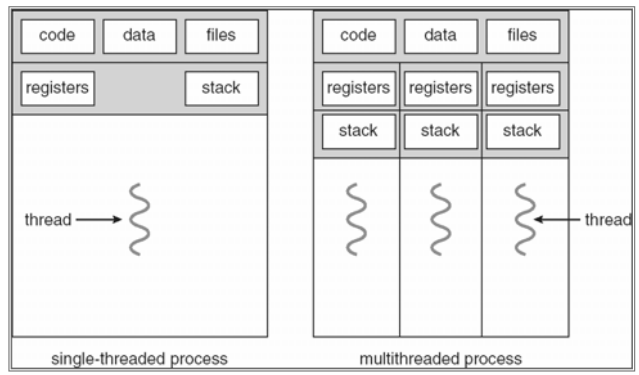
4.1. Giới thiệu chung

- Luồng là một đơn vị cơ bản của sự sử dụng CPU
- Là một dòng điều khiển trong một tiến trình. Nếu tiến trình có nhiều luồng, nó có thể thực hiện nhiều tác vụ tại một thời điểm.
- Luồng bao gồm:
  - Mã luồng (thread ID)
  - Bộ đếm chương trình (PC)
  - Tập thanh ghi (register set)
  - stack
- Các luồng trong một tiến trình chia sẻ với nhau đoạn mã (code), đoạn dữ liệu (data) và các tài nguyên hệ thống khác như các tệp mở, các tín hiệu.





Các tiến trình đơn luồng và đa luồng







Sự thúc đẩy

- Tạo tiến trình là một công việc "nặng nhọc"
- Nhiều phần mềm chạy trên các PC hiện nay là đa luồng (multithreaded). Một ứng dụng thường được thực hiện như một tiến trình riêng với một vài luồng điều khiển.
- Vd1: Trình soạn thảo văn bản
  - 1 luồng hiển thị ảnh, chữ
  - 1 luồng đọc phím nhấn bởi người sử dụng
  - 1 luồng thực hiện việc kiểm tra chính tả và ngữ pháp
- Vd2: web-server tạo 1 luồng nghe các yêu cầu từ client. Khi có yêu cầu, thay vì tạo 1 tiến trình khác, nó sẽ tạo một luồng khác để phục vụ yêu cầu.





Lợi ích của tiến trình đa luồng

- **Đáp ứng nhanh**: cho phép chương trình tiếp tục thực hiện thậm chí khi một bộ phận của nó bị khóa hoặc đang thực hiện một hoạt động dài.
- **Chia sẻ tài nguyên**: lợi ích của chia sẻ code là cho phép một ứng dụng có một số luồng khác nhau hoạt động trong cùng một không gian địa chỉ.
- **Kinh tế**: tạo và chuyển ngữ cảnh luồng kinh tế hơn so với tiến trình. Trong HĐH Solaris 2, tạo tiến trình chậm hơn 30 lần, chuyển ngữ cảnh tiến trình chậm hơn 5 lần với luồng.
- **Thực hiện trong kiến trúc multiprocessor**: lợi ích của đa luồng tăng lên trong kiến trúc multiprocessor, vì các luồng có thể chạy song song trên các processor.





User Threads

- Được hỗ trợ trên kernel và được thực hiện bởi một thư viện luồng tại mức người sử dụng (user level).
- Tất cả sự tạo luồng và lập lịch được thực hiện trong không gian người sử dụng. Do đó, các user-level thread nói chung nhanh để tạo và quản lý.
- Tuy nhiên, chúng cũng có hạn chế: khi kernel là đơn luồng, nếu có 1 user-level thread thực hiện một system call khóa, nó sẽ gây cho toàn bộ tiến trình bị khóa, mặc dù các tiến trình khác vẫn có thể chạy trong ứng dụng.
- Vd: POSIX Pthreads, Win32 threads, Java threads





Kernel Threads

- Được hỗ trợ trực tiếp bởi HĐH.
- Kernel thực hiện tạo luồng, lập lịch và quản lý trong không gian kernel. Do đó, tạo và quản lý các kernel thread nói chung chậm hơn các user thread.
- Nếu một luồng thực hiện một system call khóa, kernel có thể lập lịch một luồng khác để thực hiện. Trong môi trường multiprocessor, kernel có thể lập lịch các luồng trên các processor khác nhau.
- Vd: Các HĐH hiện nay: Windows NT/2000/XP, Solaris, Tru64 UNIX, LINUX, Mac OS X.

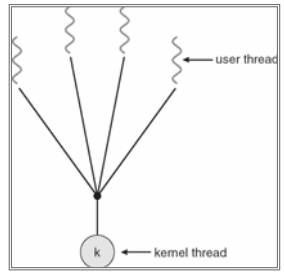


4.2. Các mô hình đa luồng

- Để chạy trên CPU, các user thread cuối cùng cũng phải được ánh xạ vào một kernel thread.
- Nhiều HĐH hỗ trợ cả user thread và kernel thread, thể hiện trong các mô hình đa luồng phổ biến:
  - Many-to-One
  - One-to-One
  - Many-to-Many
  - 2-level

Mô hình Many-to-One

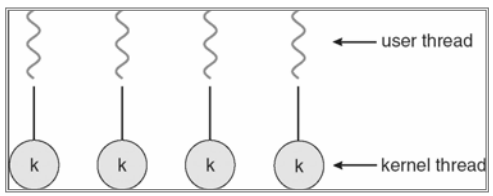
- Nhiều user-level thread được ánh xạ vào 1 kernel thread



- Quản lý luồng được thực hiện trong không gian người sử dụng → nhanh nhưng tiến trình dễ bị khóa.
- Các luồng không thể chạy song song trong các hệ thống multiprocessor.
- Vd: Solaris Green Theads, GNU Portable Threads

Mô hình One-to-One

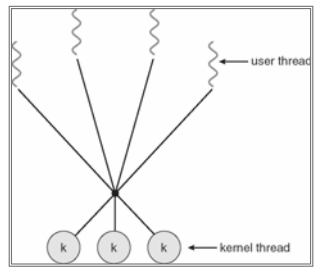
- Mỗi user-level thread được ánh xạ vào 1 kernel thread



- Cho phép tiến trình khác chạy khi có 1 tiến trình tạo system call khóa.
- Cho phép nhiều luồng chạy song song trên multiprocessor.
- Cần giới hạn số luồng được hỗ trợ bởi HĐH
- Vd: Windows NT/2000/XP, Linux, Solaris 9 trở đi

Mô hình Many-to-Many

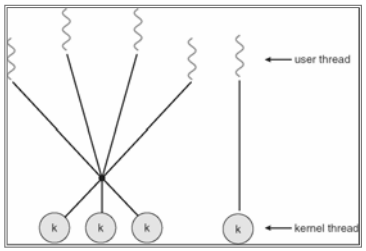
- Nhiều user-level thread (n) được ánh xạ vào nhiều kernel thread (m)
- $m \leq n$



- Người phát triển có thể tạo bao nhiêu user thread tùy ý, các kernel thread tương ứng có thể chạy song song trên multiprocessor. Khi 1 thread thực hiện 1 system call khóa, kernel có thể lập lịch 1 thread khác để thực hiện.
- Vd: Solaris trước phiên bản 9, Windows 2000/NT với gói *ThreadFiber*

Mô hình 2 mức

- Tương tự như Many-to-many, chỉ khác là nó cho phép 1 user thread được giới hạn bởi 1 kernel thread
- Ví dụ
  - IRIX
  - HP-UX
  - Tru64 UNIX
  - Solaris 8 trở về trước



4.3. Các vấn đề về luồng

- Các system call **fork()** và **exec()**
- Hủy luồng
- Xử lý tín hiệu
- Thread pools
- Dữ liệu riêng cho luồng
- Giao tiếp giữa kernel và thư viện luồng

4.3.1. Các system call fork và exec (trong UNIX)

- Nếu một luồng trong chương trình gọi `fork()`, một số HĐH UNIX có 2 phiên bản của fork
  - Một sao lại tất cả các thread
  - Một chỉ sao lại thread đã gọi fork
- Nếu 1 luồng gọi `exec`, chương trình được xác định trong tham số của exec sẽ thay thế toàn bộ tiến trình (gồm tất cả các luồng).

4.3.2. Hủy bỏ luồng

- Là tác vụ thực hiện hủy bỏ 1 thread trước khi nó kết thúc.
- Vd: nếu nhiều luồng cùng đang tìm kiếm trong CSDL, nếu 1 luồng tìm thấy, các luồng còn lại nên được dừng lại.
- Sự hủy luồng có thể diễn ra theo 2 cách:
  - Hủy không đồng bộ: lập tức ngừng luồng
  - Hủy trì hoãn: luồng bị hủy có thể kiểm tra tiên đoán xem nó có nên bị hủy không, cho phép nó có một cơ hội tự hủy theo cách có trật tự.



4.3.3. Xử lý tín hiệu

- Các tín hiệu được sử dụng trong HĐH UNIX để báo cho tiến trình biết có một sự kiện đặc biệt đã xuất hiện.
- Các tín hiệu được xử lý bởi 1 trình xử lý theo các bước:
  1. Tín hiệu được sinh ra bởi một sự kiện đặc biệt
  2. Tín hiệu được đưa đến 1 tiến trình
  3. Sau đó, tín hiệu được xử lý.
- Các lựa chọn:
  - Đưa tín hiệu tới luồng tương ứng dành cho tín hiệu
  - Đưa tín hiệu tới tất cả luồng trong tiến trình
  - Đưa tín hiệu tới một số luồng trong tiến trình
  - Ấn định một luồng chuyên nhận tất cả các tín hiệu cho tiến trình





4.3.4. Thread Pools

- Tư tưởng chung đằng sau một thread pool là tạo nhiều luồng tại lúc bắt đầu tiến trình và đặt chúng vào một *pool* - nơi chúng "ngồi" và đợi việc.
- Khi server nhận một yêu cầu, nó "đánh thức" một luồng trong pool - nếu nó sẵn sàng - truyền cho nó yêu cầu để phục vụ. Khi hoàn thành, luồng lại trở về pool chờ công việc khác.
- Lợi ích:
  - Dùng luồng đã tồn tại phục vụ nhanh hơn so với chờ đợi để tạo luồng.
  - thread pool giới hạn số luồng tồn tại ở một thời điểm. Điều này đặc biệt quan trọng trên các hệ thống không thể hỗ trợ số lượng lớn các luồng cùng lúc.





4.3.5. Dữ liệu riêng của luồng

- Các luồng thuộc 1 tiến trình có thể chia sẻ tài nguyên của tiến trình.
- Nhưng một số trường hợp: mỗi luồng cần dữ liệu riêng
- Ví dụ: trong một hệ thống xử lý giao dịch, ta nên phục vụ mỗi giao dịch trong 1 luồng riêng. Hơn nữa mỗi giao dịch có thể được gán 1 id duy nhất $\Rightarrow$ sử dụng dữ liệu riêng cho luồng.
- Lợi ích:
  - Cho phép mỗi luồng có bản copy dữ liệu riêng của nó.
  - Hữu ích khi bạn không có kiểm soát tiến trình tạo luồng (nghĩa là khi sử dụng 1 thread pool).





4.3.6. Giao tiếp kernel - thư viện luồng

- Cả mô hình many-to-many và mô hình 2-mức đều yêu cầu sự giao tiếp để duy trì số lượng thích hợp các kernel thread phân phối cho ứng dụng.
- **upcall** –cơ chế giao tiếp giữa kernel và thư viện luồng:
  - Kernel cung cấp một ứng dụng gồm một tập các BXL ảo
  - ứng dụng có thể lập lịch các user thread vào một BXL ảo khả dụng.
  - Kernel phải thông báo cho ứng dụng về các sự kiện nào đó
- Sự giao tiếp này cho phép một ứng dụng duy trì số lượng kernel thread đúng đắn.





4.4. Một số loại luồng

4.4.1. Pthreads

- Là chuẩn POSIX (IEEE 1003.1c), định ra một API cho việc tạo và đồng bộ hóa luồng.
- API xác định hành vi (behavior) của thư viện luồng, sự thực thi (implementation) phụ thuộc vào sự phát triển của thư viện.
- Phổ biến trong các HĐH dạng UNIX (Solaris, Linux, Mac OS X, True64 UNIX)





4.4.2. Luồng Windows XP

- Áp dụng cho "họ" Windows: 95/98/2000/XP/NT (Win32 API)
- Một ứng dụng Windows chạy như một tiến trình riêng, mỗi tiến trình có thể chứa một hoặc nhiều luồng.
- Sử dụng mô hình ánh xạ one-to-one
- Cũng hỗ trợ thư viện fiber, cung cấp chức năng của mô hình many-to-many.
- Bằng cách sử dụng thư viện luồng, mọi luồng của tiến trình có thể truy nhập không gian địa chỉ của tiến trình.
- Các thành phần của luồng:
  - một thread ID
  - tập thanh ghi biểu diễn trạng thái của processor
  - một user stack và một kernel stack
  - một vùng lưu trữ riêng.

context của luồng





4.4.3. Luồng LINUX

- Ngoài `fork()`, Linux cũng cung cấp system call tương tự là `clone()` để tạo luồng, nhưng thay vì tạo 1 bản copy của tiến trình gọi, nó tạo 1 tiến trình mới (tiến trình con).
- Tiến trình mới trỏ vào cấu trúc dữ liệu của tiến trình cha, do đó cho phép tiến trình con chia sẻ bộ nhớ và các tài nguyên khác của cha.
- Điều thú vị là Linux không phân biệt giữa tiến trình và luồng. Thực tế, Linux thường dùng thuật ngữ *task* để chỉ một dòng điều khiển trong chương trình.





4.4.4. Luồng Java

- Java là một trong số ít các NNLT có cung cấp sự hỗ trợ tạo và quản lý luồng tại mức ngôn ngữ: có các lệnh tạo và thao tác với các luồng điều khiển trong chương trình.
- Các luồng được quản lý bởi JVM (Java Virtual Machine), không phải bởi thư viện luồng hay bởi kernel.
- Tất cả các chương trình Java chứa ít nhất 1 luồng điều khiển đơn (khi chương trình chỉ có 1 phương thức `main`).



Các kỹ thuật tạo luồng trong CT Java

1. Tạo một lớp dẫn xuất của lớp `Thread` và chồng phương thức `run()`.
   - Một đối tượng của lớp dẫn xuất sẽ chạy như 1 luồng điều khiển riêng trong JVM. Phương thức `start` sẽ thực sự tạo luồng mới.
2. Định nghĩa một lớp thực thi `Runnable` interface:

```
public interface Runnable{
   public abstract void run();
}
```

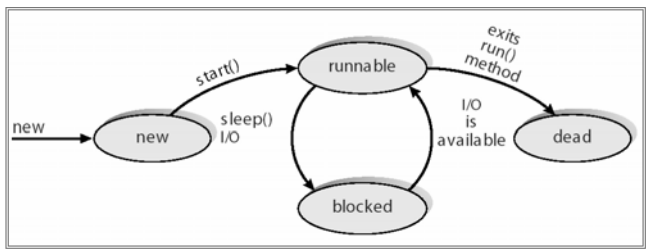

Vd: Tạo luồng trong Java

```
class Worker1 extends Thread {
  public void run() {
    System.out.println("I Am a Worker Thread");
  }
}

public class ThreadTester {
  public static void main(String args[]) {
    Worker1 runner = new Worker1();
    runner.start();

    System.out.println("I Am The Main Thread");
  }
}
```


Các trạng thái của luồng Java



End of Chapter 4

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 5: Lập lịch CPU

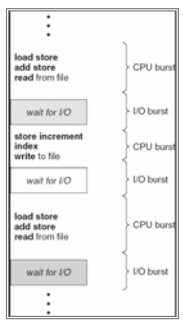
Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung

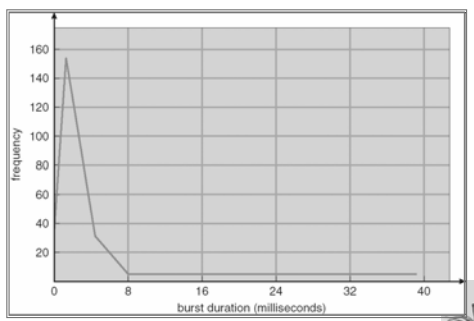
Nội dung chương 5

- Các khái niệm cơ bản
- Các tiêu chuẩn lập lịch
- Các giải thuật lập lịch
- Lập lịch multiprocessor
- Lập lịch thời gian thực
- Lựa chọn giải thuật

5.1. Các khái niệm cơ bản

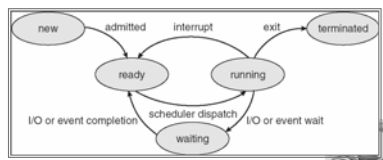
- multi-programming giúp đạt được sự sử dụng CPU tối đa
- Chu kỳ sử dụng CPU–I/O (CPU–I/O Burst Cycle): Sự thực hiện tiến trình gồm một chu kỳ thực hiện của CPU và chờ vào-ra.
- Sự phân phối sử dụng CPU giúp lựa chọn giải thuật lập lịch CPU





Trình lập lịch CPU - CPU Scheduler

- Mỗi khi CPU rỗi, HĐH cần chọn trong số các tiến trình đã sẵn sàng thực hiện trong bộ nhớ (ready queue), và phân phối CPU cho một trong số đó.
- Tiến trình được thực hiện bởi trình lập lịch ngắn kỳ (short-term scheduler, CPU scheduler)
- Các quyết định lập lịch CPU có thể xảy ra khi một tiến trình:
  1. Chuyển từ trạng thái chạy sang trạng thái chờ (vd: I/O request)
  2. Chuyển từ trạng thái chạy sang trạng thái sẵn sàng (vd: khi một ngắt xuất hiện)
  3. Chuyển từ trạng thái đợi sang trạng thái sẵn sàng (vd: I/O hoàn thành)
  4. Kết thúc



Preemptive/nonpreemptive Scheduling

- Lập lịch CPU khi 1 và 4 là *không được ưu tiên trước* (*nonpreemptive*):
  - Không có sự lựa chọn: phải chọn 1 tiến trình mới để thực hiện.
  - Khi 1 tiến trình được phân phối CPU, nó sẽ sử dụng CPU cho đến khi nó giải phóng CPU bằng cách kết thúc hoặc chuyển sang trạng thái chờ.
  - Các tiến trình sẵn sàng nhường điều khiển của CPU.
- Lập lịch khi 2 và 3 là *được ưu tiên trước* (*preemptive*)
  - Khi 2: tiến trình đá bật CPU ra. Cần phải chọn tiến trình kế tiếp.
  - Khi 3: tiến trình có thể đá bật tiến trình khác ra khỏi CPU.

Trình điều vận - Dispatcher

- Môđun trình điều vận trao quyền điều khiển của CPU cho tiến trình được lựa chọn bởi trình lập lịch CPU; các bước:
  - chuyển ngữ cảnh
  - chuyển sang user mode
  - nhảy tới vị trí thích hợp trong chương trình của người sử dụng để khởi động lại chương trình đó
- *Trễ điều vận (Dispatch latency)* – thời gian cần thiết để trình điều vận dừng một tiến trình và khởi động một tiến trình khác chạy.

5.2. Các tiêu chuẩn lập lịch

Max
- **CPU utilization** – giữ cho CPU càng bận càng tốt (0-100%)

Max
- **Throughput** – số tiến trình được hoàn thành trong một đơn vị thời gian

Min
- **Turnaround time** – tổng lượng thời gian để thực hiện một tiến trình: t/g chờ được đưa vào bộ nhớ + t/g chờ trong ready queue + t/g thực hiện bởi CPU + t/g thực hiện vào-ra

Min
- **Waiting time** – lượng thời gian mà một tiến trình chờ đợi ở trong ready queue

Min
- **Response time** – lượng thời gian tính từ khi có một yêu cầu được gửi đến khi có sự trả lời đầu tiên được phát ra, không phải là thời gian đưa ra kết quả của sự trả lời đó. → là tiêu chuẩn tốt.

5.3. Các giải thuật lập lịch

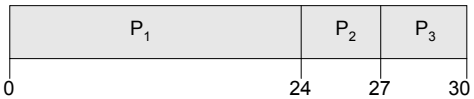
- First Come, First Served (FCFS)
- Shortest Job First (SJF)
- Priority
- Round Robin (RR)
- Multilevel Queue
- Multilevel Feedback-Queue

1) Giải thuật First-Come, First-Served (FCFS)

- Tiến trình nào yêu cầu CPU trước sẽ được phân phối CPU trước
  → Giải thuật FCFS là không được ưu tiên trước.
- Là giải thuật đơn giản nhất

Process	Burst Time (thời gian sử dụng CPU, ms)
P_1	24
P_2	3
P_3	3

- Giả định rằng các tiến trình đến theo thứ tự: P_1, P_2, P_3 thì biểu đồ Gantt (Gantt Chart) của lịch biểu như sau:



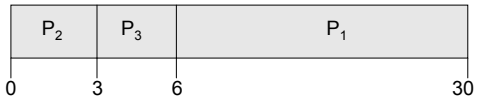
Thời gian chờ đợi của các tiến trình: P_1 = 0; P_2 = 24; P_3 = 27

- Thời gian chờ đợi trung bình: (0 + 24 + 27)/3 = 17

Giải thuật FCFS (tiếp)

Giả định rằng các tiến trình đến theo thứ tự P_2 , P_3 , P_1

- Biểu đồ Gantt của lịch biểu như sau:



- Thời gian chờ đợi của các tiến trình: P_1 = 6; P_2 = 0; P_3 = 3
- Thời gian chờ đợi trung bình: (6 + 0 + 3)/3 = 3
- Tốt hơn nhiều so với trường hợp trước
- *Convoy effect* (hiệu ứng hộ tống): tiến trình ngắn đứng sau tiến trình dài, như là các xe máy đi sau xe buýt vậy.

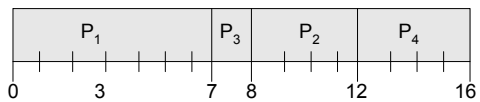
2) Giải thuật Shortest-Job-First (SJF)

- Gắn với mỗi tiến trình là thời gian sử dụng CPU *tiếp sau* của nó. Thời gian này được sử dụng để lập lịch các tiến trình với thời gian ngắn nhất.
- Hai phương pháp:
  - không ưu tiên trước (non-preemptive)– một tiến trình nếu sử dụng CPU thì không nhường cho tiến trình khác cho đến khi nó kết thúc.
  - có ưu tiên trước (preemptive)– nếu một tiến trình đến có thời gian sử dụng CPU *ngắn hơn* thời gian còn lại của tiến trình đang thực hiện thì ưu tiên tiến trình mới đến trước. Phương pháp này còn được gọi là Shortest-Remaining-Time-First (SRTF).
- SJF là tối ưu – cho thời gian chờ đợi trung bình của các tiến trình là nhỏ nhất.

Ví dụ SJF không ưu tiên trước

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)

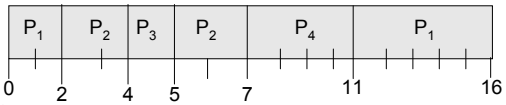


- Thời gian chờ đợi của các tiến trình: P_1 = 0; P_2 = 6; P_3 = 3, P_4 = 7
- Thời gian chờ đợi trung bình = (0 + 6 + 3 + 7)/4 = 4

Ví dụ SJF có ưu tiên trước

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



- Thời gian chờ đợi trung bình = (9 + 1 + 0 +2)/4 = 3

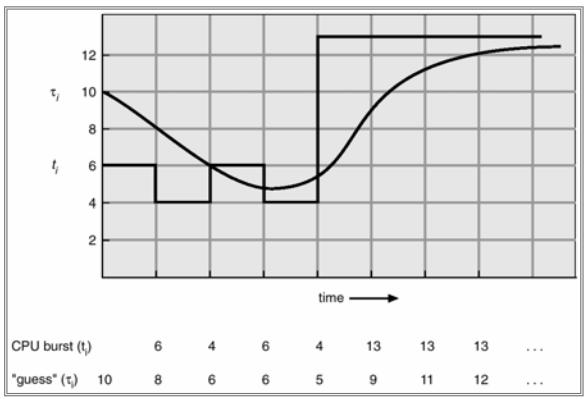
Xác định thời gian sử dụng CPU tiếp sau

- Không thể biết chính xác thời gian sử dụng CPU tiếp sau của tiến trình nhưng có thể đoán giá trị xấp xỉ của nó dựa vào thời gian sử dụng CPU trước đó và sử dụng công thức đệ quy:

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\tau_n.$$

  - τ_{n+1} = giá trị dự đoán cho thời gian sử dụng CPU tiếp sau
  - t_n = thời gian thực tế của sự sử dụng CPU thứ n
  - α, $0 \le \alpha \le 1$
  - τ_0 là một hằng số
- $\alpha = 0$: $\tau_{n+1} = \tau_n = \tau_0$.
  - Thời gian thực tế sử dụng CPU gần đây không có tác dụng gì cả.
- $\alpha = 1$: $\tau_{n+1} = t_n$.
  - Chỉ tính đến thời gian sử dụng CPU thực tế ngay trước đó.

Minh họa khi $\alpha = 1/2$ và $\tau_0 = 10$



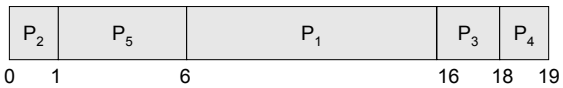
3) Lập lịch theo mức ưu tiên

- Mỗi tiến trình được gắn một số ưu tiên (số nguyên). VD: 0-127
- CPU được phân phối cho tiến trình có mức ưu tiên cao nhất (có số ưu tiên nhỏ nhất)
  - Preemptive
  - nonpreemptive
- SJF là trường hợp riêng của lập lịch theo mức ưu tiên: mức ưu tiên chính là thời gian sử dụng CPU tiếp sau dự đoán được.
- Vấn đề gặp phải là: những tiến trình có mức ưu tiên thấp có thể không bao giờ được thực hiện (starvation).
- Giải pháp ≡ Aging: kỹ thuật tăng mức ưu tiên của các tiến trình chờ đợi lâu trong hệ thống.
  - VD: Sau 1-15 phút giảm số ưu tiên một lần.

Ví dụ lập lịch theo mức ưu tiên

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Nonpreemptive:



- T/gian chờ đợi trung bình = (0 + 1 + 6 + 16 + 18)/5 = 8.2

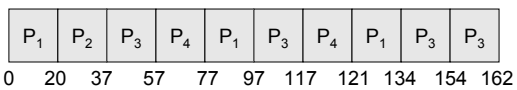
4) Giải thuật Round-Robin (RR)

- Mỗi tiến trình sử dụng một lượng nhỏ thời gian của CPU (*time quantum – định lượng thời gian, q*), thường là 10-100 ms. Sau đó nó được ưu tiên đưa vào cuối của ready queue.
- Ready queue được tổ chức dạng FIFO (FCFS)
- Nếu tiến trình có thời gian sử dụng CPU < $q \Rightarrow$ Tiến trình sẽ tự nguyện nhường CPU khi kết thúc. Trình lập lịch sẽ chọn tiến trình kế tiếp trong ready queue.
- Nếu tiến trình có thời gian sử dụng CPU > $q \Rightarrow$ bộ định thời (timer) sẽ đếm lùi và gây ngắt HĐH khi nó = 0. Việc chuyển ngữ cảnh được thực hiện và tiến trình hiện tại được đưa xuống cuối ready queue để nhường CPU cho tiến trình kế tiếp.

Ví dụ lập lịch RR với q = 20

Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

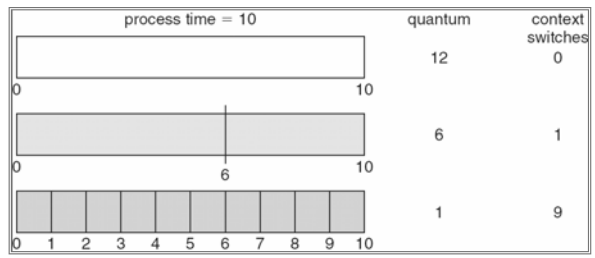
- Biểu đồ Gantt:



- T/g chờ đợi TB = ((57+24)+20+(37+40+17)+(57+40))/4 = 73
- Thường thì *RR* có turnaround trung bình cao hơn SJF, nhưng có *response* tốt hơn (thấp hơn).

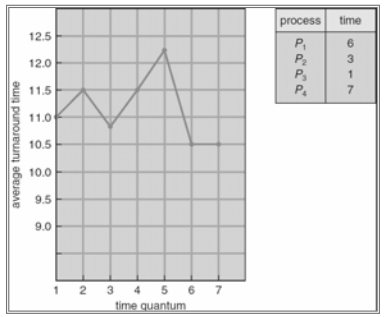
Quan hệ giữa q và hiệu năng

- Nếu q lớn $\Rightarrow$ tương tự như FCFS.
- Nếu q nhỏ $\Rightarrow$ số lần chuyển ngữ cảnh càng nhiều, làm giảm hiệu năng.



Phụ thuộc của Turnaround Time vào q

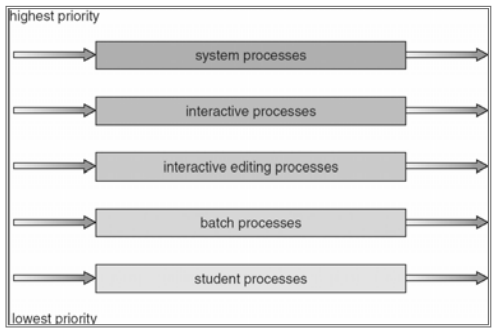
- Turnaround Time trung bình của một tập tiến trình không phải luôn được cải thiện khi q tăng lên.
- Luật: 80% các CPU burst nên nhỏ hơn q.



5) Lập lịch đa mức hàng đợi

- Ready queue được chia thành nhiều queue riêng biệt:
  foreground (chứa các *interactive process*)
  background (chứa các *batch process*)
- Mỗi hàng đợi có giải thuật lập lịch riêng:
  - foreground – RR
  - background – FCFS
- Phải có sự lập lịch giữa các queue:
  - Lập lịch với mức ưu tiên cố định; vd: phục vụ tất cả tiến trình từ foreground, tiếp theo từ background (có thể xảy ra starvation).
  - Phân chia thời gian: mỗi queue nhận được một lượng thời gian CPU nào đó mà nó có thể lập lịch các tiến trình của nó.
    - vd: 80% cho foreground và 20% cho background queue

Multilevel Queue Scheduling



➢ Tiến trình trong queue có mức ưu tiên thấp hơn chỉ có thể chạy khi các queue có mức ưu tiên cao hơn rỗng.

➢ Tiến trình có mức ưu tiên cao hơn khi vào ready queue không ảnh hưởng đến tiến trình đang chạy có mức ưu tiên thấp hơn.

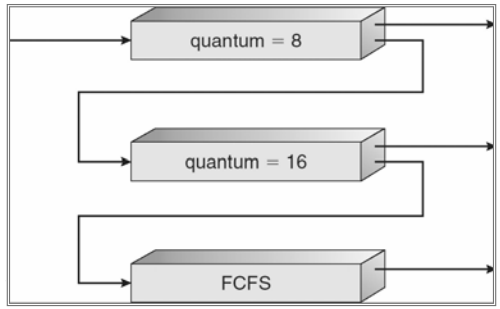
6) Lập lịch đa mức hàng đợi thông tin phản hồi

- Một tiến trình có thể di chuyển giữa các queue khác nhau; có thể thực hiện aging
- Trình lập lịch đa mức hàng đợi thông tin phản hồi được xác định bởi các tham số sau:
  - số lượng queue
  - giải thuật lập lịch cho mỗi queue
  - phương pháp được sử dụng để xác định khi nào thì tăng/giảm mức ưu tiên của một tiến trình
  - phương pháp được sử dụng để xác định queue nào mà tiến trình sẽ đến khi nó cần được phục vụ.

Ví dụ Multilevel Feedback Queue

- Ba queue:
  - Q_0 – thời gian định lượng 8 ms
  - Q_1 – thời gian định lượng 16 ms
  - Q_2 – FCFS
- Lập lịch:
  - Một tiến trình vào queue Q_0 và được phục vụ FCFS. Khi nó giành được CPU, tiến trình nhận được 8 ms. Nếu nó không hoàn thành trong 8 ms, tiến trình được chuyển tới queue Q_1.
  - Tại Q_1 tiến trình tiếp tục được phục vụ FCFS với 16 ms nữa. Nếu nó vẫn chưa hoàn thành thì nó được ưu tiên và được chuyển đến queue Q_2.

Multilevel Feedback Queues



5.4. Lập lịch multiprocessor

- Lập lịch CPU khi có nhiều processor phức tạp hơn nhiều
- Các loại processor trong multiprocessor
  - *Đồng nhất (Homogeneous):* tất cả có cùng kiến trúc.
  - *Không đồng nhất (Heterogeneous)*: một số tiến trình có thể không tương thích với kiến trúc của các CPU.
- *Cân bằng tải (Load balancing/sharing)*: một ready queue cho tất cả các processor, CPU nhàn rỗi được gán cho tiến trình ở đầu queue.
- *Đa xử lý không đối xứng - Asymmetric multiprocessing:*
  - chỉ một processor (master processor) truy nhập các cấu trúc dữ liệu hệ thống, làm giảm sự cần thiết bảo vệ dữ liệu chia sẻ.

5.5. Lập lịch thời gian thực

- *Hard real-time* systems – yêu cầu hoàn thành một tác vụ găng (critical task) trong thời gian được đảm bảo.
  - **resource reservation**: khi tiến trình được gửi đến cùng với lệnh cho biết thời gian cần thiết của nó, trình lập lịch có thể chấp nhận và đảm bảo nó sẽ kết thúc đúng hạn, hoặc từ chối tiến trình.
- *Soft real-time* computing – yêu cầu các tiến trình găng nhận mức ưu tiên trên các tiến trình kém may mắn hơn.
  - có thể phân phối tài nguyên không hợp lý, thời gian trễ lâu, starvation.

  → phải cẩn thận trong thiết kế trình lập lịch và các khía cạnh liên quan của HĐH:
    - lập lịch có ưu tiên, các tiến trình thời gian thực có mức ưu tiên cao nhất.
    - trễ điều vận (dispatch latency) phải nhỏ.

5.6. Lập lịch luồng

- Lập lịch cục bộ (Local Scheduling):
  - Bằng cách nào Thư viện luồng quyết định chọn luồng nào để đặt vào một CPU ảo khả dụng:
    - Thường chọn luồng có mức ưu tiên cao nhất
  - Sự cạnh tranh CPU diễn ra giữa các luồng của cùng một tiến trình.
  - Trong các HĐH sử dụng mô hình Many-to-one, Many-to-many.
- Lập lịch toàn cục (Global Scheduling)
  - Bằng cách nào kernel quyết định kernel thread nào để lập lịch CPU chạy tiếp.
  - Sự cạnh tranh CPU diễn ra giữa tất cả các luồng trong hệ thống.
  - Trong các HĐH sử dụng mô hình One-to-one (Windows XP, Linux, Solaris 9)

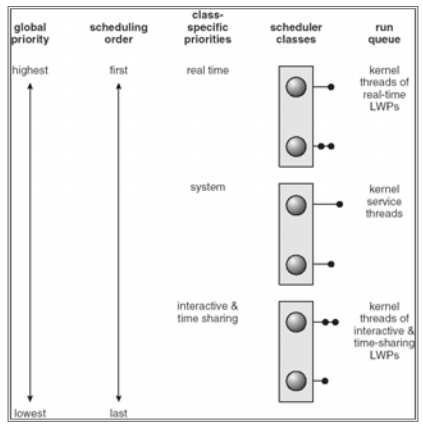
5.7. Các ví dụ lập lịch trong HĐH

- Solaris scheduling
- Windows XP scheduling
- Linux scheduling

1) Solaris 2 Scheduling

- Lập lịch dựa trên mức ưu tiên
- Xác định 4 lớp lập lịch có thứ tự ưu tiên như sau
  1. Real time
  2. System
  3. Time sharing
  4. Interactive
- Trong mỗi lớp có các mức ưu tiên khác nhau và các giải thuật lập lịch khác nhau.

Solaris 2 Scheduling (tiếp)



Lớp cho các luồng tương tác và chia sẻ thời gian

- Lớp lập lịch mặc định cho tiến trình là Time sharing. Chính sách lập lịch tự động thay đổi mức ưu tiên và gán các q khác nhau nhờ dùng một multilevel feedback queue.
- Mặc định: Quan hệ tỷ lệ nghịch: mức ưu tiên càng cao, q càng nhỏ và ngược lại.
- Các tiến trình tương tác có mức ưu tiên cao hơn các tiến trình CPU-bound
  - Thời gian đáp ứng tốt cho các tiến trình tương tác
  - Thông lượng tốt cho các tiến trình CPU-bound

Bảng điều vận của Solaris cho các luồng tương tác và chia sẻ thời gian

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

Giải thích

- **Priority**: mức ưu tiên phụ thuộc lớp của các lớp tương tác và chia sẻ thời gian.
  - Số càng lớn mức ưu tiên càng cao.
- **Time quantum**: lượng thời gian cho mức ưu tiên tương ứng.
  - Tỷ lệ nghịch với mức ưu tiên
- **Time quantum expired**: mức ưu tiên mới của một luồng mà đã sử dụng hết lượng thời gian của nó mà chưa kết thúc.
  - Thấp hơn mức ưu tiên gốc
- **Return from sleep**: mức ưu tiên của một luồng đang trở về từ trạng thái ngủ (vd đợi vào/ra).
  - Cao hơn mức ưu tiên gốc, giúp giảm thời gian đáp ứng cho các tiến trình tương tác.

Lớp cho các luồng thời gian thực và luồng hệ thống

- Lớp hệ thống
  - Để chạy các tiến trình kernel, ví dụ như trình lập lịch, trình quản lý phân trang bộ nhớ.
  - Các tiến trình người sử dụng chạy trong kernel mode không ở trong lớp hệ thống.
  - Mức ưu tiên của các tiến trình hệ thống là không thay đổi.
- Lớp thời gian thực
  - Chứa các luồng có mức ưu tiên cao nhất ⇒ tiến trình thời gian thực sẽ được chạy trước tiến trình thuộc các lớp khác
  - Ít tiến trình thuộc lớp này.



Solaris 9 scheduling

- Có thêm 2 lớp lập lịch mới cho các luồng:
- Fixed priority:
  - Dải mức ưu tiên giống như lớp chia sẻ thời gian
  - Khác ở chỗ mức ưu tiên không được điều chỉnh tự động
- Fair share:
  - Sử dụng sự chia sẻ CPU thay vì dùng mức ưu tiên





Chọn luồng tổng thể?

- Mỗi lớp có tập mức ưu tiên riêng (cục bộ).
- Trình lập lịch
  - chuyển đổi mức ưu tiên cục bộ thành mức ưu tiên toàn cục
  - chọn luồng có mức ưu tiên toàn cục lớn nhất để chạy.
- Luồng được chọn sẽ dùng CPU cho đến khi:
  - Nó chuyển trạng thái khóa (vd để vào/ra)
  - Nó sử dụng hết lượng thời gian q
  - Bị một luồng có mức ưu tiên cao hơn giành quyền
- Khi có nhiều luồng có cùng mức ưu tiên, trình lập lịch sử dụng một hàng đợi round-robin.





2) Windows XP Scheduling

- Sử dụng giải thuật lập lịch có ưu tiên trước dựa trên mức ưu tiên.
- Chịu trách nhiệm lập lịch luồng: trình điều vận (dispatcher)
- Một luồng được chọn sẽ chạy đến khi:
  - Nó gọi system call khóa (vd để vào/ra)
  - Nó sử dụng hết lượng thời gian q
  - Bị một luồng có mức ưu tiên cao hơn giành quyền
- Trình điều vận sử dụng dải ưu tiên 32 mức, chia làm 2 lớp:
  - **Variable class**: chứa các luồng có mức từ 1 đến 15
  - **Real-time class**: chứa các luồng có mức từ 16 đến 31.
- Mỗi mức ưu tiên có 1 queue. Trình điều vận duyệt qua các queue từ cao nhất đến thấp nhất cho đến khi tìm thấy 1 luồng sẵn sàng. Nếu không có luồng như vậy, trình điều vận thực hiện luồng đặc biệt – **idle thread**.





Quan hệ mức ưu tiên của kernel và Win32 API

- Win32 API định nghĩa các lớp ưu tiên:
  - REALTIME_PRIORITY_CLASS
  - HIGH_PRIORITY_CLASS
  - ABOVE_NORMAL_PRIORITY_CLASS
  - NORMAL_PRIORITY_CLASS
  - BELOW_NORMAL_PRIORITY_CLASS
  - IDLE_PRIORITY_CLASS

  Mức ưu tiên của một luồng thuộc một trong những lớp này là có thể thay đổi
- Trong mỗi lớp ưu tiên gồm các giá trị ưu tiên:
  - TIME_CRITICAL
  - HIGHEST
  - ABOVE_NORMAL
  - NORMAL
  - BELOW_NORMAL
  - LOWEST
  - IDLE



Quan hệ mức ưu tiên của kernel và Win32 API

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
Base Priority → normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

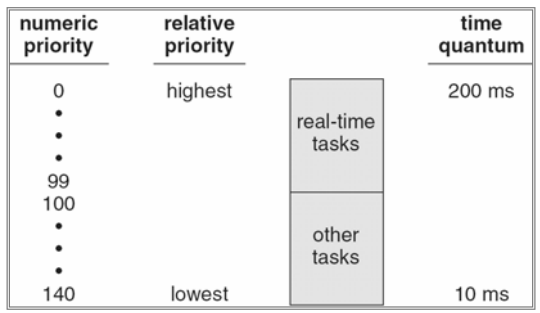
Chiến lược tăng, giảm mức ưu tiên

- Khi lượng thời gian q của luồng hết, luồng bị ngắt. Nếu luồng thuộc lớp có thể thay đổi mức ưu tiên thì mức của nó bị giảm. Tuy nhiên không giảm đến dưới mức ưu tiên cơ sở.
  ⇒ có xu hướng giới hạn sử dụng CPU của các luồng CPU-bound
- Khi luồng thuộc lớp có thể thay đổi mức ưu tiên được giải phóng khỏi hoạt động đợi, trình điều vận tăng mức ưu tiên cho nó. Lượng tăng phụ thuộc luồng đã đợi cái gì, ví dụ:
  - Đợi vào/ra bàn phím ⇒ tăng mạnh
  - Đợi hoạt động đĩa ⇒ tăng vừa
- Chiến lược này có xu hướng cho thời gian đáp ứng tốt với các luồng tương tác sử dụng chuột và cửa sổ, cho phép các luồng I/O-bound giữ các thiết bị vào/ra "bận rộn", và các luồng CPU-bound sử dụng các chu kỳ CPU dư thừa trong "lặng lẽ".
- Chiến lược này một số HĐH chia sẻ thời gian sử dụng, vd UNIX.
- Ngoài ra, cửa sổ mà user đang tương tác được nâng mức ưu tiên.

3) Linux Scheduling

- Linux kernel từ phiên bản 2.5,
  - Cung cấp giải thuật lập lịch chạy trong thời gian hằng - *O*(1) – bất kể số lượng luồng trong hệ thống.
  - Hỗ trợ tốt hơn cho các hệ thống đa xử lý đối xứng (SMP)
  - Cung cấp sự công bằng và hỗ trợ cho các tác vụ (task) tương tác
- Trình lập lịch Linux dùng giải thuật có ưu tiên trước dựa trên mức ưu tiên với 2 dải riêng:
  - Dải **real-time**: có mức từ 0 đến 99
  - Dải **nice**: có mức từ 100 đến 140.
- Số càng nhỏ, mức ưu tiên càng cao
- Không giống với nhiều HĐH khác, Linux gán các task có mức ưu tiên càng cao càng được nhiều lượng thời gian q.

Quan hệ giữa Priorities và Time-slice length





Linux Scheduling (tiếp)

- Kernel duy trì danh sách các task sẵn sàng trong một **runqueue**
  - Nếu có nhiều bộ xử lý, mỗi BXL có 1 runqueue và lập lịch độc lập
- Mỗi runqueue có 2 mảng ưu tiên:
  - **active**: chứa tất cả task vẫn còn trong lượng thời gian của chúng
  - **expired**: chứa tất cả task đã hết lượng thời gian
- Trình lập lịch chọn task có mức ưu tiên cao nhất trong mảng active. Khi mảng active rỗng, 2 mảng sẽ tráo đổi cho nhau.
- Các real-time task được gán mức ưu tiên tĩnh
- Các task khác có mức ưu tiên động: = nice value ± 5
  - Task có thời gian ngủ đợi vào/ra dài hơn thường là tương tác hơn
    ⇒ tăng mức ưu tiên bằng cách -5
  - Task có thời gian ngủ ngắn hơn thường là CPU-bound
    ⇒ giảm mức ưu tiên bằng cách +5





5.8. Lựa chọn giải thuật

- Chọn giải thuật lập lịch CPU nào cho hệ thống cụ thể?
- Trước tiên, xác định sử dụng tiêu chuẩn nào? Vd:
  - Tối đa CPU utilization với ràng buộc response time lớn nhất là 1s
  - Tối đa throughput để turnaround time là tỷ lệ tuyến tính với thời gian thực hiện

1. Phân tích hiệu năng của từng giải thuật đối với các tiến trình
2. Sử dụng chuẩn hàng đợi: công thức Little: $n = \lambda \times W$

   n: độ dài queue trung bình

   W: thời gian chờ đợi trung bình trong queue

   λ: tốc độ đến queue của tiến trình (số tiến trình/giây)
3. Mô phỏng: lập trình mô hình hệ thống để đánh giá
4. Thực hiện: đặt giải thuật cụ thể trong hệ thống thực để đánh giá



End of Chapter 5

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 6: Đồng bộ hóa tiến trình

Phạm Quang Dũng

Bộ môn Khoa học máy tính

Khoa Công nghệ thông tin

Trường Đại học Nông nghiệp Hà Nội

Website: fita.hua.edu.vn/pqdung



Nội dung

- Cơ sở
- Vấn đề đoạn găng
- Giải pháp của Peterson
- Phần cứng đồng bộ hóa
- Kỹ thuật cờ báo (Semaphores)





6.1. Cơ sở

- Sự truy nhập đồng thời đến dữ liệu chia sẻ có thể gây ra sự mâu thuẫn.
- Để duy trì tính nhất quán dữ liệu cần có cơ chế đảm bảo thực hiện các tiến trình hợp tác theo thứ tự.
- Giả sử rằng chúng ta muốn đưa ra một giải pháp cho vấn đề tiến trình sản xuất - tiến trình tiêu thụ mà đều điền vào buffer. Chúng ta có thể làm được bằng cách có một biến nguyên count để theo dõi số phần tử trong buffer.
  - Khởi tạo count=0.
  - Nó được tăng bởi tiến trình sản xuất khi thêm vào buffer 1 phần tử.
  - Nó bị giảm bởi tiến trình tiêu thụ khi lấy khỏi buffer 1 phần tử





Producer

```
while (true) {
        /*  produce an item and put in nextProduced  */
        while (count == BUFFER_SIZE)
                ; // do nothing
            buffer [in] = nextProduced;
            in = (in + 1) % BUFFER_SIZE;
            count++;
}
```



Consumer

```
while (true)  {
      while (count == 0)
            ; // do nothing
            nextConsumed =  buffer[out];
             out = (out + 1) % BUFFER_SIZE;
              count--;

                /*  consume the item in nextConsumed
}
```


Trạng thái tranh đua (Race condition)

- count++ có thể được thực thi như sau:

```
register1 = count
register1 = register1 + 1
count = register1
```

- count-- có thể được thực thi như sau:

```
register2 = count
register2 = register2 - 1
count = register2
```

- Xét sự thực hiện đan xen với ban đầu "count = 5":

S0: producer execute register1 = count	{register1 = 5}
S1: producer execute register1 = register1 + 1	{register1 = 6}
S2: consumer execute register2 = count	{register2 = 5}
S3: consumer execute register2 = register2 - 1	{register2 = 4}
S4: producer execute count = register1	{count = 6}
S5: consumer execute count = register2	{count = 4}

6.2. Vấn đề đoạn găng (Critical-Section)

- Xét hệ thống gồm *n* tiến trình {P_0, P_1, …, P_{n-1}}.
- Mỗi tiến trình có một đoạn mã, gọi là **đoạn găng**, mà tại đó tiến trình có thể thay đổi các biến chung, cập nhật bảng, ghi tệp…
- Đặc điểm quan trọng của hệ thống là tại mỗi thời điểm chỉ có 1 tiến trình thực hiện trong đoạn găng của nó.

⇔ sự thực hiện các đoạn găng là *loại trừ lẫn nhau* theo thời gian.

- Vấn đề đoạn găng là thiết kế một giao thức mà các tiến trình sử dụng để hợp tác. Mỗi tiến trình phải yêu cầu sự cho phép để bước vào đoạn găng của nó. Đoạn mã thực hiện yêu cầu này được gọi là **đoạn vào**. Sau đoạn găng có thể có **đoạn ra**.

Cấu trúc tổng quát của tiến trình P_i

```
do {
      đoạn vào
      đoạn găng
      đoạn ra
      đoạn còn lại
} while (TRUE) ;
```



Giải pháp cho vấn đề đoạn găng

Một giải pháp cho vấn đề đoạn găng phải thỏa mãn 3 yêu cầu:

1. **Loại trừ lẫn nhau**: nếu tiến trình P_i đang thực hiện trong đoạn găng của nó thì các tiến trình khác không được thực hiện trong đoạn găng của chúng.
2. **Chọn tiến trình tiếp theo được vào đoạn găng**: nếu không có tiến trình nào đang trong đoạn găng của nó và một số tiến trình muốn vào đoạn găng của chúng thì chỉ những tiến trình đang không trong đoạn còn lại mới là ứng cử viên.
3. **Chờ đợi có hạn**: tồn tại giới hạn số lần các tiến trình khác được phép vào đoạn găng của chúng sau khi một tiến trình yêu cầu vào đoạn găng đến trước khi yêu cầu đó được đáp ứng.





Các phương pháp xử lý đoạn găng

- **kernel không ưu tiên trước**: không cho phép một tiến trình bị ưu tiên trước khi nó đang chạy trong kernel mode; tiến trình đó sẽ chạy cho đến khi nó thoát khỏi kernel mode.
  - Không gây tình trạng đua tranh trong cấu trúc
  - Windows 2000/XP, UNIX cũ, Linux trước phiên bản 2.6
- **kernel có ưu tiên trước**: cho phép một tiến trình bị ưu tiên trước khi nó đang chạy trong kernel mode.
  - Cần thiết kế cẩn thận để tránh tình trạng đua tranh, nhất là với kiến trúc đa xử lý đối xứng (SMP). Vì sao?
  - Thích hợp hơn với lập trình thời gian thực, vì nó sẽ cho phép 1 tiến trình thời gian thực ưu tiên trước 1 tiến trình khác đang chạy trong kernel.
  - Linux 2.6, một số phiên bản thương mại của UNIX (Solaris, IRIX)





6.3. Giải pháp của Peterson

- Giải pháp cho 2 tiến trình P_0, P_1
- Giả sử các lệnh LOAD và STORE là nguyên tử (atomic); nghĩa là không thể bị ngắt.
- Hai tiến trình chia sẻ 2 biến:
  - int turn;
  - boolean flag[2]
- Biến turn bằng 0/1. `turn==i` thì P_i được phép vào đoạn găng.
- `flag[i]=true` cho biết tiến trình P_i sẵn sàng vào đoạn găng.





Thuật toán cho tiến trình P_i

```
while (true) {
      flag[i] = TRUE;
      turn = j;
      while (flag[j] && turn == j);

            ĐOẠN_GĂNG

      flag[i] = FALSE;

            ĐOẠN_CÒN_LẠI

}
```

Chứng minh thuật toán trên thỏa mãn cả 3 điều kiện của giải pháp?





6.4. Phần cứng đồng bộ hóa

- Nhiều HĐH cung cấp sự hỗ trợ phần cứng cho mã đoạn găng
- Đơn bộ xử lý – có thể vô hiệu các ngắt
  - Đoạn mã đang chạy thực hiện mà không bị giành ưu tiên
  - Nói chung rất không hiệu quả với các hệ thống đa bộ xử lý
    - Việc chuyển thông điệp đến tất cả các bộ xử lý tốn rất nhiều thời gian, làm trễ sự vào đoạn găng của các tiến trình
- Nhiều HĐH hiện đại cung cấp các lệnh phần cứng nguyên tử
  - Nguyên tử = không thể bị ngắt
  - Hoặc là **test** từ nhớ (memory word) và **set** giá trị
  - Hoặc là hoán đổi (**swap**) nội dung của 2 từ nhớ





Lệnh TestAndSet

- Định nghĩa:

```
boolean TestAndSet (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```





Giải pháp dùng TestAndSet

- Biến boolean chia sẻ là lock, được khởi tạo là false.
- Giải pháp cho mỗi tiến trình:

```
while (true) {
        while (TestAndSet (&lock))
                ;   /* do nothing

                //    đoạn găng

        lock = FALSE;

                //      đoạn còn lại

}
```





Lệnh Swap

- Định nghĩa:

```
void Swap (boolean *a, boolean *b)
{
        boolean temp = *a;
        *a = *b;
        *b = temp;
}
```



Giải pháp dùng Swap

- Biến boolean chia sẻ là lock, được khởi tạo là false; Mỗi tiến trình có một biến boolean cục bộ là key.
- Giải pháp cho mỗi tiến trình:

```
while (true)  {
        key = TRUE;
        while (key == TRUE)
                Swap (&lock, &key);

                  //    đoạn găng

         lock = FALSE;

                  //    đoạn còn lại

 }
```


6.5. Kỹ thuật dùng cờ báo (Semaphore)

- Công cụ đồng bộ hóa dễ dùng hơn với người lập trình ứng dụng.
- Semaphore *S* – biến integer
- Hai hoạt động nguyên tử chuẩn có thể thay đổi S:
  - wait() và signal(), còn được gọi là P() và V()

```
wait (S) {
     while S <= 0
          ; // no-op
     S--;
 }

signal (S) {
   S++;
}
```


Semaphore – Công cụ đồng bộ hóa tổng quát

- Counting semaphore – giá trị S có thể không bị giới hạn
- Binary semaphore – giá trị S chỉ có thể bằng 0 hoặc 1; dễ thực hiện hơn
  - Còn được gọi là khóa loại trừ (mutex locks)
- Có thể thực thi counting semaphore S như binary semaphore
- Cung cấp sự loại trừ lẫn nhau
  - Semaphore S; // khởi tạo bằng 1
  - wait (S);
    Đoạn găng
    signal (S);

Thực thi Semaphore

- Phải đảm bảo rằng không thể có 2 tiến trình có thể thực hiện wait () và signal () trên cùng semaphore tại cùng thời điểm.
- Do đó, sự thực thi trở thành vấn đề đoạn găng: mã của wait và signal được đặt trong đoạn găng.
- Khi 1 tiến trình trong đoạn găng, các tiến trình khác cố gắng vào đoạn găng phải lặp liên tục trong mã đoạn vào, làm lãng phí các chu kỳ CPU – gọi là busy waiting.

Thực thi Semaphore không có Busy waiting

- Với mỗi semaphore có một waiting queue. Mỗi phần tử trong waiting queue có 2 trường dữ liệu:
  - value (kiểu integer)
  - pointer, con trỏ tới bản ghi kế tiếp trong list
- Hai hoạt động:
  - block – đặt tiến trình gọi vào waiting queue thích hợp
    - Khi tiến trình thực hiện wait(), nếu giá trị S không dương thì thay vì đợi busy waiting, tiến trình có thể gọi block()
    - Trạng thái tiến trình được chuyển thành *waiting*
  - wakeup – loại 1 tiến trình khỏi waiting queue và đặt nó vào ready queue.
    - Khi 1 tiến trình khác gọi signal (), tiến trình được khởi động lại bởi wakeup()

Thực thi Semaphore không có Busy waiting (tiếp)

- Sự thực thi của wait:

```
wait (S){
    value--;
    if (value < 0) {
        thêm tiến trình này vào waiting queue
        block(); }
}
```

- Sự thực thi của signal:

```
signal (S){
    value++;
    if (value <= 0) {
        loại tiến trình P khỏi waiting queue
        wakeup(P); }
}
```


Deadlock và Starvation

- Deadlock (bế tắc) – hai hoặc nhiều tiến trình đang đợi vô hạn một sự kiện chỉ có thể được gây ra bởi một trong những tiến trình đợi đó.
- Gọi **S** và **Q** là hai semaphore được khởi tạo bằng 1

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);

- Starvation – khóa vô hạn. Một tiến trình có thể không bao giờ được đưa ra khỏi waiting queue tương ứng của semaphore.
  - Có thể xuất hiện khi waiting queue tổ chức dạng LIFO

End of Chapter 6

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 7: Bế tắc (Deadlocks)

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung



Nội dung chương 7

- Mô hình hệ thống
- Mô tả bế tắc
- Các phương pháp xử lý bế tắc
- Ngăn ngừa bế tắc
- Tránh khỏi bế tắc
- Phát hiện bế tắc
- Phục hồi từ bế tắc
- Phương pháp kết hợp xử lý bế tắc





Mục tiêu

- Mô tả bế tắc, tình trạng ngăn cản các tiến trình đồng thời hoàn thành tác vụ
- Giới thiệu các phương pháp khác nhau để ngăn ngừa hoặc tránh khỏi bế tắc trong một hệ thống máy tính.





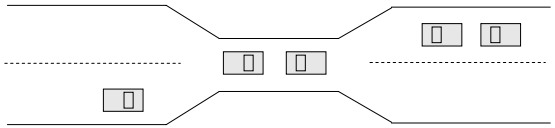
Vấn đề bế tắc (Deadlock)

- Trong môi trường đa chương trình, một số tiến trình có thể tranh nhau một số tài nguyên hạn chế.
- Một tiến trình yêu cầu các tài nguyên, nếu tài nguyên không thể đáp ứng tại thời điểm đó thì tiến trình sẽ chuyển sang trạng thái chờ.
- Các tiến trình chờ có thể sẽ không bao giờ thay đổi lại trạng thái được vì các tài nguyên mà nó yêu cầu bị giữ bởi các tiến trình chờ khác.

$\Rightarrow$ ví dụ: tắc nghẽn trên cầu



Ví dụ qua cầu



- Hai (hay nhiều hơn) ô tô đối đầu nhau trên một cây cầu hẹp chỉ đủ độ rộng cho một chiếc.
- Mỗi đoạn của cây cầu có thể xem như một tài nguyên.
- Nếu bế tắc xuất hiện, nó có thể được giải quyết nếu một hay một số ô tô lùi lại nhường đường rồi tiến sau.

7.1. Mô hình hệ thống

- Các loại tài nguyên R_1, R_2, . . ., R_m

  Các chu kỳ CPU, không gian bộ nhớ, các tệp, các thiết bị vào-ra

- Mỗi loại tài nguyên R_i có W_i cá thể (instance).
  - vd: hệ thống có 2 CPU, có 5 máy in

  $\Rightarrow$ có thể đáp ứng yêu cầu của nhiều tiến trình hơn
- Mỗi tiến trình sử dụng tài nguyên theo các bước sau:
  1. **yêu cầu** (request): nếu yêu cầu không được giải quyết ngay (vd khi tài nguyên đang được tiến trình khác sử dụng) thì tiến trình yêu cầu phải đợi cho đến khi nhận được tài nguyên.
  2. **sử dụng** (use)
  3. **giải phóng** (release)

7.2. Mô tả bế tắc

Deadlock có thể xảy ra nếu 4 điều kiện sau đồng thời tồn tại:

- **Ngăn chặn lẫn nhau:** tại một thời điểm, chỉ một tiến trình có thể sử dụng một tài nguyên.
- **Giữ và đợi:** một tiến trình đang giữ ít nhất một tài nguyên và đợi để nhận được tài nguyên khác đang được giữ bởi tiến trình khác.
- **Không có ưu tiên:** một tài nguyên chỉ có thể được tiến trình (tự nguyện!) giải phóng khi nó đã hoàn thành công việc.
- **Chờ đợi vòng tròn:** tồn tại một tập các tiến trình chờ đợi {P_0, P_1, …, P_n, P_0}
  - P_0 đang đợi tài nguyên bị giữ bởi P_1,
  - P_1 đang đợi tài nguyên bị giữ bởi P_2, **…**
  - P_{n-1} đang đợi tài nguyên bị giữ bởi P_n,
  - và P_n đang đợi tài nguyên bị giữ bởi P_0.

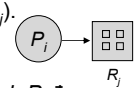
Biểu đồ phân phối tài nguyên

Một tập các đỉnh V và một tập các cạnh E.

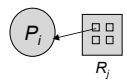
- V được chia thành 2 loại:
  - $P = \{P_1, P_2, \ldots, P_n\}$, tập tất cả các tiến trình.
  - $R = \{R_1, R_2, \ldots, R_m\}$, tập các loại tài nguyên.
    - Mỗi cá thể là một hình vuông bên trong



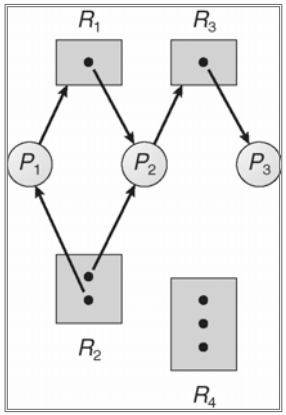
- cạnh yêu cầu – cạnh có hướng $P_i \rightarrow R_j$. *(tiến trình P_i* đang đợi nhận một hay nhiều cá thể của tài nguyên R_j).



- cạnh chỉ định – cạnh có hướng $R_j \rightarrow P_i$. *(tiến trình P_i* đang giữ một hay nhiều cá thể của tài nguyên R_j).



Vd đồ thị phân phối tài nguyên không chu trình

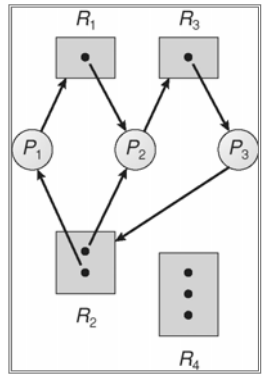


Nếu đồ thị không chu trình thì sẽ không có tiến trình nào bị bế tắc

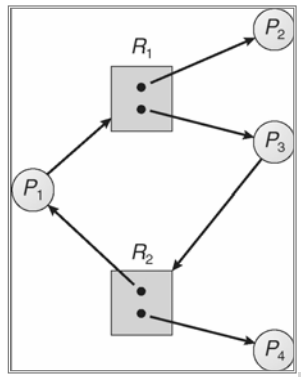
Nếu đồ thị có chu trình thì ***có thể*** tồn tại bế tắc

Vd đồ thị phân phối tài nguyên có chu trình

Bế tắc



Không bế tắc: P_4 hoặc P_2 có thể kết thúc, khiến P_1 và P_3 kết thúc được.



Kết luận về đồ thị

- Nếu đồ thị không chu trình
  $\Rightarrow$ không xảy ra bế tắc.
- Nếu đồ thị có chu trình $\Rightarrow$
  - nếu mỗi loại tài nguyên chỉ một cá thể thì ***chắc chắn*** xảy ra bế tắc.
  - nếu mỗi loại tài nguyên có một vài cá thể thì ***có thể*** xảy ra bế tắc.

7.3. Các phương pháp xử lý bế tắc

- Sử dụng một phương thức để ngăn ngừa hoặc tránh xa, đảm bảo rằng hệ thống sẽ không bao giờ đi vào trạng thái bế tắc.
- Cho phép hệ thống đi vào trạng thái bế tắc rồi khôi phục lại.
- Bỏ qua vấn đề này và vờ như bế tắc không bao giờ xuất hiện trong hệ thống. Giải pháp này được sử dụng trong hầu hết các HĐH, bao gồm cả UNIX.



7.4. Ngăn ngừa bế tắc

Ngăn cản các cách tạo yêu cầu: đảm bảo ít nhất một trong bốn điều kiện không thể xuất hiện

- **Ngăn cản lẫn nhau** – đảm bảo là hệ thống không có các file không thể chia sẻ.
  - một tiến trình không bao giờ phải chờ tài nguyên có thể chia sẻ
    - vd: read-only files
  - một số tài nguyên là không thể chia sẻ
    - vd: chế độ toàn màn hình
- **Giữ và đợi** – phải đảm bảo rằng mỗi khi một tiến trình yêu cầu một tài nguyên, nó không giữ bất kỳ tài nguyên nào khác.
  - Đòi hỏi tiến trình yêu cầu và được phân phối tất cả các tài nguyên của nó trước khi nó bắt đầu thực hiện, hoặc chỉ cho phép tiến trình yêu cầu các tài nguyên khi nó không giữ tài nguyên nào cả.
  - Hiệu quả sử dụng tài nguyên thấp, có thể xảy ra starvation.





Ngăn ngừa bế tắc (tiếp)

- **Không có ưu tiên**
  - Nếu một tiến trình đang giữ một số tài nguyên và yêu cầu tài nguyên khác mà không thể được phân phối ngay cho nó thì tất cả các tài nguyên nó đang giữ được giải phóng.
  - Các tài nguyên ưu tiên được thêm vào danh sách tài nguyên dành cho tiến trình đang chờ đợi.
  - Tiến trình sẽ được khởi động lại chỉ khi nó có thể lấy lại các tài nguyên cũ của nó cũng như các tài nguyên mới mà nó đang yêu cầu.
- **Chờ đợi vòng tròn** – áp dụng một thứ tự tuyệt đối cho tất cả các loại tài nguyên: mỗi loại được gắn một số nguyên
  - mỗi tiến trình yêu cầu các tài nguyên theo thứ tự tăng dần: chỉ có thể nhận được tài nguyên có trọng số cao hơn của bất kỳ tài nguyên nào nó đang giữ
  - $\Rightarrow$ Muốn có tài nguyên *i*, tiến trình phải giải phóng tất cả các tài nguyên có trọng số $j > i$ (nếu có)





7.5. Tránh khỏi bế tắc

Yêu cầu HĐH phải có một số thông tin ưu tiên

- Mô hình hữu dụng nhất và đơn giản nhất yêu cầu mỗi tiến trình công bố *số lượng tài nguyên lớn nhất* của mỗi loại mà nó có thể cần đến.
- Giải thuật tránh bế tắc luôn kiểm tra trạng thái phân phối tài nguyên để đảm bảo rằng sẽ không bao giờ có tình trạng chờ đợi vòng tròn.
- Trạng thái phân phối tài nguyên được xác định bởi số tài nguyên khả dụng và đã được phân phối cũng như sự yêu cầu tối đa từ các tiến trình.





7.5.1. Safe State

- Một trạng thái là an toàn nếu hệ thống có thể phân phối các tài nguyên cho mỗi tiến trình mà vẫn tránh được bế tắc.
- Khi một tiến trình yêu cầu một tài nguyên còn rỗi, hệ thống phải quyết định liệu phân phối ngay lập tức có làm cho hệ thống mất an toàn hay không?
- Hệ thống ở trong trạng thái an toàn nếu tồn tại một chuỗi an toàn của tất cả các tiến trình.



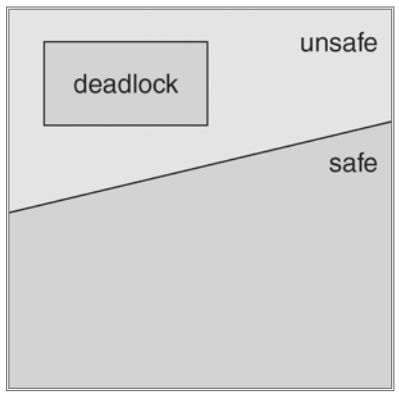
Safe State (tiếp)

- Chuỗi <P1, P2, …, Pn> là an toàn nếu với mỗi Pi, tài nguyên mà nó yêu cầu có thể được cung cấp bởi tài nguyên khả dụng hiện tại và các tài nguyên đang được giữ bởi Pj, với j<i.
  - Nếu tài nguyên Pi cần đang bị Pj giữ thì nó có thể đợi cho đến khi tất cả các Pj kết thúc.
  - Khi Pj kết thúc, Pi có thể giành được các tài nguyên cần thiết, thực hiện, rồi trả lại các tài nguyên đó và kết thúc.
  - Khi Pi kết thúc, P(i+1) có thể giành được tài nguyên cần thiết, …

Safe State: thực tế dễ nhận

- Nếu hệ thống ở trạng thái an toàn ⇒ không có bế tắc.
- Nếu hệ thống ở trạng thái không an toàn ⇒ có thể có bế tắc.
- Sự tránh khỏi bế tắc ⇒ đảm bảo rằng hệ thống sẽ không bao giờ bước vào trạng thái không an toàn.
  - Mỗi loại tài nguyên có một cá thể: giải thuật đồ thị phân phối tài nguyên.
  - Mỗi loại tài nguyên có nhiều cá thể: giải thuật chủ nhà băng.

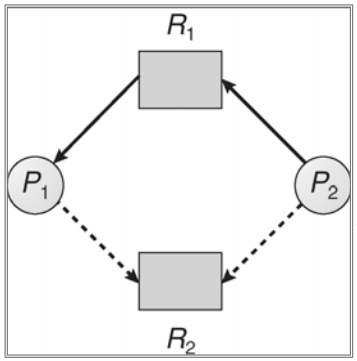
Safe, Unsafe, Deadlock State



7.5.2. Giải thuật đồ thị phân phối tài nguyên

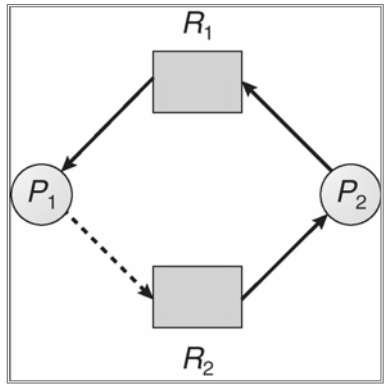
- *Cạnh muốn yêu cầu* (*claim edge*) $P_i \rightarrow R_j$: tiến trình P_j có thể yêu cầu tài nguyên R_j; được biểu diễn bởi một đường đứt nét.
- Cạnh muốn yêu cầu biến thành cạnh yêu cầu (request edge) khi một tiến trình yêu cầu một tài nguyên.
- Khi tài nguyên được một tiến trình giải phóng, cạnh yêu cầu trở lại thành cạnh muốn yêu cầu.
- Hệ thống ở trong trạng thái an toàn miễn là đồ thị không chứa chu trình nào.
  - Chúng ta coi các cạnh muốn yêu cầu như là các cạnh yêu cầu

Giải thuật đồ thị phân phối tài nguyên tránh khỏi bế tắc



Giả sử P_2 yêu cầu R_2. Dù R_2 vẫn đang tự do, chúng ta vẫn không thể phân phối nó cho P_2 vì sẽ tạo ra một chu trình → hệ thống trong trạng thái không an toàn. Nếu P_1 yêu cầu R_2 và P_2 yêu cầu R_1 thì bế tắc sẽ xuất hiện.

Trạng thái không an toàn trong đồ thị phân phối tài nguyên



7.5.3. Giải thuật chủ nhà băng

- Có tên như trên vì giải thuật này có thể được sử dụng trong hệ thống nhà băng để đảm bảo rằng nhà băng không bao giờ phân phối quá số tiền khả dụng của nó đến mức mà nó có thể thỏa mãn mọi yêu cầu từ các khách hàng.
- Khi một tiến trình mới đi vào hệ thống, nó phải khai báo số lượng tối đa cá thể của mỗi loại tài nguyên mà nó có thể cần đến. Số này có thể vượt quá tổng tài nguyên trong hệ thống.
- Khi user yêu cầu tài nguyên, hệ thống phải xác định liệu sự phân phối có giữ hệ thống trong trạng thái an toàn không:
  - Nếu có → phân phối tài nguyên
  - Nếu không → tiến trình phải chờ đến khi các tiến trình khác giải phóng đủ tài nguyên.

Giải thuật chủ nhà băng: cấu trúc dữ liệu

- ***n*** = số tiến trình.
- ***m*** = số loại tài nguyên.
- ***Available****:* vectơ độ dài *m* – các tài nguyên khả dụng của mỗi loại.
  - Available[*j*] = *k*: có *k* cá thể của loại tài nguyên R_j là khả dụng.
- ***Max****: ma trận n x m:* xác định số tối đa yêu cầu của mỗi tiến trình.
  - *Max*[*i,j*] = *k*: tiến trình P_i có thể yêu cầu tối đa tối đa k cá thể của loại tài nguyên R_j.
- ***Allocation****: ma trận n* x *m:* xác định số tài nguyên mỗi loại hiện đang phân phối cho mỗi tiến trình.
  - Allocation[*i,j*] = *k*: P_i hiện đang được phân phối *k* cá thể của R_j.
- ***Need****: ma trận n* x *m: xác định số tài nguyên còn thiếu cho mỗi tiến trình*
  - *Need*[*i,j*] = *k*: P_i có thể cần *k* cá thể nữa của R_j:

    Need[*i,j*] = *Max*[*i,j*] - *Allocation* [*i,j*].

Giải thuật chủ nhà băng: Kiểm tra an toàn

Tư tưởng: chúng ta tìm một chuỗi an toàn. Nếu tìm được, trạng thái là an toàn, trái lại trạng thái là không an toàn.

1. Gán *Work* và *Finish* là các vectơ độ dài *m* và *n*. Khởi tạo:

   Work := *Available*

   Finish[*i*] := (*Allocation*$_i$ == 0) với *i* = 1,2, …, *n*.

2. Tìm *i* thỏa mãn cả 2 điều kiện:

   (a) *Finish*[*i*] = *false* và

   (b) *Need*$_i \le$ *Work*

   Nếu không có *i* như vậy, nhảy đến bước 4.

   Tìm P_i chưa kết thúc và có nhu cầu không vượt quá khả dụng, nếu có thì phân phối tài nguyên cho nó.

3. *Work* := *Work* + *Allocation*$_i$
   Finish[*i*] := *true*
   nhảy đến bước 2.

   Lượng khả dụng được cộng thêm số tài nguyên đã phân phối cho P_i vì P_i đã có đủ tài nguyên để thực hiện rồi kết thúc

4. Nếu *Finish*[*i*] = true với ∀i thì hệ thống ở trạng thái an toàn.

Giải thuật yêu cầu tài nguyên cho tiến trình P_i

Request = vectơ yêu cầu cho tiến trình P_i. Nếu *Request*$_i$[*j*] = *k* thì tiến trình P_i muốn *k* cá thể của loại tài nguyên R_j.

1. Nếu *Request*$_i \le$ *Need*$_i$, chuyển sang bước 2. Trái lại, dựng lên trạng thái lỗi vì tiến trình đã vượt quá yêu cầu tối đa của nó.
2. Nếu *Request*$_i \le$ *Available*, chuyển sang bước 3. Trái lại P_i phải đợi vì tài nguyên chưa sẵn sàng.
3. Giả vờ phân phối các tài nguyên cho P_i bằng cách sửa trạng thái như sau:

   Available = *Available* – *Request*$_i$;

   Allocation$_i$ = *Allocation*$_i$ + *Request*$_i$;

   Need$_i$ = *Need*$_i$ – *Request*$_i$;

   - *Nếu an toàn* ⇒ *phân phối tài nguyên cho* P_i.
   - *Nếu không an toàn* ⇒ P_i *phải đợi, và trạng thái phân phối tài nguyên cũ được phục hồi.*

Ví dụ giải thuật chủ nhà băng

- 5 tiến trình P_0 … P_4;

  3 loại tài nguyên *A* (10 cá thể), *B* (5 cá thể), và *C* (7 cá thể).

- Giả sử tại thời điểm T_0:

  = Max - Allocation

	Max	*Allocation*	*Need*	*Available*
	A B C	*A B C*	*A B C*	*A B C*
P_0	7 5 3	0 1 0	7 4 3	3 3 2
P_1	3 2 2	2 0 0	1 2 2	
P_2	9 0 2	3 0 2	6 0 0	
P_3	2 2 2	2 1 1	0 1 1	
P_4	4 3 3	0 0 2	4 3 1	

  = 10 – (0+2+3+2+0)
  = W_A - Σ Allocation$_A$

- Hệ thống có đang ở trạng thái an toàn?

Ví dụ (tiếp)

- Áp dụng giải thuật kiểm tra an toàn:

1. Work = Available, Finish[i] = false với mọi i vì Allocation[i] ≠ 0
2. Tìm thấy i=1 thỏa mãn Need$_1$ (1 2 2) ≤ Work (3 3 2)
3. Work = Work + Allocation$_1$ = (3 3 2) + (2 0 0) = (5 3 2)
   Finish[1] = true (đánh dấu tiến trình P1 kết thúc được)
2. Tìm thấy i=3 thỏa mãn Need$_3$ (0 1 1) ≤ Work (5 3 2)
3. Work = Work + Allocation$_3$ = (5 3 2) + (2 1 1) = (7 4 3)
   Finish[3] = true

   ……..

- Hệ thống đang ở trạng thái an toàn vì chuỗi <P_1, P_3, P_4, P_2, P_0> thỏa mãn các điều kiện an toàn.

Ví dụ P_1 yêu cầu (1,0,2)

- Kiểm tra Request ≤ Need_1 ↔ (1,0,2) ≤ (1,2,2) ⇒ true.
- Kiểm tra Request ≤ Available ↔ (1,0,2) ≤ (3,3,2) ⇒ true. Giả vờ đáp ứng yêu cầu, hệ thống sẽ đến trạng thái sau:

	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 1	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Việc thực hiện giải thuật kiểm tra an toàn cho thấy chuỗi <P1, P3, P4, P0, P2> vẫn thỏa mãn các yêu cầu an toàn → có thể chấp nhận ngay yêu cầu từ P1
- Có thể chấp nhận yêu cầu (3,3,0) từ P4?
- Có thể chấp nhận yêu cầu (0,2,0) từ P0?

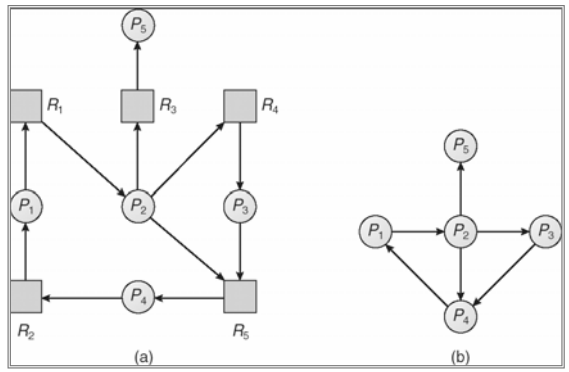
7.6. Phát hiện bế tắc

- Nếu một hệ thống không thể thực hiện được việc ngăn ngừa hay tránh xa bế tắc thì bế tắc có thể xuất hiện. Trong môi trường này, hệ thống phải cung cấp:
  - Giải thuật phát hiện bế tắc
  - Giải thuật phục hồi từ bế tắc

Mỗi loại tài nguyên có 1 cá thể

- Khi tất cả tài nguyên chỉ có một cá thể, giải thuật xác định bế tắc sử dụng một biến thể của đồ thị phân phối tài nguyên, bằng cách bỏ đi các nút của loại tài nguyên và bỏ đi các cạnh thích hợp ⇒ đồ thị *wait-for*
  - Các nút là các tiến trình.
  - $P_i \rightarrow P_j$ nếu P_i đang đợi P_j.
- Định kỳ sử dụng giải thuật tìm kiếm chu trình trong đồ thị. Giải thuật đó đòi hỏi n^2 phép toán, với n là số đỉnh trong đồ thị: có chu trình → có thể có bế tắc.

Resource-Allocation Graph and Wait-for Graph



Đồ thị phân phối tài nguyên Đồ thị wait-for tương ứng

Mỗi loại tài nguyên có nhiều cá thể

- *Available:* vectơ độ dài *m* xác định số tài nguyên khả dụng của mỗi loại.
- *Allocation:* ma trận *n* x *m* xác định các tài nguyên của mỗi loại hiện đang được phân phối cho mỗi tiến trình.
- *Request:* ma trận *n* x *m* xác định yêu cầu hiện tại của mỗi tiến trình. Nếu *Request* [*i,j*] = *k*, thì tiến trình P_i đang yêu cầu *k* cá thể nữa của loại tài nguyên R_j.

Giải thuật phát hiện bế tắc

1. Gán *Work* và *Finish* là các vectơ độ dài *m* và *n*. Khởi tạo:
   (a) *Work* := *Available*
   (b) For *i* := 1 to n do If $Allocation_i \neq 0$ then *Finish*[i] := false
   Else *Finish*[i] := *true*
2. Tìm chỉ số *i* thỏa mãn cả 2 điều kiện:
   (a) *Finish*[*i*] = *false*
   (b) $Request_i \leq Work$
   nếu không tồn tại *i*, nhảy sang bước 4.
3. *Work* := *Work* + $Allocation_i$
   Finish[*i*] := *true*
   nhảy sang bước 2.
4. Nếu *Finish*[*i*] = true với ∀i thì hệ thống không có bế tắc
   Nếu *Finish*[*i*] = false, với một số i, $1 \leq i \leq n$, thì P_i bị bế tắc, hệ thống ở trong trạng thái bế tắc.

Độ phức tạp tính toán của giải thuật là $O(m \times n^2)$

Ví dụ giải thuật phát hiện bế tắc

- 5 tiến trình P_0 … P_4;
  3 loại tài nguyên: *A* (7 cá thể), *B* (2 cá thể), và *C* (6 cá thể).
- Giả sử hệ thống tại thời điểm T_0:

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Chuỗi <P_0, P_2, P_3, P_1, P_4> sẽ cho kết quả *Finish*[*i*] = true với tất cả các $i \rightarrow$ không có bế tắc.

Ví dụ (tiếp)

- P_2 yêu cầu thêm 1 cá thể loại *C (0 0 1)*.

	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

- Trạng thái của hệ thống?
  - Có thể phục hồi các tài nguyên bị giữ bởi tiến trình P_0 khi nó kết thúc, nhưng không đủ tài nguyên để hoàn thành các tiến trình khác.
  - Bế tắc xuất hiện, gồm các tiến trình P_1, P_2, P_3, và P_4.



Cách sử dụng giải thuật

- Thời điểm và mức thường xuyên cần đến giải thuật phụ thuộc:
  - Bế tắc có khả năng thường xuyên xảy ra như thế nào?
  - Có bao nhiêu tiến trình bị tác động khi bế tắc xuất hiện
- Nếu giải thuật phát hiện bế tắc ít được sử dụng, có thể có nhiều chu trình trong biểu đồ tài nguyên và do đó ta không thể tìm được những tiến trình nào "gây ra" bế tắc.
- Nếu phát hiện được bế tắc, chúng ta cần phục hồi lại bằng một trong hai cách:
  - Dừng các tiến trình
  - Buộc chúng phải giải phóng tài nguyên (ưu tiên trước)





7.7. Phục hồi từ bế tắc
7.7.1. Dừng tiến trình

- Hủy bỏ tất cả các tiến trình bị bế tắc (có *Finish[i] = false*).
- Hủy bỏ một tiến trình tại một thời điểm đến khi chu trình bế tắc được loại trừ.
- Chúng ta nên chọn hủy bỏ theo trình tự nào?
  - Theo mức ưu tiên của tiến trình.
  - Theo thời gian tiến trình đã thực hiện, và thời gian cần thiết còn lại để hoàn thành.
  - Theo tài nguyên tiến trình đã sử dụng.
  - Theo tài nguyên tiến trình cần để hoàn thành.
  - Theo số tiến trình sẽ cần bị dừng.
  - Tiến trình là tiến trình tương tác hay tiến trình bó?





7.7.2. Ưu tiên trước tài nguyên

- Chọn một tiến trình nạn nhân dựa vào giá trị nhỏ nhất (mức ưu tiên, số tài nguyên đang dùng…).
- Rollback – quay lại trạng thái an toàn trước, khởi động lại tiến trình ở trạng thái đó.
- Starvation – 1 tiến trình có thể luôn bị chọn làm nạn nhân khiến nó không thể kết thúc. Phải đảm bảo rằng một tiến trình được chọn làm nạn nhân chỉ trong khoảng thời gian ngắn.
  - Giải pháp: Thêm các rollback vào yếu tố chi phí.





7.8. Phương pháp kết hợp xử lý bế tắc

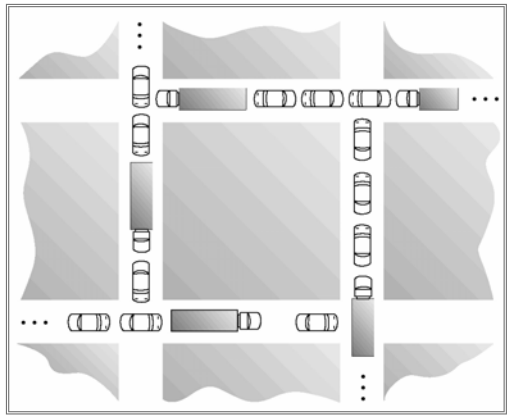
- Kết hợp 3 phương pháp cơ bản
  - ngăn ngừa - prevention
  - tránh khỏi - avoidance
  - phát hiện - detection

  tạo thành phương pháp tối ưu đối với mỗi tài nguyên trong hệ thống.
- Phân chia các tài nguyên thành các lớp theo thứ tự phân cấp.
- Sử dụng kỹ thuật thích hợp nhất để xử lý bế tắc trong mỗi lớp.





Traffic Deadlock





End of Chapter 7

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 8: Bộ nhớ chính

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
DĐ: 0979784189
Website: fita.hua.edu.vn/pqdung



Nội dung chương 8

- Kiến thức cơ bản
- Swapping
- Phân phối bộ nhớ liên tiếp - Contiguous Allocation
- Phân trang - Paging
- Phân đoạn - Segmentation
- Kết hợp phân đoạn với phân trang
  - Segmentation with Paging
- Ví dụ: Intel Pentium





Mục tiêu

- Cung cấp mô tả chi tiết các cách tổ chức phần cứng bộ nhớ.
- Thảo luận các kỹ thuật quản lý bộ nhớ khác nhau, bao gồm phân trang và phân đoạn.
- Cung cấp mô tả chi tiết về Intel Pentium, bộ xử lý hỗ trợ cả phân đoạn đơn thuần và phân đoạn kết hợp với phân trang.





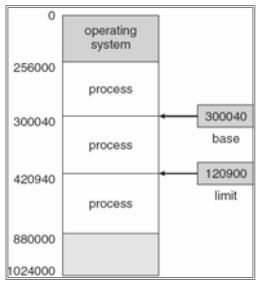
8.1. Kiến thức cơ bản
8.1.1. Về phần cứng

- Chương trình phải được đưa từ đĩa vào bộ nhớ chính và được đặt vào một tiến trình để nó có thể chạy.
- Bộ nhớ chính, cache, các thanh ghi là bộ nhớ mà CPU có thể truy nhập trực tiếp.
- Yêu cầu bộ nhớ phải được bảo vệ để đảm bảo sự hoạt động đúng đắn.

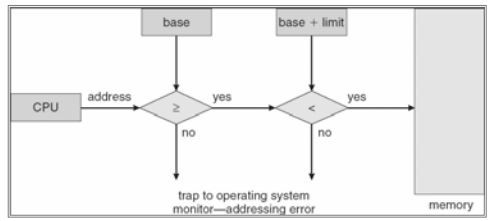


Các thanh ghi base và limit

- Cặp thanh ghi base và limit xác định không gian địa chỉ hợp lệ mà tiến trình của người sử dụng được phép truy nhập.



Bảo vệ bộ nhớ với các thanh ghi base & limit



Sự bảo vệ bộ nhớ được thực hiện bằng cách so sánh mọi địa chỉ trong user mode với các thanh ghi base và limit. Nếu địa chỉ đó nằm ngoài khoảng địa chỉ xác định bởi 2 thanh ghi thì mắc bẫy chuyển sang monitor mode.

Sử dụng 2 thanh ghi base và limit (tiếp)

- Các lệnh nạp các thanh ghi *base* và *limit* là các lệnh đặc quyền.
- Khi thực hiện trong monitor mode, HĐH không hạn chế truy nhập bộ nhớ.

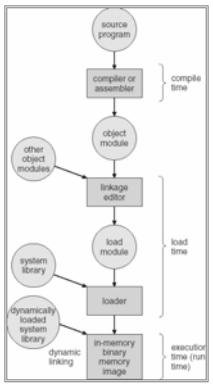
  ⇒ cho phép HĐH nạp chương trình của người sử dụng vào bộ nhớ, đưa các chương trình đó ra ngoài khi có lỗi.

8.1.2.Liên kết các lệnh và dữ liệu tới bộ nhớ

Sự liên kết địa chỉ của các lệnh và dữ liệu (của tiến trình) tới các địa chỉ bộ nhớ có thể xảy ra 3 giai đoạn khác nhau:

- **Compile time**: Nếu vị trí bộ nhớ được biết trước, **mã chính xác** (*absolute code*) có thể được sinh ra; phải biên dịch lại mã nếu vị trí bắt đầu thay đổi. vd chương trình .COM của MS DOS.
- **Load time**: Phải sinh ra **mã có thể tái định vị** (*relocatable code*) nếu vị trí bộ nhớ không được biết ở giai đoạn biên dịch.
- **Execution time**: Sự liên kết bị hoãn lại đến giai đoạn chạy nếu trong quá trình thực hiện tiến trình có thể bị chuyển từ một đoạn bộ nhớ đến đoạn khác. Cần có sự hỗ trợ phần cứng để ánh xạ địa chỉ (ví dụ, *base* và *limit registers*). Hầu hết các HĐH đa năng sử dụng phương pháp này.

Các bước xử lý chương trình người sử dụng



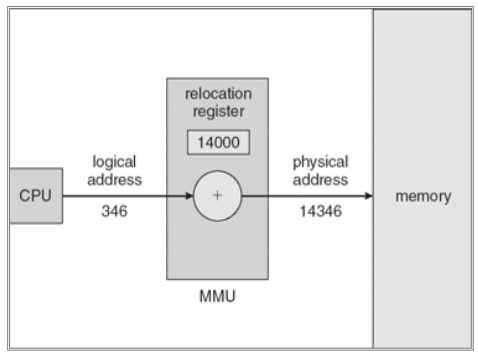
8.1.3. Logical vs. Physical Address Space

- Khái niệm *không gian địa chỉ logic (logical address space)* được tách riêng với *không gian địa chỉ vật lý (physical address space)* để quản lý bộ nhớ thích hợp.
  - *Logical address* – được tạo ra bởi CPU; còn được gọi là địa chỉ ảo (*virtual address)*.
  - *Physical address* – địa chỉ được nhận biết bởi đơn vị quản lý bộ nhớ (memory unit).
- Các địa chỉ logic (ảo) và vật lý là như nhau trong các giai đoạn liên kết địa chỉ compile-time và load-time; chúng khác nhau trong execution-time.

Memory-Management Unit (MMU)

- Là thiết bị phần cứng ánh xạ địa chỉ ảo tới địa chỉ vật lý.
- Trong lược đồ MMU, giá trị trong thanh ghi định vị (relocation register) được cộng với tất cả địa chỉ được sinh ra bởi tiến trình của người sử dụng tại thời điểm nó được gửi tới bộ nhớ.
- Chương trình của người sử dụng làm việc với các địa chỉ logic; nó không bao giờ nhận ra các địa chỉ vật lý thực.

Định vị động sử dụng thanh ghi định vị



8.1.4. Dynamic Loading

- Tiến trình chỉ được nạp vào bộ nhớ khi nó được gọi.
- Sử dụng không gian bộ nhớ tốt hơn; tiến trình không dùng đến thì không bao giờ được nạp.
- Hữu ích trong trường hợp số lượng lớn mã cần xử lý nhưng hiếm khi xuất hiện.
- Không yêu cầu sự hỗ trợ đặc biệt từ HĐH, được thực hiện thông qua thiết kế chương trình.

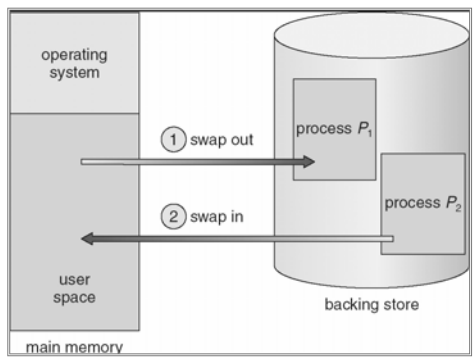
8.1.5. Dynamic Linking

- Việc liên kết hoãn lại đến execution time.
- Đoạn mã nhỏ, *stub*, được sử dụng để định vị thường trình thư viện cư trú trong bộ nhớ (memory-resident library routine) thích hợp, hoặc để nạp thư viện nếu thường trình hiện tại chưa sẵn sàng.
- Khi được thực hiện, stub kiểm tra thường trình cần đến có trong bộ nhớ của tiến trình chưa,
  - **nếu chưa** thì chương trình nạp thường trình vào bộ nhớ;
  - **nếu rồi**: stub tự thay thế chính nó bởi địa chỉ của thường trình rồi thực hiện thường trình đó.
- Liên kết động đặc biệt hữu dụng đối với các thư viện chương trình, nhất là trong việc cập nhật thư viện (vd sửa lỗi)

8.2. Swapping

- Một tiến trình có thể được tạm thời đưa ra khỏi bộ nhớ tới *backing store* (swap out), và sau đó được đưa trở lại bộ nhớ để thực hiện tiếp (swap in).
- Backing store – đĩa tốc độ nhanh, đủ lớn để lưu trữ bản sao của tất cả hình ảnh bộ nhớ cho tất cả người sử dụng; phải cung cấp sự truy nhập trực tiếp tới các hình ảnh bộ nhớ này.
- Hệ thống duy trì 1 ready queue chứa các tiến trình sẵn sàng chạy có ảnh bộ nhớ được chứa trên backing store hoặc trong bộ nhớ.
- Khi trình lập lịch CPU quyết định thực hiện 1 tiến trình, nó gọi trình điều vận.
- Trình điều vận kiểm tra tiến trình tiếp theo trong queue có trong bộ nhớ chưa, nếu chưa có và không còn vùng nhớ rỗi đủ lớn, nó đưa 1 tiến trình trong bộ nhớ ra backing store và đưa tiến trình mong muốn vào.

Giản đồ Swapping



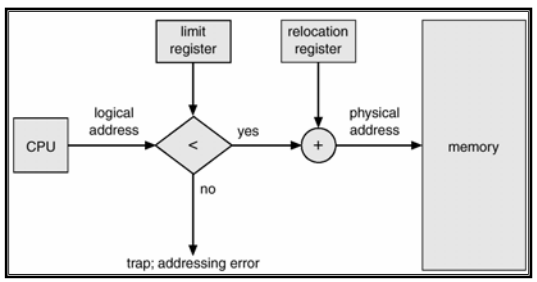
Swapping (tiếp)

- Phần lớn thời gian hoán đổi là thời gian chuyển dữ liệu; tổng thời gian chuyển tỷ lệ thuận với dung lượng bộ nhớ hoán đổi.
  - Vd: giả sử dung lượng tiến trình người sử dụng là 10 MB
    backing store là đĩa cứng có tốc độ truyền dữ liệu là 40 MB/s
    Thời gian truyền 1 chiều từ/đến bộ nhớ = 1/4 s = 250 ms
    giả sử trễ trung bình = 8 ms ⇒ Tổng truyền 1 chiều = 258 ms
    ⇒ Tổng swap in + swap out = 258 x 2 = 516 ms.
- Roll out, roll in – biến thể hoán đổi được sử dụng cho thuật giải lập lịch dựa trên mức ưu tiên (priority-based scheduling); tiến trình có mức ưu tiên thấp hơn bị thay ra để tiến trình có mức ưu tiên cao hơn có thể được nạp và thực hiện.
- Sự hoán đổi khác nhau ở các HĐH UNIX, Linux, Windows.

8.3. Phân phối bộ nhớ liên tiếp

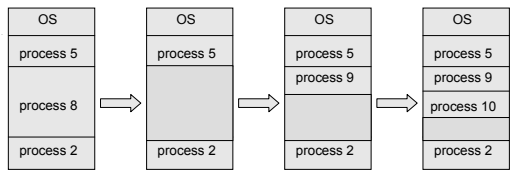
- Bộ nhớ chính được chia thành 2 phần:
  - Nơi HĐH cư trú, thường ở vùng nhớ thấp, chứa bảng vector ngắt.
  - Các tiến trình của người sử dụng được chứa trong vùng nhớ cao.
- Phân phối đơn partition (Single-partition allocation)
  - Các thanh ghi định vị được sử dụng để bảo vệ các tiến trình của người sử dụng không ảnh hưởng lẫn nhau và không thay đổi dữ liệu và mã HĐH.
  - Relocation register chứa giá trị địa chỉ vật lý nhỏ nhất; limit register chứa dải các địa chỉ logic - mỗi địa chỉ logic phải nhỏ hơn limit register.
  - MMU ánh xạ địa chỉ logic *theo cách động*.

Ví dụ các thanh ghi Relocation và Limit



Phân phối liên tiếp (tiếp)

- Phân phối đa partition (Multiple-partition allocation)
  - *Hole* – khối bộ nhớ khả dụng; các hole có kích thước khác nhau nằm rải rác khắp bộ nhớ.
  - Khi một tiến trình đến, nó được phân phối cho một hole đủ lớn.
  - HĐH duy trì thông tin về:
    a) các vùng nhớ đã được phân phối - allocated partitions
    b) các vùng nhớ còn tự do - free partitions (hole)



Bài toán phân phối vùng nhớ động

Làm thế nào để đáp ứng một yêu cầu kích thước *n* từ danh sách các free holes.

- First-fit: Phân phối *hole đầu tiên* có đủ độ lớn.
- Best-fit: Phân phối *hole nhỏ nhất* có đủ độ lớn; phải tìm kiếm toàn bộ danh sách, trừ khi DS được sắp xếp theo dung lượng.
  - Tạo ra các hole còn lại nhỏ nhất
- Worst-fit: Phân phối *hole lớn nhất*; cũng phải tìm kiếm toàn bộ danh sách.
  - Tạo ra các hole còn lại lớn nhất

First-fit và best-fit tốt hơn worst-fit về mặt tốc độ và sử dụng bộ nhớ.

Sự phân mảnh - Fragmentation

- **External Fragmentation** – tổng không gian bộ nhớ tự do thực tế đủ đáp ứng yêu cầu, nhưng nó không nằm kề nhau.
- **Internal Fragmentation** – bộ nhớ được phân phối có thể lớn hơn không đáng kể so với bộ nhớ được yêu cầu; sự khác biệt kích thước này là bộ nhớ bên trong một partition, nhưng không được sử dụng.
- Có thể làm giảm external fragmentation bằng cách nén lại (compaction)
  - Di chuyển các nội dung bộ nhớ để đặt tất cả các vùng nhớ tự do lại với nhau thành một khối lớn.
  - Kết khối chỉ có thể tiến hành nếu sự tái định vị là động, và nó được thực hiện trong execution time.
- 1 phương pháp khác là phân phối bộ nhớ không liên tục.

8.4. Phân trang - Paging

- Không gian địa chỉ logic của một tiến trình có thể không kề nhau; tiến trình được phân phối bộ nhớ vật lý bất kỳ lúc nào sau đó khi bộ nhớ sẵn có.
- Chia bộ nhớ vật lý thành những khối có kích thước cố định là bội số của 2 (512 bytes - 16 MB),được gọi là các **frame.**
- Chia bộ nhớ logic thành các khối cùng kích thước - các **page**.
- Luôn theo dõi tất cả các frame còn trống.
- Để chạy một chương trình có kích thước *n* pages, cần phải tìm *n* frames còn trống và nạp chương trình.
- Thiết lập một bảng phân trang (page table) để biên dịch (translate) các địa chỉ logic thành địa chỉ vật lý.
- Sự phân đoạn diễn ra bên trong (Internal fragmentation).

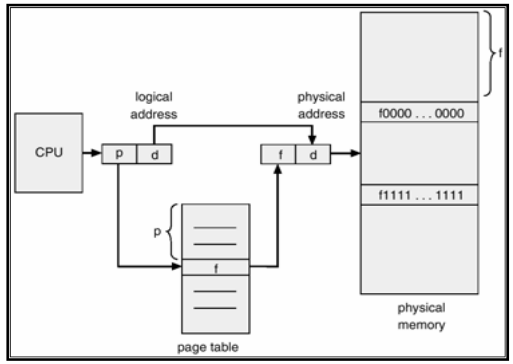
Lược đồ biên dịch địa chỉ

- Địa chỉ được tạo ra bởi CPU được chia thành:
  - *Page number (p)* – được sử dụng làm index tới *page table*, chứa địa chỉ cơ sở (base address) của mỗi trang trong bộ nhớ vật lý.
  - *Page offset (d)* – kết hợp với địa chỉ cơ sở để xác định địa chỉ bộ nhớ vật lý.

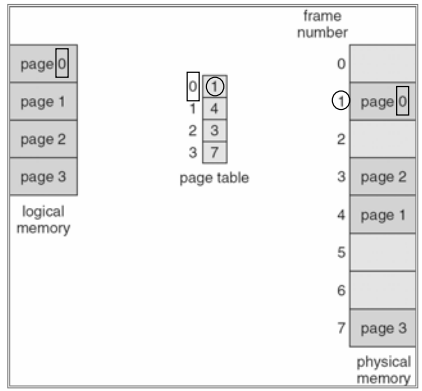
page number	page offset
p	*d*
m - n	*n*

  - 2^n = kích thước của 1 page (byte hoặc word)
  - 2^m = kích thước không gian địa chỉ logic

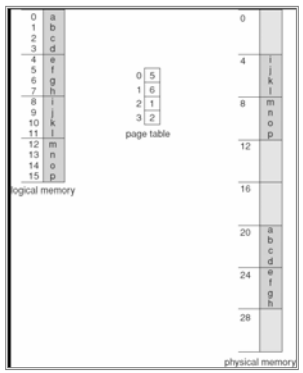
Lược đồ dịch địa chỉ (tiếp)



Mô hình phân trang



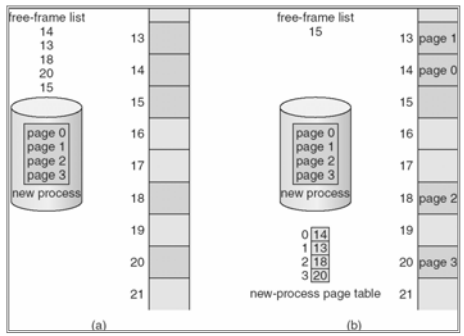
Ví dụ phân trang



Vd: Địa chỉ logic 4 (page 1, offset 0); theo bảng phân trang, trang 1 được ánh xạ tới frame 6 ⇒ địa chỉ logic ánh xạ tới địa chỉ vật lý = 6 x 4 + 0 = 24

32-byte memory and 4-byte pages

Các Frame rỗi



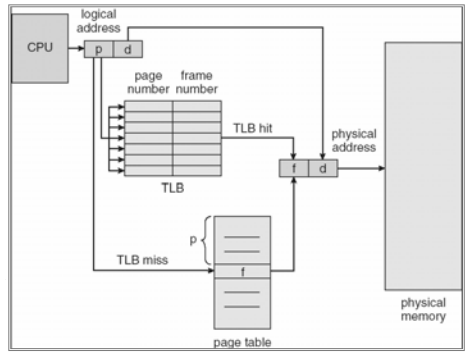
Trước khi phân phối

Sau khi phân phối

Sự thực thi của Page Table

- Page table được lưu trong bộ nhớ chính.
- *Page-table base register (*PTBR) chỉ tới page table.
- *Page-table length register* (PRLR) cho biết kích thước của page table.
- Trong lược đồ phân trang, mọi sự truy nhập lệnh/dữ liệu yêu cầu 2 lần truy nhập bộ nhớ: một cho page table và một cho lệnh/dữ liệu → truy nhập chậm hơn.
- Vấn đề 2 lần truy nhập bộ nhớ có thể được giải quyết bằng cách sử dụng một bộ nhớ cache tra cứu nhanh đặc biệt gọi là *bộ nhớ liên kết - associative memory (TLB).*

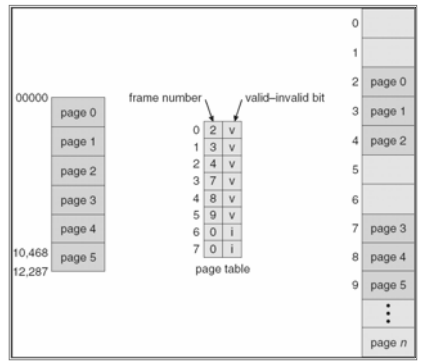
Phân trang với TLB



Bảo vệ bộ nhớ

- Sự bảo vệ bộ nhớ được thực thi bằng cách kết hợp protection bit với mỗi frame
- Mỗi phần tử trong bảng phân trang được gắn thêm một **valid–invalid bit**
  - "valid" (=1) cho biết trang tương ứng ở trong không gian địa chỉ logic của tiến trình, và do đó là một trang hợp lệ.
  - "invalid" (=0) cho biết rằng trang không có trong không gian địa chỉ logic.

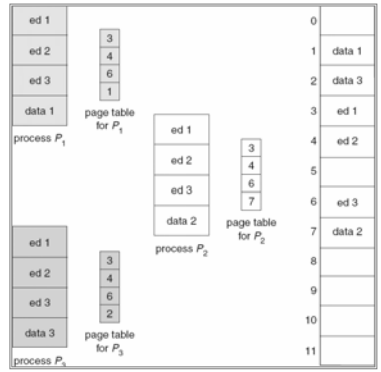
Valid (v) - Invalid (i) Bit trong bảng phân trang



Các trang chia sẻ

- Một lợi điểm của phân trang là khả năng chia sẻ mã chung, đặc biệt quan trọng trong môi trường chia sẻ thời gian.
- **Shared code**
  - Một bản copy của read-only code được chia sẻ giữa các tiến trình (i.e., text editors, compilers, window systems).
  - Mã chia sẻ phải xuất hiện tại cùng vị trí trong không gian địa chỉ logic của tất cả các tiến trình.
- **Private code and data**
  - Mỗi tiến trình giữ một bản copy mã và dữ liệu riêng.
  - Các trang của mã và dữ liệu riêng có thể xuất hiện tại bất kỳ đâu trong không gian địa chỉ logic.

Shared Pages Example



Vd: hệ thống hỗ trợ 40 user, mỗi user thực hiện một text editor gồm 150KB mã và 50KB dữ liệu.

- Nếu không chia sẻ dữ liệu, cần 8000KB.

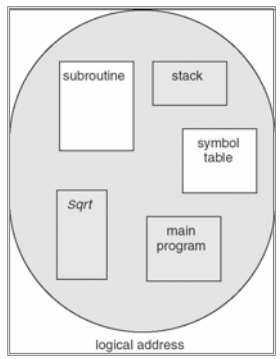
- Nếu chia sẻ 150KB mã, chỉ tốn 150 + 40 x 50 = 2150 KB bộ nhớ.

8.5. Phân đoạn - Segmentation

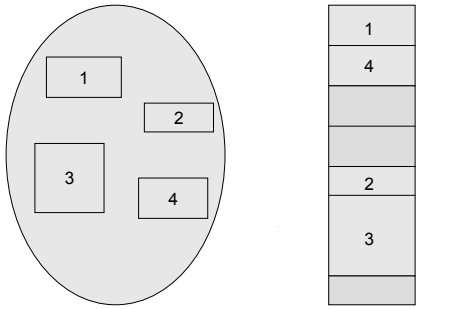
- Là lược đồ quản lý bộ nhớ hỗ trợ cách nhìn của người sử dụng đối với bộ nhớ:
- Mỗi chương trình là một tập hợp các đoạn. Mỗi đoạn là một đơn vị logic như:

  main program,
  procedure,
  function,
  method,
  object,
  local variables, global variables,
  common block,
  stack,
  symbol table, arrays

Chương trình dưới góc nhìn của người sử dụng



Góc nhìn logic của sự phân đoạn



user space physical memory space

Kiến trúc phân đoạn

- Địa chỉ logic gồm 2 thành phần:

  <segment-number, offset>,

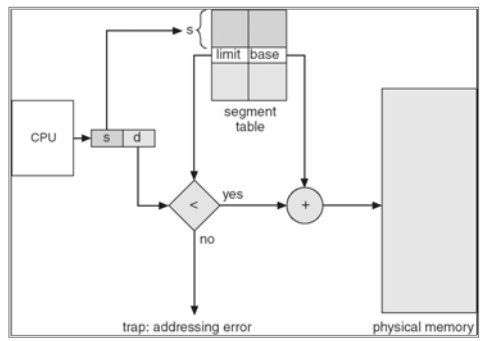
- *Segment table* – tương tự bảng phân trang, mỗi mục của bảng gồm có:
  - **base** – chứa địa chỉ vật lý đầu tiên của đoạn trong bộ nhớ.
  - **limit** – xác định độ dài của đoạn.
- *Segment-table base register (STBR):* trỏ tới vị trí của bảng phân đoạn trong bộ nhớ.
- *Segment-table length register (STLR)*: xác định số đoạn mà một chương trình sử dụng;

  segment number *s* là hợp lệ nếu *s* < STLR.

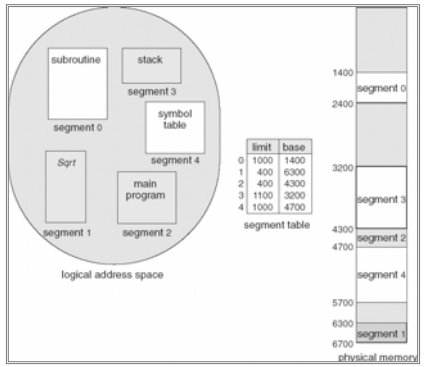
Kiến trúc phân đoạn (tiếp)

- Phân đoạn
  - các đoạn có kích thước khác nhau (khác với phân trang)
- Định vị
  - động
  - được thực hiện bởi bảng phân đoạn
- Phân phối bộ nhớ
  - giải quyết bài toán phân phối bộ nhớ động
  - first fit/best fit
  - có sự phân mảnh ngoài

Lược đồ phân đoạn



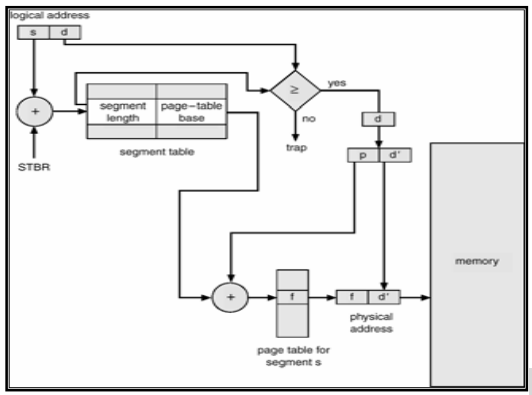
Ví dụ phân đoạn



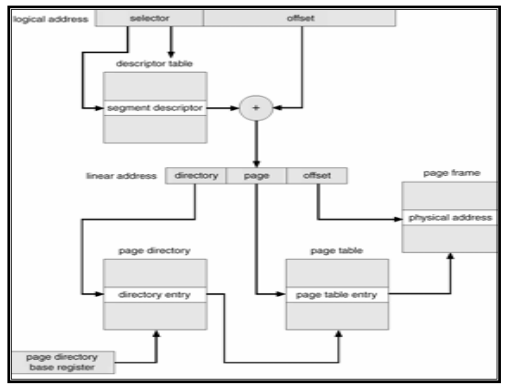
8.6. Kết hợp phân đoạn với phân trang MULTICS

- Bộ nhớ được phân thành các đoạn, sau đó mỗi đoạn lại được phân trang.
- Hệ thống MULTICS giải quyết được vấn đề phân mảnh ngoài và thời gian tìm kiếm dài.
- Giải pháp khác so với phân đoạn ở chỗ mỗi mục của bảng phân đoạn không chứa địa chỉ cơ sở của đoạn mà chứa địa chỉ cơ sở của bảng phân trang của đoạn đó.

Lược đồ MULTICS



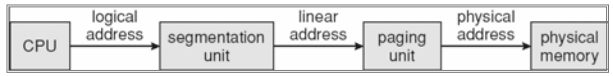
Lược đồ MULTICS của Intel 80386



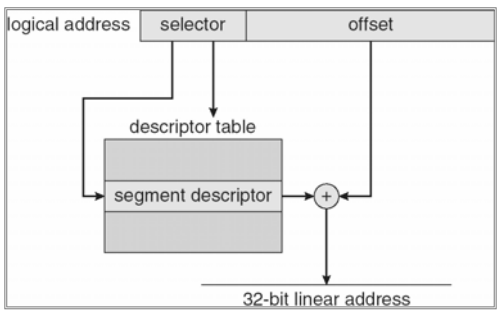
8.7. Example: The Intel Pentium

- Hỗ trợ cả 2 cách quản lý bộ nhớ: phân đoạn, phân đoạn kết hợp với phân trang
- CPU sinh ra các địa chỉ logic
  - địa chỉ này được đưa tới segmentation unit (đơn vị phân đoạn)
    - segmentation unit tạo ra 1 địa chỉ tuyến tính ứng với mỗi địa chỉ logic.
  - Địa chỉ tuyến tính được đưa tới paging unit (đơn vị phân trang)
    - paging unit sinh ra địa chỉ vật lý trong bộ nhớ chính
    - segmentation unit và paging unit tạo thành sự tương ứng của MMU.

Sự biên dịch địa chỉ logic thành địa chỉ vật lý trong Pentium



Intel Pentium Segmentation



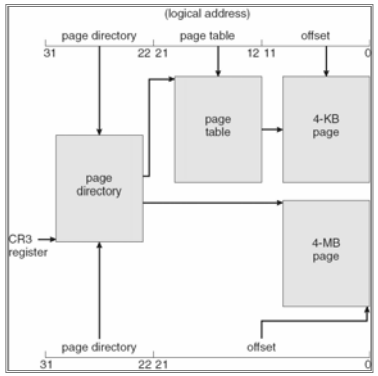
Pentium Paging

- Kích thước trang bằng 4 KB hoặc 4 MB.
- cờ Page Size (trong page directory):
  - = 1: kích thước page frame là 4 MB
    - phân chia địa chỉ tuyến tính 32 bit:

      <page number, page offset> = <10, 22>
  - = 0: kích thước page frame là chuẩn 4 KB
    - Sử dụng lược đồ phân trang 2 mức, phân chia địa chỉ tuyến tính 32-bit như sau:

page number		page offset
p_1	p_2	d
10	10	12

Pentium Paging Architecture

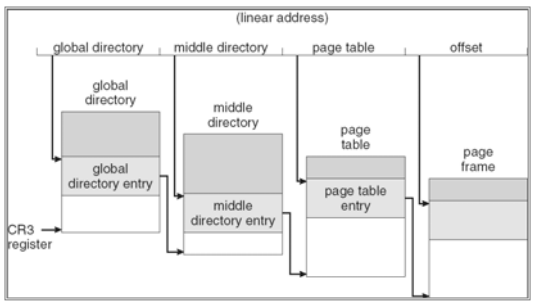


Linear Address in Linux

Được chia thành 4 phần:

global directory	middle directory	page table	offset

Three-level Paging in Linux



End of Chapter 8

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 9: Bộ nhớ ảo

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung



Nội dung chương 9

- Kiến thức cơ bản
- Phân trang theo yêu cầu - Demand Paging
- Thay trang - Page Replacement
- Phân phối các Frames - Allocation of Frames
- Thrashing
- Phân đoạn theo yêu cầu - Demand Segmentation





Mục tiêu

- Mô tả các lợi ích của bộ nhớ ảo.
- Giải thích các khái niệm phân trang theo yêu cầu, các giải thuật thay trang, phân phối các page frame



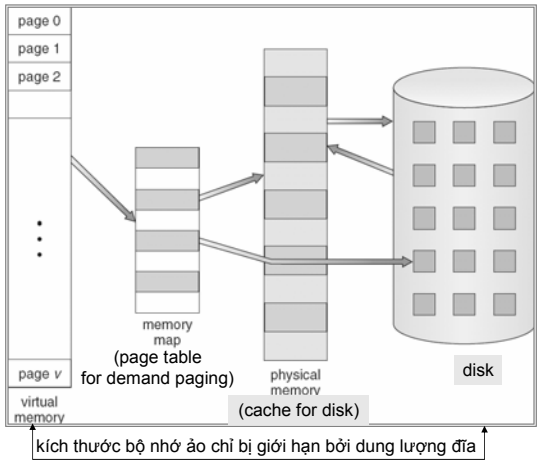


9.1. Kiến thức cơ bản

- **Virtual memory** – sự tách riêng của bộ nhớ logic người sử dụng khỏi bộ nhớ vật lý.
  - Kích thước bộ nhớ vật lý có hạn ⇒ giới hạn kích thước ch.trình
  - Thực tế, chỉ một phần của chương trình cần phải đưa vào bộ nhớ (vật lý) để thực hiện ⇒ có thể chứa chương trình ở đâu? – VIRTUAL MEMORY – phần đĩa cứng được quản lý như RAM
  - Do đó không gian địa chỉ logic có thể lớn hơn không gian địa chỉ vật lý rất nhiều ⇒ cung cấp bộ nhớ rất lớn cho người lập trình
  - Cho phép một số tiến trình chia sẻ không gian địa chỉ.
  - Cho phép tạo tiến trình hiệu quả hơn.
- Bộ nhớ ảo có thể được thực thi thông qua:
  - Demand paging (Windows, Linux…)
  - Demand segmentation (IBM OS/2)

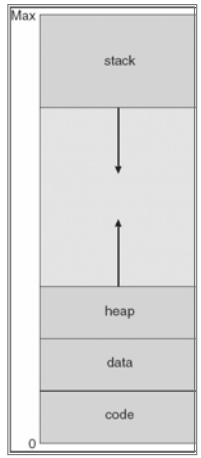


Bộ nhớ ảo lớn hơn bộ nhớ vật lý

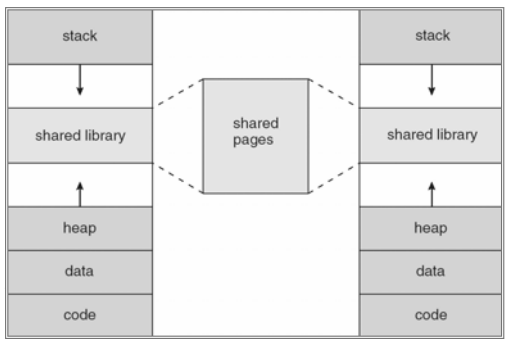


Không gian địa chỉ ảo của tiến trình

- Là cách nhìn logic (ảo) về cách mà tiến trình được lưu trong bộ nhớ.
- Tiến trình được lưu trong vùng nhớ liên tục, dù thực tế trong bộ nhớ vật lý nó có thể được lưu trong các page frame không liên tục.
- MMU đảm nhận việc ánh xạ các trang logic vào các page frame trong bộ nhớ vật lý.



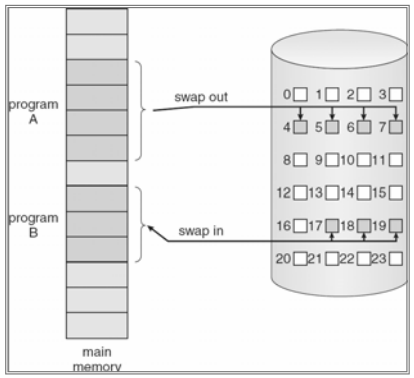
Shared Library Using Virtual Memory



9.2. Phân trang theo yêu cầu

- Đưa một trang vào bộ nhớ chỉ khi nó được cần đến.
  - Cần ít thao tác vào-ra hơn
  - Cần ít bộ nhớ hơn
  - Đáp ứng nhanh hơn: tiến trình bắt đầu ngay sau khi số trang tối thiểu được nạp vào bộ nhớ
  - Nhiều người sử dụng/tiến trình hơn: do mỗi tiến trình dùng ít bộ nhớ hơn
- Khi trang được cần đến (khi tiến trình tham chiếu đến nó)
  - tham chiếu không hợp lệ ⇒ hủy bỏ
  - không trong bộ nhớ ⇒ đưa vào bộ nhớ
  - có trong bộ nhớ ⇒ truy nhập

Chuyển một vùng nhớ phân trang tới không gian đĩa



Demand paging ⇔ Paging with swapping

However, Demand paging use a **lazy swapper**

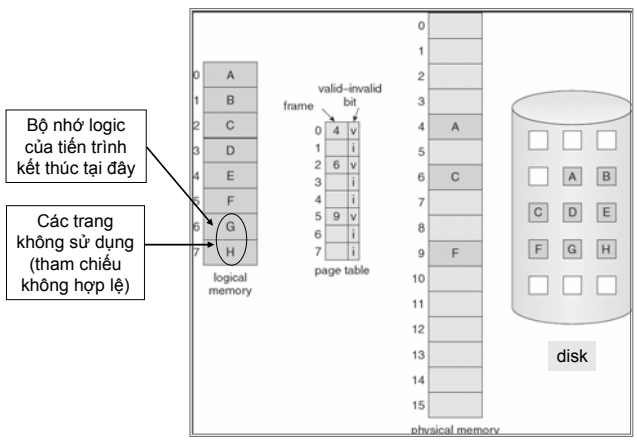
Valid-Invalid Bit

- Mỗi phần tử trong bảng phân trang được gắn thêm một valid–invalid bit (1 ⇒ có trong bộ nhớ, 0 ⇒ không trong bộ nhớ)
- Ban đầu khởi tạo tất cả các v–inv bit bằng 0
- Ví dụ bảng phân trang:

Frame #	valid-invalid bit
	1
	1
	1
	1
	0
⋮	
	0
	0

- Khi thông dịch địa chỉ, nếu v–inv bit bằng 0 ⇒ page fault.

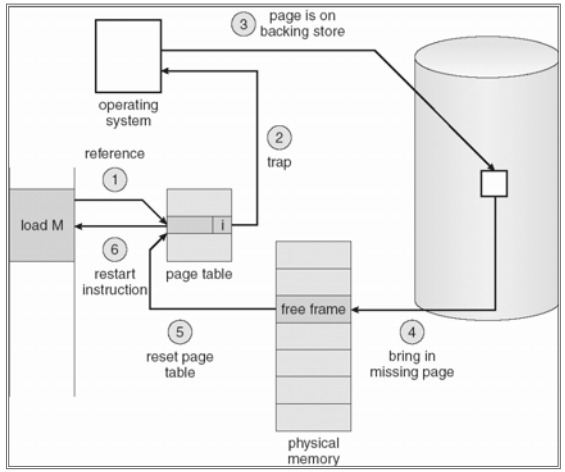
Bảng phân trang khi một số trang không ở trong bộ nhớ chính



Các bước xử lý page fault

1. Khi có tham chiếu tới trang, đầu tiên tham chiếu sẽ lập bẫy tới HĐH ⇒ phát hiện page fault
2. HĐH tìm trong bảng khác để quyết định:
   - tham chiếu không hợp lệ ⇒ hủy bỏ.
   - không có trong bộ nhớ ⇒ đưa vào bộ nhớ.
3. Nhận frame rỗi
4. Copy/Hoán đổi trang vào frame.
5. Cập nhật lại bảng phân trang (thiết lập v-inv bit = 1), cập nhật danh sách frame rỗi.
6. Khởi động lại lệnh gây page fault

Các bước xử lý page fault (tiếp)



Hiệu năng của phân trang theo yêu cầu

- Tỷ lệ Page Fault - *p* : $0 \le p \le 1$
  - *p*=0 : không có page faults;
  - *p*=1 : mọi tham chiếu đều là fault.
- Thời gian truy nhập hiệu quả-Effective Access Time (EAT)

  EAT = (1 – *p*) x *ma* + *p* x [*thời gian xử lý page-fault*]

  Trong đó:

  + *ma: memory access - thời gian truy nhập bộ nhớ (10-200 ns)*

  + thời gian xử lý page-fault: gồm 3 vấn đề chính:

  - phục vụ ngắt page-fault (1-100 μs, có thể giảm bằng coding)

  - đọc trang vào bộ nhớ (≈ 8 ms)

  - khởi động lại tiến trình (1-100 μs, có thể giảm bằng coding)

Ví dụ

- Thời gian xử lý page-fault: 8 ms
- Thời gian truy nhập bộ nhớ (ma): 200 ns
- EAT = (1-*p*) x 200 + *p* x 8,000,000 ns

  = 200 + 7,999,800 x *p* ns

  Nếu 1000 truy nhập có 1 page fault, EAT = 8.2 ms. Máy tính chậm hơn 40 lần vì phân trang có yêu cầu.
- EAT tỷ lệ trực tiếp với tỷ lệ page-fault, nếu p càng lớn thì EAT càng lớn → máy càng chậm.
- Muốn hiệu năng giảm dưới 10%, ta cần có:

  220 > 200 + 7,999,800 x p

  → p < 0.0000025

Điều gì xảy ra khi không có frame rỗi?

- Thay trang – tìm một số trang trong bộ nhớ nhưng đang không được sử dụng để đưa ra ngoài.
  - giải thuật?
  - hiệu năng? – muốn có một giải thuật tác động đến số lượng tối thiểu page faults.
- Một trang có thể được đưa vào bộ nhớ nhiều lần.

9.3. Thay trang

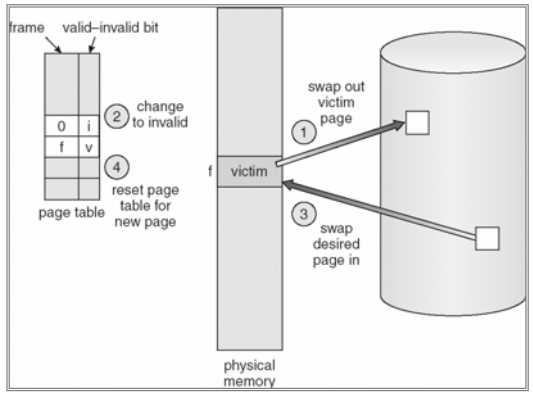
Các bước thay trang:

1. Tìm vị trí của trang được yêu cầu trên đĩa.
2. Tìm một frame rỗi:

   - Nếu có frame rỗi thì sử dụng nó.

   - Nếu không có, sử dụng một giải thuật thay trang để chọn một frame *nạn nhân*.
3. Đưa trang được yêu cầu vào frame rỗi. Cập nhật bảng phân trang và bảng quản lý frame rỗi.
4. Khởi động lại tiến trình.

Page Replacement

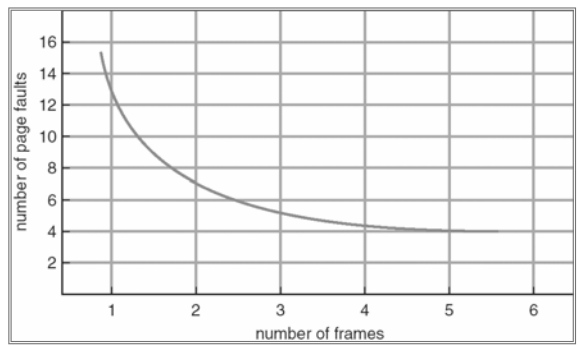


Các giải thuật thay trang

- Mục đích: tỷ lệ page-fault thấp nhất.
- Đánh giá giải thuật bằng cách chạy nó trên một chuỗi cụ thể các tham chiếu bộ nhớ và tính số page faults trên chuỗi đó.
- Trong tất cả các ví dụ, chuỗi tham chiếu là

  1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.

Đồ thị liên hệ giữa Page Faults và số Frame



a) First-In-First-Out (FIFO) Algorithm

- Chuỗi tham chiếu: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (với mỗi tiến trình: 3 trang có thể nạp vào bộ nhớ tại một thời điểm)

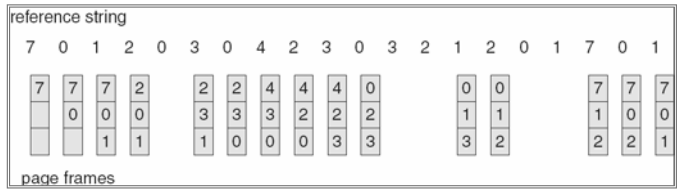
1	1	4	5	
2	2	1	3	9 page faults
3	3	2	4	

- 4 frames

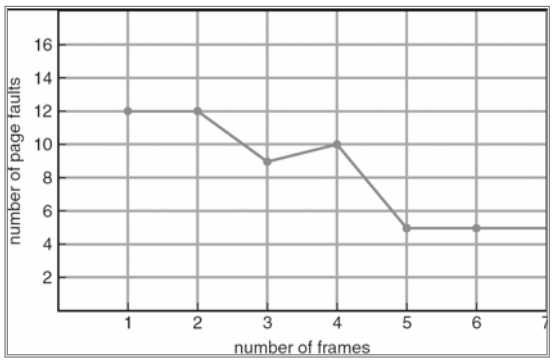
1	1	5	4	
2	2	1	5	10 page faults
3	3	2		
4	4	3		

- FIFO Replacement – Sự bất thường của Belady
  - nhiều frames ⇒ nhiều page faults

FIFO Page Replacement



Minh họa sự bất thường của Belady



b) Optimal Algorithm

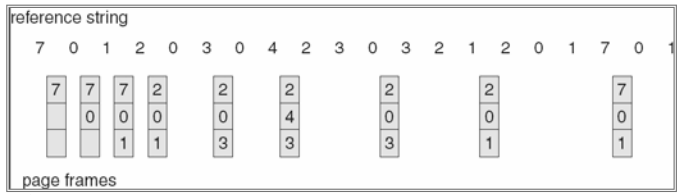
- Thay trang có thời gian sẽ không sử dụng đến lâu nhất.
- ví dụ 4 frames

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1	4		
2		6 page faults	
3			
4	5		

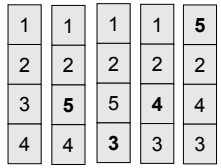
- Làm cách nào để biết trang nào sẽ không được sử dụng nữa? → Không thể biết được.
- Được sử dụng làm tiêu chuẩn đánh giá các giải thuật khác.

Optimal Page Replacement



c) Least Recently Used (LRU) Algorithm

- Thay trang có khoảng thời gian không dùng lâu nhất
- Chuỗi tham chiếu: 1, 2, 3, 4, 1, 2, **5**, 1, 2, **3**, **4**, **5**

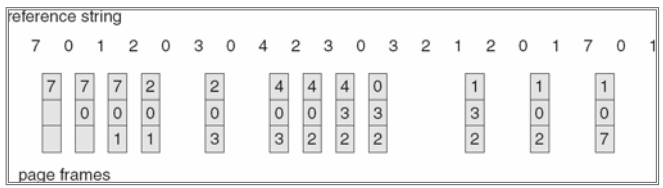


8 page faults

Sự thực hiện của Bộ đếm (Counter)

- Mọi phần tử bảng có một bộ đếm, mọi thời điểm trang được tham chiếu qua phần tử bảng này, copy clock vào trong bộ đếm.
- Khi trang cần được hoán đổi, tìm trong bộ đếm để xác định trang nào làm nạn nhân.

LRU Page Replacement

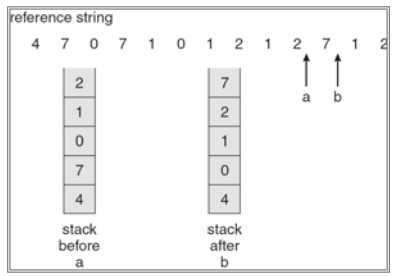


- Cách xác định trang không sử dụng lâu nhất?

1. Sử dụng Bộ đếm (Counter)
   - Mọi phần tử bảng có một bộ đếm, mọi thời điểm trang được tham chiếu qua phần tử bảng này, copy clock vào trong bộ đếm.
   - Khi trang cần được hoán đổi, tìm trong bộ đếm để xác định trang nào làm nạn nhân.

LRU Algorithm (tiếp)

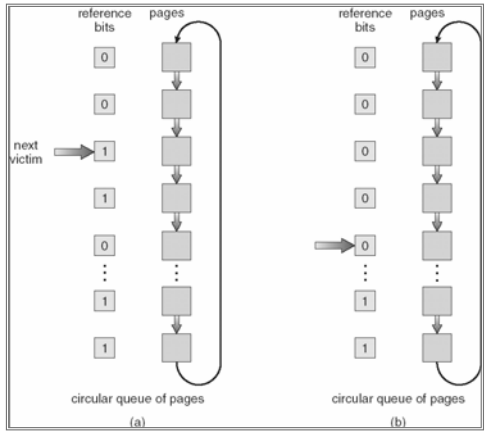
2. Sử dụng Stack
   - Dùng một stack của các số trang
   - Khi trang được tham chiếu:
     - chuyển nó lên đỉnh ⇒ trang không dùng lâu nhất sẽ luôn dưới đáy
     - dùng một danh sách liên kết kép, cần 6 con trỏ để đổi trang
   - Không cần tìm kiếm để thay thế



d) Các giải thuật tương tự LRU

- Giải thuật dùng thêm bit tham chiếu
  - Gắn một bit vào mỗi trang, khởi tạo = 0
  - Khi trang được tham chiếu, bit đó được thiết lập = 1.
  - Thay trang có bit tham chiếu = 0, nếu có tồn tại
- Giải thuật cơ hội thứ hai (Second chance)
  - Cần có bit tham chiếu. Nếu bit tham chiếu = 0 thì thay trang. Trái lại cho trang đó cơ hội thứ hai và chuyển đến trang tiếp sau
    - thiết lập bit tham chiếu = 0.
    - để trang lại trong bộ nhớ.
    - thay trang kế tiếp (theo FIFO) với luật tương tự.
  - Một cách thực hiện giải thuật là sử dụng queue vòng tròn.

Giải thuật thay trang "cơ hội thứ hai"



e) Các giải thuật dựa trên số liệu thống kê

- Dành ra một bộ đếm số tham chiếu đến mỗi trang.
- Least Frequently Used (LFU) Algorithm: thay trang đếm được ít nhất (có tần số truy nhập nhỏ nhất).
- Most Frequently Used (MFU) Algorithm: thay trang đếm được nhiều nhất (có tần số truy nhập cao nhất), dựa trên lý luận rằng trang đếm được ít nhất là có thể vừa được đưa vào bộ nhớ và chưa kịp được sử dụng.

9.4. Phân phối các Frame

- Mỗi tiến trình cần số lượng trang tối thiểu để thực hiện.
- Ví dụ: IBM 370 – cần 6 trang để thực hiện lệnh SS MOVE:
  - lệnh độ dài 6 bytes, có thể chứa trong 2 trang.
  - 2 trang để thực hiện **from**.
  - 2 trang để thực hiện **to**.
- Hai cách phân phối chính:
  - phân phối cố định (fixed allocation)
  - phân phối theo mức ưu tiên (priority allocation)

Phân phối cố định

1. Phân phối công bằng – vd nếu có 100 frames và 5 tiến trình, cho mỗi tiến trình 20 trang.
2. Phân phối theo kích thước của tiến trình

s_i = size of process p_i
$S = \sum s_i$
m = total number of frames
a_i = allocation for $p_i = \frac{s_i}{S} \times m$

$m = 64$
$s_i = 10$
$s_2 = 127$
$a_1 = \frac{10}{137} \times 64 \approx 5$
$a_2 = \frac{127}{137} \times 64 \approx 59$

Phân phối theo mức ưu tiên

- Nếu tiến trình P_i phát sinh một page fault,
  - chọn thay thế một trong số các frame của nó.
  - frame thay vào đó được chọn từ một tiến trình có mức ưu tiên thấp hơn.

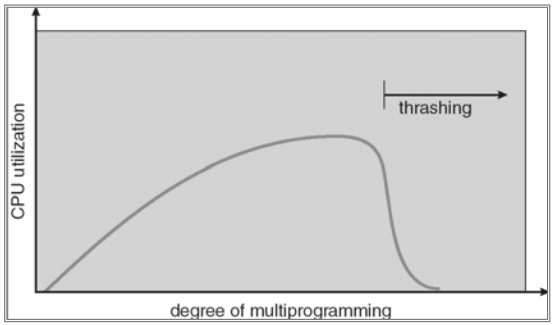
Global vs. Local Allocation

- **Global** replacement – tiến trình chọn một frame thay thế từ tập tất cả các frame; một tiến trình có thể lấy một frame từ một tiến trình khác.
- **Local** replacement – mỗi tiến trình chỉ chọn một frame thay thế từ chính tập các frame đã phân phối cho nó.

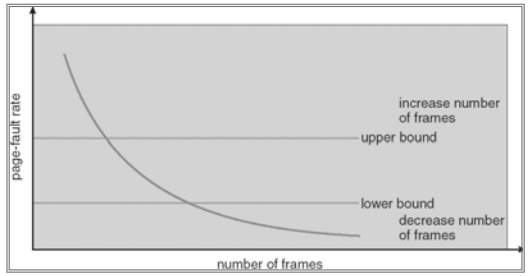
9.5. Thrashing

- Nếu một tiến trình không có "đủ" trang, tỷ lệ page fault là rất cao. Điều này dẫn đến:
  - sử dụng CPU thấp.
  - HĐH cho rằng nó cần phải tăng mức đa chương trình.
  - một tiến trình khác được thêm vào hệ thống, nó cố gắng giành frame từ các tiến trình đang thực hiện ⇒ tỷ lệ page fault càng tăng…
- **Thrashing** ≡ các tiến trình mất nhiều thời gian (bận rộn) với việc hoán đổi các trang vào-ra hơn là thời gian thực hiện.

Thrashing (tiếp)



Lược đồ tần số Page-Fault



- Thiết lập tỷ lệ page-fault "có thể chấp nhận được"
  - Nếu tỷ lệ thực tế quá thấp, tiến trình làm mất frame.
  - Nếu tỷ lệ thực tế quá cao, tiến trình tăng thêm frame.

Windows XP

- Sử dụng phân trang theo yêu cầu có **phân cụm** (**clustering**): đưa vào bộ nhớ fauling page và một số trang tiếp sau.
- Các tiến trình được ấn định **working set minimum** và **working set maximum**
- Working set minimum là số lượng trang nhỏ nhất mà tiến trình được đảm bảo có trong bộ nhớ.
- Một tiến trình có thể được gán cho càng nhiều trang càng tốt không vượt quá working set maximum của nó.
- Khi dung lượng bộ nhớ rỗi của hệ thống nhỏ hơn một ngưỡng, **automatic working set trimming** được thực hiện để khôi phục lại dung lượng bộ nhớ rỗi.
- Working set trimming loại bỏ số trang của các tiến trình vượt quá working set minimum của chúng.

Solaris

- Duy trì danh sách các trang rỗi để gán các faulting process.
- *Lotsfree* – tham số ngưỡng (dung lượng bộ nhớ rỗi) để bắt đầu phân trang.
- *Desfree* – tham số ngưỡng để tăng phân trang
- *Minfree* – tham số ngưỡng để thực hiện swapping
- Phân trang được thực hiện bởi tiến trình *pageout*
- Pageout quét các trang sử dụng giải thuật clock có sửa đổi
- *Scanrate* là tốc độ các trang được quét. Dải này từ *slowscan* đến *fastscan*
- Pageout được gọi càng thường xuyên phụ thuộc vào dung lượng bộ nhớ rỗi khả dụng.



Solaris 2 Page Scanner





End of Chapter 9

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 10: Giao diện Hệ thống File

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp HN
Website: fita.hua.edu.vn/pqdung



Nội dung chương 10

- Khái niệm File
- Các phương pháp truy nhập - Access Methods
- Cấu trúc thư mục - Directory Structure
- Chia sẻ file - File Sharing
- Protection





Mục tiêu

- Giải thích chức năng của hệ thống file
- Miêu tả giao diện của hệ thống file
- Thảo luận việc cân bằng các yếu tố thiết kế hệ thống file, bao gồm các phương pháp truy nhập, chia sẻ file, khóa file, các cấu trúc thư mục
- Giải thích sự bảo vệ hệ thống file





10.1. Khái niệm File

- File là một tập hợp của các thông tin liên quan, được ghi trên bộ nhớ thứ cấp (là bộ nhớ ổn định) và được đặt tên.
- Từ góc nhìn của người sử dụng, file là đơn vị bộ nhớ logic nhỏ nhất. Các file được ánh xạ bởi HĐH vào các thiết bị nhớ vật lý.
- Kiểu tệp:
  - Data
    - số - numeric
    - ký tự - character
    - nhị phân - binary
  - Program
- Nói chung, file là một chuỗi các bit, byte, dòng hoặc bản ghi



Cấu trúc File

- Không cấu trúc - chuỗi các words, bytes
- Cấu trúc bản ghi đơn giản
  - các dòng (lines)
  - độ dài cố định
  - độ dài thay đổi
- Các cấu trúc phức tạp
  - văn bản có định dạng - Formatted document
  - file nạp có thể tái định vị - Relocatable load file
- Ai quyết định cấu trúc file?
  - HĐH
  - Chương trình

Thuộc tính File

- **Name** – chỉ là thông tin ở dạng người đọc được.
- **Type** – cần thiết cho các HĐH hỗ trợ nhiều kiểu file.
- **Location** – con trỏ tới vị trí file trên thiết bị.
- **Size** – kích thước hiện tại của file.
- **Protection** – kiểm soát ai có thể đọc, ghi, thực hiện file.
- **Time**, **date**, **user identification** – dữ liệu dùng cho protection, security, và theo dõi sử dụng.
- Thông tin về file được lưu trong cấu trúc thư mục, cũng được lưu trên đĩa.

Thao tác với File

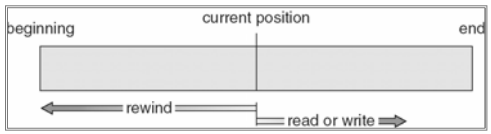
- Tạo file
- Ghi file
- Đọc file
- Định vị trong file – file seek
- Xóa file
- Cắt bớt file (truncate)
- Open(F_i) – tìm chỉ mục F_i trong cấu trúc thư mục trên đĩa rồi chuyển nội dung của chỉ mục vào bộ nhớ.
- Close (F_i) – chuyển nội dung của chỉ mục F_i trong bộ nhớ ra cấu trúc thư mục trên đĩa.

File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

10.2. Các phương pháp truy nhập

- **Truy nhập tuần tự - Sequential Access**
  - Truy nhập tuần tự qua các bản ghi từ đầu tệp đến cuối tệp
    read next
    write next
    reset
    no *read after last write*
  - Một số HĐH cho phép nhảy tới hoặc lui ***n*** bản ghi.
  - Các trình soạn thảo và trình biên dịch thường truy nhập tệp theo phương pháp này.



Các phương pháp truy nhập (tiếp)

- **Truy nhập trực tiếp - Direct Access**
  - Tệp được tạo bởi các bản ghi có kích thước cố định
  - Có thể truy nhập các bản ghi tại vị trí bất kỳ trong tệp mà không cần theo thứ tự.
  - Các CSDL thường được tổ chức theo phương pháp này
  - Sử dụng các phương thức:
    read n
    write n
    position to n
      read next
      write next
    rewrite n

    n = số hiệu bản ghi cần truy nhập, có thể bắt đầu từ 0 hoặc 1 tùy thuộc HĐH

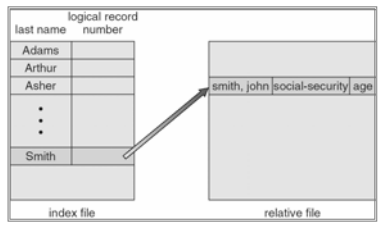
Minh họa cách truy nhập tuần tự trong một file truy nhập trực tiếp

cp - current position: biến xác định vị trí hiện tại

sequential access	implementation for direct access
reset	*cp* = 0;
read next	*read cp*; *cp* = *cp* + 1;
write next	*write cp*; *cp* = *cp* + 1;

Các phương pháp truy nhập (tiếp)

- **Truy nhập index-relative**
  - Tệp index chứa các con trỏ tới các bản ghi trong tệp relative
  - Để truy nhập các bản ghi trong tệp relative, trước tiên tìm index, tiếp theo dùng con trỏ để truy nhập trực tiếp tệp relative để tìm bản ghi.
  - Hữu dụng khi tìm kiếm trong các tệp lớn vì số lần thực hiện vào-ra ít
  - Có thể có nhiều hơn một mức index: index-index-relative

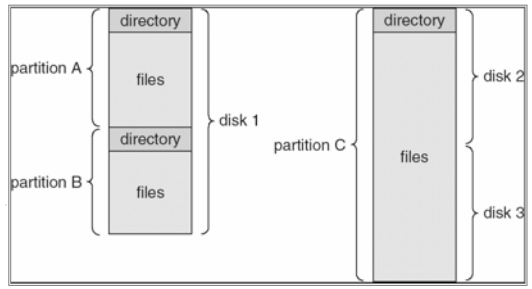


10.3. Cấu trúc thư mục

Để quản lý số lượng lớn các tệp trên đĩa, tổ chức chúng trong 2 bước:

- Đầu tiên, chia đĩa thành một hay nhiều *partition* (minidisk-IBM, volume-PC & Macintosh)
  - partition - cấu trúc mức thấp, để chứa các tệp và thư mục
  - một số HĐH cho phép patition lớn hơn đĩa
- Tiếp theo, mỗi partition có một **device directory** (directory) ghi thông tin về tất cả các tệp trên partition đó: tên tệp, vị trí, kích thước, kiểu tệp…

Một tổ chức hệ thống file cơ bản



Các thao tác trên một thư mục

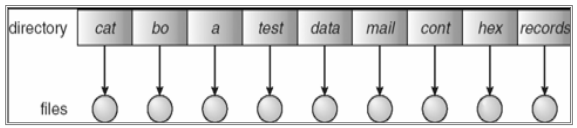
- Tìm kiếm 1 tệp
- Tạo 1 tệp
- Xóa 1 tệp
- Liệt kê danh sách tệp trong thư mục
- Đổi tên 1 tệp
- Truy nhập toàn bộ hệ thống file

Tổ chức logic thư mục để sử dụng

- **Hiệu quả (Efficiency)** – định vị file nhanh chóng.
- **Đặt tên (Naming)** – thuận tiện cho người sử dụng.
  - nhiều tệp có thể có cùng tên.
  - 1 tệp có thể có nhiều tên.
- **Gom nhóm (Grouping)** – nhóm logic các tệp theo thuộc tính, (vd: all Java programs, all games, …)

10.3.1. Single-Level Directory

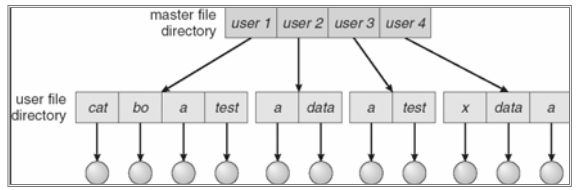
- Một thư mục chứa tất cả các tệp.



- Ưu:
  - dễ hiểu, dễ quản lý
  - kích thước nhỏ
- Nhược:
  - vấn đề đặt tên: mỗi tệp phải có tên duy nhất
  - vấn đề gom nhóm: không thể

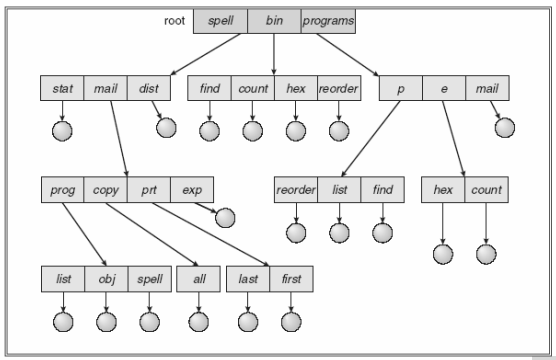
10.3.2. Two-Level Directory

- Mỗi user có một thư mục riêng



- Để truy nhập 1 tệp ở thư mục khác, cần có đường dẫn đầy đủ
- Có thể có các tệp trùng tên cho các user khác nhau
- Tìm kiếm hiệu quả hơn
- Không có khả năng gom nhóm

10.3.3. Tree-Structured Directories

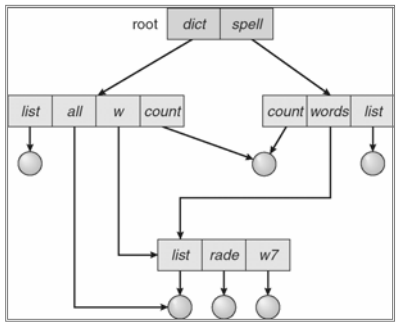


Tree-Structured Directories (tiếp)

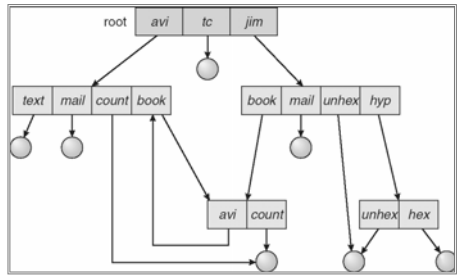
- Có 1 thư mục gốc (root)
- Mỗi tệp có 1 đường dẫn duy nhất:
  - tuyệt đối, vd: **C:\Windows\php.ini**
  - quan hệ (với thư mục hiện tại), vd: **.\System32\test.dll**
- Mỗi thư mục chứa các tệp và/hoặc các thư mục con
- Tìm kiếm hiệu quả
- Thuận tiện trong đặt tên
- Có khả năng gom nhóm

10.3.4. Acyclic-Graph Directories

- Có sự chia sẻ (not copy!) các thư mục con và các tệp, thuận tiện khi nhiều user làm việc trong 1 dự án.



10.3.5. General Graph Directory



- Khi một liên kết được thêm vào cấu trúc, cần đảm bảo không tạo thành chu trình → sử dụng giải thuật tìm kiếm chu trình trong đồ thị, nhưng là việc "nặng nhọc" vì đồ thị trên đĩa, không phải trong bộ nhớ trong.

10.4. File Sharing

- Yêu cầu: phải chia sẻ các file trên các hệ thống đa người dùng (multi-user systems).
- Chia sẻ file có thể được thực hiện thông qua một lược đồ *protection*.
- Trên các hệ thống phân tán (distributed systems), các file có thể được chia sẻ qua mạng (network).
- Network File System (NFS) là một phương thức chia sẻ file phân tán (distributed file-sharing method) phổ biến.
  - sử dụng trong mô hình client-server
  - các user ID phải phù hợp cả với client và server để xác nhận quyền truy nhập file trên server.

10.5. Protection

- Người tạo/sở hữu file cần có khả năng giám sát:
  - thao tác nào đã được thực hiện
  - bởi user nào?
- Các loại truy nhập
  - Read
  - Write
  - Execute
  - Append
  - Delete
  - List

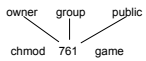


Access Lists and Groups

- Chế độ truy nhập: read, write, execute
- Ba lớp người sử dụng:

			RWX
a) **owner access**	7	⇒	1 1 1
			RWX
b) **group access**	6	⇒	1 1 0
			RWX
c) **public access**	1	⇒	0 0 1

- Yêu cầu người quản lý tạo một group G (có tên duy nhất), rồi thêm các user vào group.
- Đối với các file (vd *game*) hoặc subdirectory, xác định sự truy nhập tương tự như trên.



Gắn group cho file:

chgrp G game



End of Chapter 10

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 11: Các hệ thống lưu trữ lớn

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung



Nội dung chương 11

- Tổng quan về hệ thống lưu trữ lớn
- Cấu trúc đĩa — Disk Structure
- Gắn kết đĩa — Disk Attachment
- Lập lịch đĩa — Disk Scheduling
- Quản lý đĩa — Disk Management
- Quản lý không gian hoán đổi — Swap-Space Management
- Cấu trúc RAID — RAID Structure
- Các thiết bị lưu trữ cấp ba — Tertiary Storage Devices





Mục tiêu

- Mô tả cấu trúc vật lý của các thiết bị lưu trữ cấp hai và cấp ba, các hiệu ứng kết quả khi sử dụng thiết bị.
- Giải thích các đặc điểm hiệu năng của các thiết bị lưu trữ lớn.



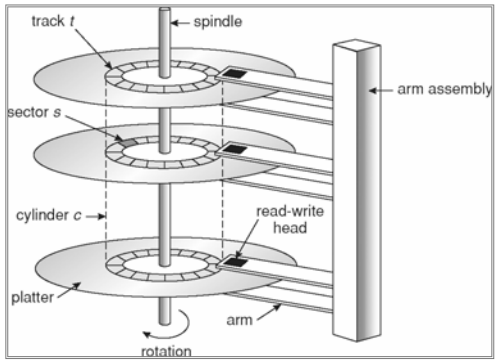


11.1. Tổng quan về Cấu trúc lưu trữ lớn

- Các đĩa từ (magnetic disk) chiếm phần lớn của phương tiện lưu trữ cấp hai trong các máy tính hiện đại
  - Tốc độ quay của ổ đĩa đạt khoảng 60-200 vòng/giây.
  - **Transfer rate** là tốc độ dòng dữ liệu truyền giữa đĩa và máy tính
  - **Positioning time** (**random-access time**) là thời gian chuyển disk arm tới cylinder mong muốn (**seek time**) và thời gian để sector cần thiết quay tới dưới disk head (**rotational latency**)
  - **Head crash** do disk head tiếp xúc với bề mặt đĩa
    - Điều này rất tồi tệ
- Các đĩa có thể là khả chuyển (removable)
- Ổ đĩa được gắn (attached) với máy tính thông qua **I/O bus**
  - Nhiều loại bus **EIDE, ATA, SATA, USB, Fibre Channel, SCSI**
  - **Host controller** trong máy tính sử dụng bus để "nói chuyện" với **disk controller** được tích hợp trong ổ đĩa hoặc chuỗi lưu trữ (storage array)



Cơ chế đĩa có đầu từ chuyển động



Tổng quan (tiếp)

- Băng từ (Magnetic tape)
  - Là phương tiện lưu trữ cấp hai trước đây
  - khá ổn định và lưu giữ được lượng lớn dữ liệu
  - Thời gian truy nhập chậm
  - Truy nhập ngẫu nhiên chậm hơn khoảng 1000 lần so với đĩa
  - Thường được dùng để sao lưu dự phòng, lưu trữ dữ liệu ít sử dụng, làm phương tiện trung chuyển giữa các hệ thống
  - Băng được giữ trong một ống cuộn và được cuốn xuôi hoặc ngược qua một read-write head
  - Khi dữ liệu ở dưới đầu từ, tốc độ truyền tương đương với đĩa
  - Dung lượng 20-200GB
  - Các công nghệ phổ biến: 4mm, 8mm, 19mm, LTO-2 và SDLT

11.2. Cấu trúc đĩa

- Ổ đĩa được đánh địa chỉ là mảng 1 chiều lớn của các khối logic, khối logic là đơn vị nhỏ nhất trong chuyển dữ liệu.
- Mảng trên được ánh xạ tuần tự vào các sector của đĩa.
  - Sector 0 là sector đầu tiên của track đầu tiên trên cylinder ngoài cùng.
  - Việc ánh xạ tiếp tục theo thứ tự qua track đó, rồi đến các track còn lại trong cylinder đó, rồi đến các cylinder còn lại từ ngoài vào trong.

11.3. Gắn kết đĩa (Disk Attachment)

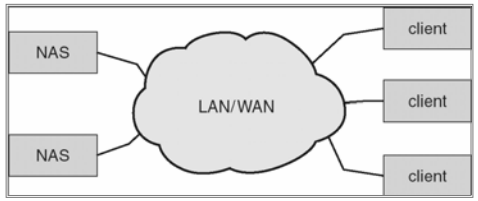
- Các đĩa có thể được nối kết theo 1 trong 2 cách:

1. **Host-attached storage:** nối kết thông qua một cổng vào-ra, sử dụng một số kỹ thuật:
   - Kiến trúc I/O bus (IDE-Integrated Drive Electronics, ATA-Advanced Technology Attachment): hỗ trợ tối đa 2 ổ đĩa trên mỗi I/O bus, sử dụng trong các máy PC.
   - Kiến trúc SCSI (Small Computer System Interface): hỗ trợ tối đa 16 thiết bị/1 bus (1 card điều khiển, 15 thiết bị lưu trữ)
   - Kiến trúc Fiber Channel (FC): sử dụng cáp quang hoặc cáp đồng, có thể kết nối hàng triệu thiết bị (2^{24}) trên mạng. Cũng có thể là dạng **arbitrated loop** (**FC-AL**) hỗ trợ nối 126 thiết bị.

Nối kết đĩa (tiếp)

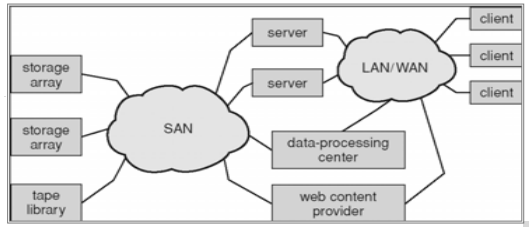
2. **Network-attached storage (NAS):**

- nối kết các thiết bị nhớ thông qua một kết nối mạng sử dụng các giao thức NFS (UNIX), CIFS (Windows), iSCSI.
- Được thực thi thông qua các remote procedure call (RPCs) giữa host và storage



Storage Area Network

- Phổ biến trong các môi trường lưu trữ lớn (và đang trở nên phổ biến hơn)
- Nhiều host được gắn kết vào các chuỗi nhiều phương tiện lưu trữ - phức tạp



11.4. Lập lịch đĩa (disk scheduling)

- HĐH chịu trách nhiệm sử dụng các ổ đĩa một cách hiệu quả, có nghĩa đĩa phải có thời gian truy nhập nhanh và dải thông rộng.
- Thời gian truy nhập có 2 thành phần chính
  - *Thời gian định vị (Seek time):* là thời gian chuyển đầu từ tới cylinder chứa sector được yêu cầu.
  - *Trễ quay (Rotational latency):* là thời gian cộng thêm chờ đĩa quay sector được yêu cầu tới đầu từ.

  $\Rightarrow$ Tối thiểu hóa seek time bằng cách lập lịch đĩa

  Seek time $\approx$ seek distance
- Dải thông đĩa (Disk bandwidth) tính bằng *tổng số byte được chuyển* chia cho *tổng thời gian giữa yêu cầu dịch vụ đầu tiên và lần chuyển cuối cùng*.
  - Seek time tốt hơn với mỗi yêu cầu sẽ cải thiện bandwidth.

Lập lịch đĩa (tiếp)

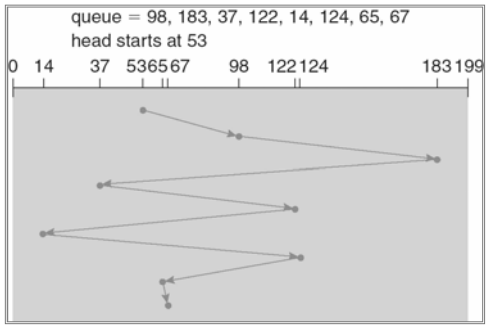
- Khi tiến trình cần thực hiện vào-ra với đĩa, nó phát 1 system call tới HĐH, HĐH cần xác định:
  - thao tác là input hay output
  - địa chỉ đĩa và địa chỉ bộ nhớ (nguồn và đích)
  - số byte cần chuyển
- Nếu ổ đĩa và controller (bộ điều khiển) sẵn sàng, yêu cầu có thể được thực hiện ngay. Trái lại, nó được đưa vào queue của đĩa để chờ được phục vụ.
- Có một số giải thuật lập lịch sự phục vụ các yêu cầu vào-ra đĩa cho một thứ tự tốt.
- Chúng ta minh họa chúng với một request queue (0-199).

  98, 183, 37, 122, 14, 124, 65, 67

  Đầu từ đĩa đang ở cylinder 53

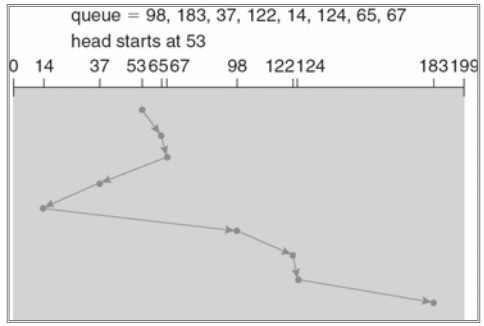
a) FCFS – Fist Come, First Served

Tổng quãng đường di chuyển của đầu từ là 640 cylinder.



b) SSTF – Shortest Seek Time First

- Chọn yêu cầu với seek time nhỏ nhất từ vị trí đầu từ hiện thời.
- Tổng quãng đường di chuyển của đầu từ là 236 cylinder.

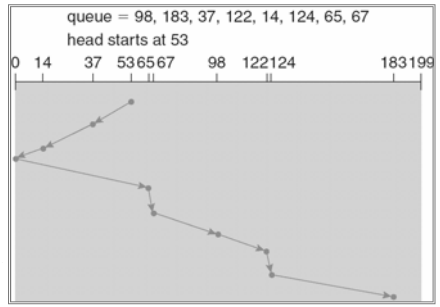


c) SCAN

- Disk arm bắt đầu tại một đầu của đĩa, tiến dần tới đầu còn lại, phục vụ yêu cầu khi nó đến mỗi cylinder, tại đầu còn lại hướng di chuyển của đầu từ sẽ đảo ngược và việc phục vụ tiếp tục.
  - Cần biết thêm hướng di chuyển của đầu từ
- Còn gọi là giải thuật thang máy - *elevator algorithm*.
- Tổng quãng đường di chuyển của đầu từ là 236 cylinder.

SCAN (tiếp)

- Tổng quãng đường di chuyển của đầu từ là 236 cylinder.



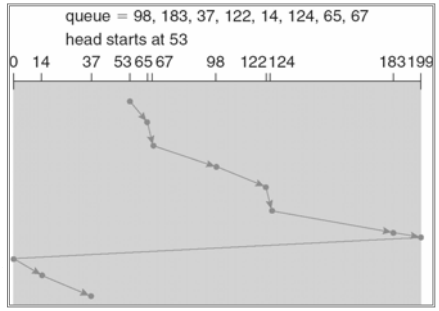
- Tổng quãng đường di chuyển của đầu từ là bao nhiêu nếu nó di chuyển theo hướng ngược lại?

d) C-SCAN (Circular SCAN)

- Tương tự như SCAN, nhưng có thời gian chờ đồng đều hơn so với SCAN.
- Đầu từ di chuyển từ một đầu đĩa tới đầu còn lại, phục vụ yêu cầu khi nó đến. Tuy nhiên, khi nó đến đầu kia thì lập tức quay về điểm đầu đĩa mà không phục vụ yêu cầu nào trên hành trình quay về đó.

C-SCAN (tiếp)

- Tổng quãng đường di chuyển của đầu từ là 382 cylinder.



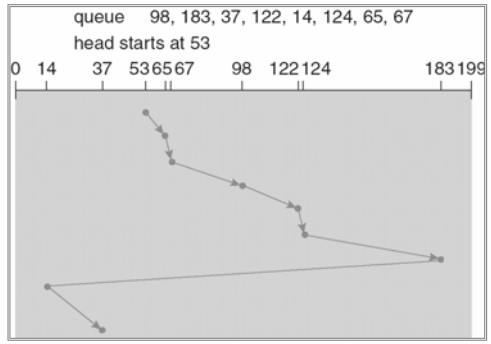
- Tổng quãng đường di chuyển của đầu từ là bao nhiêu nếu nó di chuyển theo hướng ngược lại?

e) LOOK và C-LOOK

- Là phiên bản tương ứng của SCAN và C-SCAN
- Arm chỉ đi đến yêu cầu cuối cùng trên mỗi hướng rồi lập tức đảo hướng mà không đi tất cả quãng đường lãng phí đến tận cùng đĩa.
- Gọi là LOOK vì nó tìm kiếm một yêu cầu trước khi tiếp tục di chuyển trên hướng đi.

C-LOOK (tiếp)

- Tổng quãng đường di chuyển của đầu từ là 322 cylinder.



Lựa chọn một giải thuật lập lịch đĩa

- SSTF phổ biến và có hiệu quả tốt.
- SCAN và C-SCAN thực hiện tốt hơn đối với các hệ thống đặt tải lớn lên đĩa.
- Hiệu năng phụ thuộc vào số lượng và loại yêu cầu.
- Các yêu cầu phục vụ đĩa có thể bị ảnh hưởng bởi phương thức định vị file.
- Giải thuật lập lịch đĩa có thể được viết như một module riêng của HĐH, cho phép thay thế khi cần thiết.
- Giải thuật SSTF hoặc LOOK là một lựa chọn hợp lý làm mặc định.

11.5. Quản lý đĩa
a) Disk Formatting

- *Low-level formatting*, hay *physical formatting* – chia một đĩa thành các sector (cung) để controller (bộ điều khiển đĩa) có thể đọc và ghi.
- Cấu trúc dữ liệu của mỗi sector:
  - 1 header và 1 trailer: chứa thông tin được sử dụng bởi disk controller, ví dụ số hiệu sector, mã sửa lỗi (ECC: error-correcting code)
  - Vùng chứa dữ liệu: thường 512 byte; hoặc 256 hay 1024 byte.

Disk Formatting (tiếp)

- Để sử dụng đĩa lưu giữ file, HĐH vẫn cần phải ghi cấu trúc dữ liệu của chính nó trên đĩa.
  1. Phân chia đĩa thành một hay nhiều nhóm các cylinder, được gọi là Partition. HĐH có thể xử lý mỗi Partition như một đĩa độc lập.
  2. *Logical formatting* hay "making a file system": HĐH ghi lên đĩa cấu trúc dữ liệu hệ thống file ban đầu, có thể gồm các thư mục rỗng ban đầu, bản đồ không gian bộ nhớ tự do và đã sử dụng (bảng FAT – File Allocation Table).

b) Boot Block

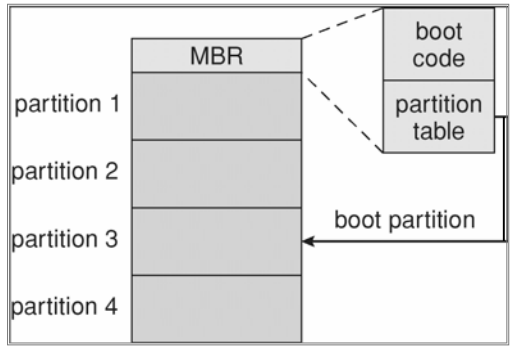
- Để MT bắt đầu chạy (khi bật máy, khi khởi động lại), cần có một chương trình khởi tạo để chạy: chương trình mồi – bootstrap.
- Bootstrap khởi động tất cả các bộ phận của máy tính, từ các thanh ghi trong CPU đến các mạch điều khiển thiết bị và nội dung của bộ nhớ chính, sau đó bắt đầu chạy HĐH.
- Để thực hiện công việc của mình, chương trình bootstrap tìm nhân (kernel) của HĐH trên đĩa để nạp vào bộ nhớ, rồi nhảy đến một địa chỉ bắt đầu sự thực hiện của HĐH.

Boot Block (tiếp)

- Bootstrap được lưu trong ROM:
  - ROM không cần khởi tạo,
  - Ở tại vị trí cố định processor có thể bắt đầu thực hiện khi được khởi động
  - ROM không bị ảnh hưởng bởi virus máy tính
- Hầu hết HĐH chỉ chứa chương trình mồi rất nhỏ trong boot ROM, giúp cho việc nạp chương trình mồi đầy đủ từ đĩa.

  ⇒ Chương trình mồi đầy đủ có thể được thay đổi dễ dàng
- Chương trình mồi đầy đủ được chứa trong một partition gọi là boot blocks, tại một vị trí cố định trên đĩa.
- Đĩa có một boot partition được gọi là boot disk hay system disk

Khởi động từ đĩa trong Windows 2000



c) Bad Blocks

- Trên các đĩa đơn giản, vd đĩa IDE, các bad block được xử lý thủ công bằng lệnh **format** của MS-DOS:
  - Thực hiện format logic, quét đĩa để tìm các bad block.
  - Nếu tìm thấy bad block, một giá trị đặc biệt được ghi vào phần tử tương ứng trong bảng FAT để báo cho các thường trình phân phối (allocation routine) không sử dụng block đó nữa.
  - Nếu các block trở thành bad trong khi hoạt động bình thường, có thể chạy một chương trình đặc biệt như **chkdsk** để tìm các bad block và xử lý chúng như ở trên.
  - Dữ liệu trên các bad block thường bị mất

Bad Blocks (tiếp)

- Trên các đĩa phức tạp, vd đĩa SCSI, việc phục hồi bad block thông minh hơn:
  - Controller duy trì một danh sách các bad block trên đĩa. DS này được khởi tạo khi format cấp thấp tại nhà máy và được cập nhật trong suốt sự tồn tại của đĩa.
  - Format cấp thấp cũng thiết lập các sector dự phòng (spare sector) vô hình đối với HĐH. Controller có thể thay thế mỗi bad sector một cách logic bởi một trong số các sector dự phòng.

    ⇒ **Sector sparing** (kỹ thuật dự phòng sector) hay **forwarding**
    - HĐH cố gắng đọc block 87. Mạch điều khiển (controller) tính toán ECC và thấy sector đó là bad. Nó thông báo cho HĐH biết.
    - Ở lần khởi động tiếp theo, một lệnh đặc biệt được chạy để ra lệnh cho Mạch điều khiển SCSI thay thế bad sector, ví dụ bởi sector 202.
    - Sau đó, mỗi khi hệ thống yêu cầu block 87, mạch điều khiển sẽ thông dịch yêu cầu sang địa chỉ của sector 202.

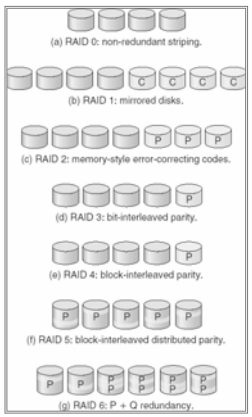
11.6. Quản lý không gian hoán đổi (Swap-Space)

- Swap-space: Bộ nhớ ảo sử dụng không gian đĩa như là sự mở rộng của bộ nhớ chính ⇒ tăng dung lượng, tăng tốc độ.
- Swap-space có thể được tạo ra:
  - Từ hệ thống file bình thường: swap-space là một file lớn do thường trình hệ thống file tạo, đặt tên và phân phối bộ nhớ.
    - Phương pháp này dễ thực hiện nhưng không hiệu quả: định vị cấu trúc thư mục và cấu trúc dữ liệu trên đĩa tốn nhiều thời gian và truy nhập đĩa nhiều lần hơn.
  - Trong 1 partition riêng (phổ biến hơn): không có cấu trúc thư mục và file trên đó,
    - 1 trình quản lý bộ nhớ hoán đổi riêng điều khiển việc phân phối và thu hồi các block. Nó sử dụng các giải thuật để tối ưu tốc độ hơn là để lưu trữ hiệu quả.

11.7. Cấu trúc RAID

- **RAID** – Redundant Arrays of Inexpensive/Independent Disk
- Sử dụng nhiều đĩa như một đơn vị lưu trữ
- Cải thiện hiệu năng và độ tin cậy của hệ thống lưu trữ bằng cách lưu trữ các dữ liệu dư thừa.
- RAID được phân cấp thành 7 mức:
  - RAID mức 0: dữ liệu được phân ra nhiều ổ đĩa nhưng không có ổ dự phòng.
  - RAID mức 1: dữ liệu được phân vào 1 dãy những ổ đĩa và dữ liệu trong mỗi ổ lại được chuyển vào một ổ đĩa lưu trữ để dự phòng.
  - RAID mức 2, 3, 4: dữ liệu được chia cho nhiều đĩa và thông tin parity được phát sinh và ghi vào 1 đĩa riêng biệt. Mỗi cấp độ có những phương pháp khác nhau khi ghi dữ liệu lên đĩa.
  - RAID mức 5: dữ liệu và các đoạn mã parity được ghi lên tất cả các ổ đĩa trong dãy các ổ đĩa. Mức này cho phép ghi nhanh vì thông tin parity được lan ra tất cả các ổ đĩa mà không phải là ghi lên từng ổ riêng biệt.

Các mức RAID



11.8. Cấu trúc lưu trữ cấp ba
(Tertiary storage structure)
11.8.1. Các thiết bị nhớ cấp ba

- Giá thành rẻ là đặc điểm nổi bật của bộ nhớ cấp ba.
- Nói chung, bộ nhớ cấp ba gồm các thiết bị khả chuyển (*removable media*)
- Ví dụ phổ biến: đĩa mềm, đĩa CD, flash memory.

a) Removable Disks

- Floppy disk – đĩa mềm dẻo mỏng được phủ lớp từ và được bảo vệ bên ngoài bởi một vỏ bằng chất dẻo.
  - Có dung lượng khoảng 1.4 MB;
- Removable magnetic disk được tạo bằng kỹ thuật tương tự
  - Có dung lượng hơn 1 GB.
  - Có tốc độ nhanh gần như hard disks, nhưng rất dễ bị hỏng.

Removable Disks (tiếp)

- Đĩa từ-quang (magneto-optic disk) ghi dữ liệu trên một mặt đĩa cứng được phủ lớp từ.
  - Ổ đĩa có 1 cuộn dây sinh từ trường. Tại nhiệt độ thường, từ trường quá rộng và yếu nên không thể từ hóa các bit trên đĩa → Sử dụng phương pháp đốt Laser: đầu đĩa chiếu 1 tia laser lên mặt đĩa để ghi một bit.
  - Ánh sáng Laser cũng được sử dụng để đọc dữ liệu.
  - Đầu đọc đĩa từ-quang bay xa mặt đĩa hơn nhiều so với đầu đọc đĩa từ, và lớp từ được phủ một lớp bảo vệ dầy bằng kính hoặc chất dẻo để chống sự phá hủy của đầu đọc.

Removable Disks (tiếp)

- Các đĩa quang (optical disk) không sử dụng hiện tượng từ tính; chúng sử dụng các chất liệu đặc biệt có thể bị biến đổi bằng ánh sáng laser thành các điểm sáng và tối.

  Vd: đĩa đổi pha:
  - Đĩa được phủ chất liệu có thể đông cứng thành trạng thái kết tinh hoặc vô định hình.
  - Trạng thái kết tinh trong suốt hơn nên tia laser sáng hơn khi đi qua chất liệu đổi pha và bật ra khỏi lớp phản chiếu.
  - Ổ đĩa đổi pha sử dụng ánh sáng laser ở 3 cường độ: cường độ yếu để đọc dữ liệu, cường độ trung bình để xóa đĩa (làm tan rồi đông cứng lại), cường độ mạnh làm tan chất liệu thành trạng thái vô định hình để ghi dữ liệu.
  - VD: re-recordable CD-RW và DVD-RW

Removable Disks (tiếp)

- Các loại trên là read-write disk: dữ liệu có thể sửa đổi nhiều lần.
- WORM Disks
  - Các WORM disk ("Write Once, Read Many Times") được ghi chỉ 1 lần.
  - Màng nhôm mỏng được xen giữa 2 mặt đĩa bằng kính hay chất dẻo.
  - Để ghi 1 bit, ổ đĩa sử dụng ánh sáng laser để đốt một hố nhỏ qua lớp nhôm; thông tin có thể bị phá hủy vì không thể thay đổi.
  - Rất bền và đáng tin cậy.
  - Vd: CD-R, DVD-R (Digital Video/Versatile Disk – Recordable)
- *Read Only* disks, ví dụ CD-ROM và DVD, sử dụng kỹ thuật tương tự (hố được ép chặt, không phải bị đốt).

b) Tapes

- So với đĩa, băng rẻ hơn và lưu chứa nhiều dữ liệu hơn, nhưng sự truy nhập ngẫu nhiên chậm hơn nhiều.
- Băng là một giải pháp tiết kiệm cho những mục đích không yêu cầu truy nhập nhanh, vd:
  - lưu trữ bản sao của dữ liệu trên đĩa.
  - sử dụng trong những trung tâm siêu máy tính lớn để lưu trữ lượng thông tin khổng lồ phục vụ nghiên cứu khoa học.

c) Công nghệ tương lai?

- Công nghệ lưu trữ ảnh giao thoa laser (Holographic storage):
  - Sử dụng ánh sáng laser để ghi các bức ảnh giao thoa laser (holographic photograph) trên các thiết bị đặc biệt.
  - Các bức ảnh đen trắng là một mảng 2 chiều các pixel, mỗi pixel biểu diễn 1 bit: 0-đen; 1-trắng
  - Một bức ảnh có thể lưu hàng triệu bit dữ liệu
  - Tất cả các pixel được truyền rất nhanh với tốc độ ánh sáng laser → tốc độ truyền dữ liệu rất cao
  - ⇒ Là công nghệ lưu trữ đầy hứa hẹn của tương lai.
- Hệ cơ khí vi điện tử (Micro electronic mechanical system - MEMS): chế tạo các chip điện tử để sản xuất các thiết bị lưu trữ nhỏ, nếu thành công sẽ cung cấp công nghệ lưu trữ dữ liệu không khả biến, nhanh hơn đĩa từ và rẻ hơn DRAM bán dẫn.

11.8.2. Hỗ trợ của HĐH để quản lý các thiết bị

- Giao diện ứng dụng:
  - Hầu hết các HĐH quản lý các đĩa khả chuyển giống như các đĩa cố định – một đĩa mới phải được format và tạo một hệ thống file rỗng trên đó.
  - Các thao tác cơ bản với ổ đĩa: read, write, seek.
- Đặt tên file:
  - Vấn đề đặt tên các file trên các thiết bị nhớ khả chuyển là đặc biệt khó khi chúng ta muốn ghi dữ liệu lên nó trên một máy tính và rồi sử dụng nó trên một máy tính khác:
    - Tìm đường dẫn đến file, kiểu ổ đĩa có tương thích?
    - Thứ tự lưu trữ các byte dữ liệu khác nhau
- Quản lý bộ nhớ kiểu phân cấp
  - cơ sở, cấp hai, cấp ba

11.8.3. Vấn đề hiệu năng
a) Tốc độ

- Hai khía cạnh tốc độ trong bộ nhớ cấp ba là dải thông (bandwidth) và trễ truy nhập (latency).
- Dải thông (số byte/giây).
  - Dải thông liên tục (Sustained bandwidth) – tốc độ dữ liệu trung bình trong suốt quá trình truyền lớn; được tính bằng số byte/thời gian truyền.

    Là tốc độ dữ liệu khi dòng dữ liệu truyền thực sự
  - Dải thông có hiệu lực (Effective bandwidth) – tính trung bình trên toàn bộ thời gian vào-ra.

    Là tốc độ dữ liệu tổng thể của ổ đĩa.

  Dải thông của ổ đĩa thường được hiểu là sustained bandwith

  Các removable disk: 0.25 - 5 MB/s; Các tape: 0.25 - 30 MB/s

Tốc độ (tiếp)

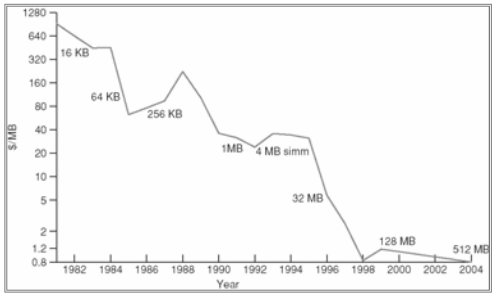
- Trễ truy nhập – khoảng thời gian cần thiết để định vị dữ liệu.
  - Thời gian truy nhập của ổ đĩa – dịch arm tới cylinder được chọn và đợi trễ quay (rotational latency); < 35 ms.
  - Truy nhập trên băng đòi hỏi phải cuộn ống băng cho đến khi khối được chọn chạm đến đầu băng; tốn hàng chục hoặc hàng trăm giây.
  - Nói chung truy nhập ngẫu nhiên trên băng chậm hơn trên đĩa hàng nghìn lần.

b) Độ tin cậy

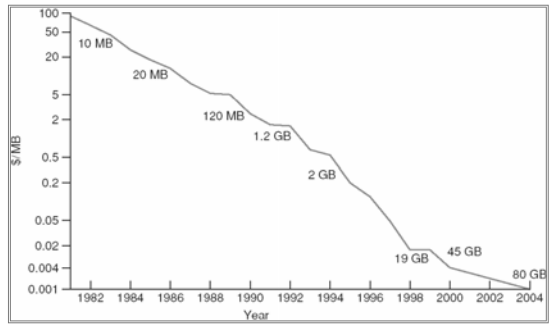
- Một ổ đĩa cố định có vẻ đáng tin cậy hơn một ổ đĩa hay ổ băng khả chuyển.
- Một đĩa quang có vẻ đáng tin cậy hơn một đĩa từ hay băng từ.
- Sự rơi đầu từ trong một đĩa cứng cố định thường phá hủy dữ liệu, trong khi đó hỏng ổ băng hay ổ đĩa quang thường không làm hỏng đến dữ liệu.

c) Giá thành (USD/1 MB)

- Bộ nhớ chính (Main memory) đắt hơn nhiều so với bộ nhớ cấp hai và cấp ba.
- Giá mỗi MB DRAM, 1981-2004

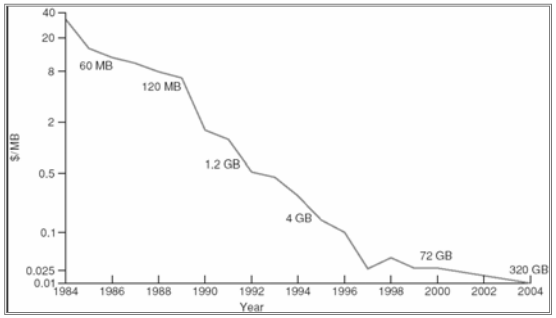


Giá mỗi MB đĩa từ cứng, 1981-2004





Giá mỗi MB ổ băng, 1984-2004





End of Chapter 11

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 12: Hệ thống vào-ra

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung

Nội dung chương 12

- Phần cứng vào-ra — I/O Hardware
- Giao diện vào-ra ứng dụng — Application I/O Interface
- Hệ thống con vào-ra — Kernel I/O Subsystem
- Chuyển yêu cầu vào ra tới thao tác phần cứng
  Transforming I/O Requests to Hardware Operations
- Streams
- Performance

Mục tiêu

- Khám phá cấu trúc I/O subsystem của HĐH
- Thảo luận các nguyên lý của phần cứng vào-ra và sự phức tạp của nó.
- Cung cấp chi tiết về các khía cạnh hiệu năng của phần cứng và phần mềm vào-ra.

Mở đầu

- Hai công việc chính của một máy tính:
  - Vào-ra
  - Xử lý
- Trong nhiều trường hợp, công việc chính là vào-ra
  - duyệt một trang web, chỉnh sửa một file
- Vai trò của HĐH trong vào-ra của máy tính là quản lý và điều khiển các thiết bị vào-ra và các hoạt động vào-ra.
- Các thiết bị vào-ra rất đa dạng về chức năng và tốc độ
  - cần các phương pháp điều khiển khác nhau, tạo thành *hệ thống con vào-ra (I/O subsystem)* của kernel.

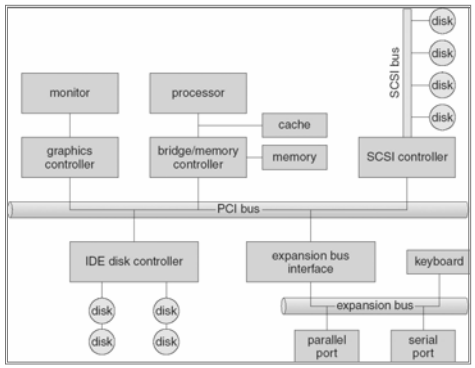
12.1. I/O Hardware

- Có vô số loại thiết bị vào-ra
- Các khái niệm chung:
  - **Port**: cổng giao tiếp giữa thiết bị vào-ra với máy tính, vd: serial, parallel, usb, PS/2 ports…
  - Bus (**daisy chain** or shared direct access): tập dây dẫn truyền thông tin giữa các thành phần trong máy tính và trong các thiết bị vào-ra, vd: PCI bus, SCSI bus
  - Controller (host adapter): tập hợp các thiết bị điện tử điều khiển cổng, bus, hoặc thiết bị vào-ra; controller có 1 hay nhiều thanh ghi dành cho các tín hiệu dữ liệu và điều khiển; vd: serial-port controller là 1 chip.

I/O hardware (tiếp)

- Bộ điều khiển thiết bị (device controller) hỗ trợ các giao thức kết nối, buffering, caching…
- Các lệnh vào-ra (I/O instructions) điều khiển các thiết bị
- Các thiết bị có địa chỉ, được sử dụng bởi:
  - Các lệnh vào-ra trực tiếp: chuyển dữ liệu qua các cổng vào-ra
  - Vào-ra theo bản đồ bộ nhớ (**Memory-mapped** I/O)
- Có bộ điều khiển thiết bị có thể hỗ trợ cả 2 kỹ thuật vào-ra trên:
  - graphics controller có các cổng vào-ra cho các hoạt động điều khiển cơ bản, đồng thời có vùng bộ nhớ ánh xạ lớn để lưu nội dung màn hình.
  - các tiến trình ghi dữ liệu lên vùng nhớ trên, controller sẽ tạo ảnh màn hình theo dữ liệu đó.
  - ghi hàng triệu byte lên vùng nhớ nhanh hơn nhiều so với thực hiện hàng triệu lệnh vào-ra trực tiếp.

Cấu trúc bus PC thông thường



Một số địa chỉ các cổng vào-ra thiết bị trên PC

I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)

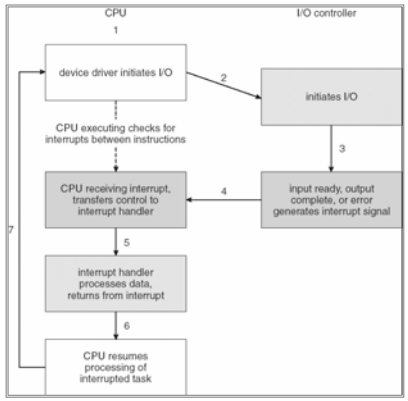
Polling - thăm dò lần lượt

- CPU cần xác định trạng thái của thiết bị ⇒ thực hiện Polling
  - Command-ready: controller sẵn sàng thực hiện lệnh
  - Busy: controller đang bận
  - Error
- Là vòng lặp busy-wait đợi vào-ra từ thiết bị:
  - đọc thanh ghi trạng thái lặp đi lặp lại cho đến khi bit *busy* = 0 để thực hiện tiếp lệnh vào-ra;
  - vd: đọc trạng thái của serial port để tránh tràn buffer gây mất dữ liệu.
- Nói chung hoạt động polling là có hiệu quả, nó chỉ không hiệu quả khi được cố gắng thực hiện lặp lại nhiều lần mà hiếm khi tìm thấy một thiết bị sẵn sàng.
  - Giải pháp: Mỗi controller nên có cơ chế báo trạng thái rỗi cho CPU.

Interrupts - Ngắt

- Thiết bị vào-ra yêu cầu ngắt CPU. Thực hiện xong mỗi lệnh, CPU đọc *dòng yêu cầu ngắt* (trên CPU) chứa các ngắt đến.
- Các ngắt có thể là:
  - Ngắt không che được (non-maskable): báo các trình trạng lỗi phải xử lý ngay, vd: lỗi chia 0, lỗi stack, page fault
  - Ngắt che được (maskable): có thể bị bỏ qua hoặc bị hoãn xử lý, vd: ngắt gây ra bởi thiết bị
- Khi có ngắt:
  - CPU lưu trạng thái đang làm việc, vd: đ/c trở về, các g/t hiện thời
  - Nhảy đến thường trình xử lý ngắt (có địa chỉ xác định trong bộ nhớ)
  - Trình xử lý ngắt (Interrupt handler) xác định nguyên nhân ngắt, xử lý và thực hiện lệnh trở về từ ngắt
  - CPU khôi phục trạng thái trước ngắt

Vòng lặp xử lý ngắt vào-ra



Interrupts (tiếp)

- Trình xử lý ngắt tìm nguyên nhân ngắt trong bảng vector ngắt
- Vectơ ngắt gửi các ngắt tới đúng trình xử lý nó dựa trên mức ưu tiên ngắt
- Cơ chế ngắt cũng được dùng để xử lý ngoại lệ

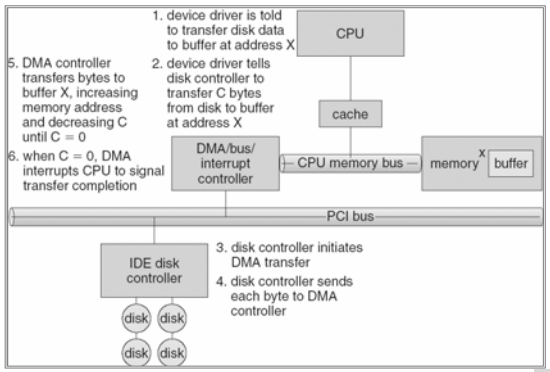
Bảng vectơ ngắt của bộ VXL Intel Pentium

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Direct Memory Access (DMA)

- Được sử dụng để tránh phải sử dụng vào-ra bằng chương trình (programmed I/O) khi lượng dữ liệu cần chuyển lớn.
- Cần có DMA controller (DMAC)
- Chuyển dữ liệu trực tiếp giữa thiết bị vào-ra và bộ nhớ mà không cần qua CPU.

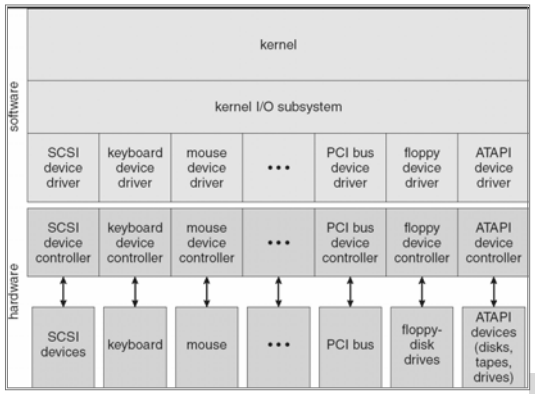
Sáu bước thực hiện truyền dữ liệu DMA



12.2. Giao diện vào-ra ứng dụng

- Làm thế nào để 1 ứng dụng có thể mở một file trên đĩa mà không cần biết loại đĩa là gì?
- Sự khác nhau của các thiết bị vào-ra được gói gọn trong các kernel module được gọi là device driver, thay đổi đối với mỗi thiết bị nhưng tuân theo những giao diện chuẩn.
- Lớp device-driver ẩn đi sự khác nhau trong các I/O controller đối với I/O subsystem của kernel.
- Các thiết bị được phân loại nhiều cách khác nhau
  - Vào-ra theo chuỗi ký tự hoặc khối
  - Truy nhập tuần tự hoặc ngẫu nhiên
  - Có thể chia sẻ hoặc chuyên dụng
  - Tốc độ xử lý
  - read-write, read only, hoặc write only

Một cấu trúc Kernel I/O



Đặc điểm của các thiết bị vào-ra

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read–write	CD-ROM graphics controller disk

Các thiết bị vào ra theo khối và theo ký tự

- Các thiết bị vào ra theo khối chủ yếu gồm các ổ đĩa
  - Các lệnh bao gồm: read, write, seek
  - Các ứng dụng thường truy nhập các thiết bị này qua một giao diện hệ thống file (file-system access) hoặc mảng tuyến tính các khối (raw I/O access)
  - Có thể truy nhập file theo bản đồ bộ nhớ
- Các thiết bị ký tự bao gồm: bàn phím, chuột, cổng nối tiếp
  - Các lệnh gồm: get, put
  - Trên đỉnh của giao diện này, các thư viện có thể được xây dựng cho phép các thao tác chỉnh sửa và xử lý buffer; vd: khi ấn phím backspace, ký tự trước đó bị vứt ra khỏi dòng vào (input stream)

Các thiết bị mạng

- Hầu hết các HĐH cung cấp giao diện vào-ra dùng riêng cho mạng, khác với giao diện vào-ra sử dụng cho các đĩa.
- Unix và Windows NT/9x/2000 dùng giao diện socket
  - Tách riêng giao thức mạng khỏi hoạt động mạng
  - Có bao gồm chức năng `select` để quản lý tập các socket.
- Nhiều phương pháp giao tiếp liên tiến trình và giao tiếp mạng khác cũng được sử dụng (pipes, FIFOs, streams, queues, mailboxes)

Clocks và Timers

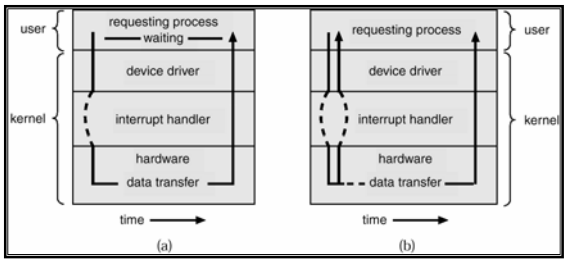
- Cung cấp thời gian hiện tại, thời gian trôi qua, thiết lập định thời (khởi sự hành động X tại thời điểm T)
- **Programmable interval timer** (bộ định thời có khoảng thời gian lập trình được) được sử dụng cho việc định thời gian, sinh các ngắt định kỳ.

Blocking and Nonblocking I/O

- Blocking – tiến trình bị treo đến khi vào-ra kết thúc
  - Dễ hiểu, dễ sử dụng
  - Không thể áp dụng cho một số trường hợp
- Nonblocking – lời gọi vào-ra lặp lại nhiều nhất có thể
  - Sử dụng trong các giao diện người sử dụng, đưa dữ liệu vào bộ nhớ đệm (buffered I/O), vd: nhận dữ liệu vào từ bàn phím và chuột, trong khi xử lý và hiển thị trên màn hình.
  - Được thực hiện thông qua đa luồng
  - Lời gọi vào-ra lặp lại nhanh, với giá trị trả về cho biết bao nhiêu byte đã được chuyển
- Asynchronous – lập tức gọi return, không đợi vào-ra kết thúc, tiến trình vẫn chạy trong khi vào-ra đang được thực hiện → Khó sử dụng
  - I/O subsystem báo cho tiến trình biết khi nào vào-ra kết thúc

Hai phương pháp vào-ra

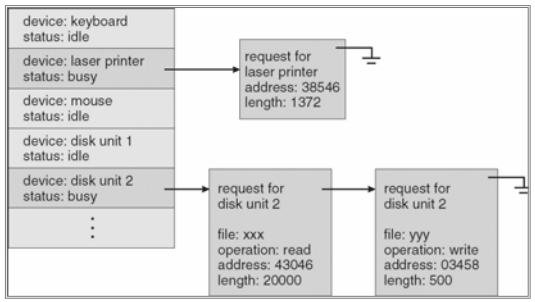
Đồng bộ - Synchronous Không đồng bộ - Asynchronous



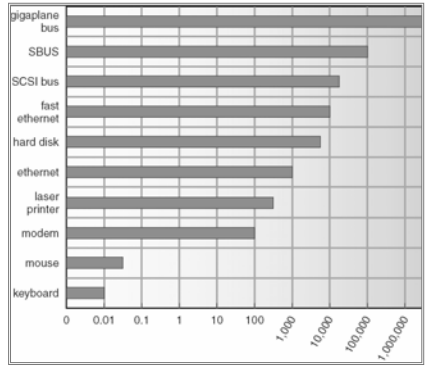
12.3. Kernel I/O Subsystem

- Kernel I/O Subsystem cung cấp nhiều dịch vụ liên quan đến vào-ra: scheduling, buffering, caching, spooling, error handling…
- I/O Scheduling
  - Lập lịch các yêu cầu vào-ra nhằm xác định một thứ tự thực hiện chúng hợp lý.
  - Thực hiện lập lịch vào-ra bằng cách duy trì một queue chứa các yêu cầu cho mỗi thiết bị
  - HĐH cố gắng đạt tối ưu: cải thiện hiệu năng hệ thống, giảm thời gian đáp ứng trung bình…
- Buffering – chứa dữ liệu trong bộ nhớ trong khi vẫn truyền dữ liệu giữa các thiết bị
  - Để giải quyết vấn đề không khớp tốc độ giữa các thiết bị
  - Để giải quyết vấn đề không khớp kích thước dữ liệu truyền giữa các thiết bị

Bảng trạng thái thiết bị



Tốc độ truyền dữ liệu của các thiết bị trên Sun Enterprise 6000



Kernel I/O Subsystem (tiếp)

- Caching – bộ nhớ nhanh giữ bản copy của dữ liệu
  - Luôn chỉ giữ một bản copy
  - Là chìa khóa của hiệu năng
- Spooling – giữ output cho một thiết bị
  - Nếu thiết bị chỉ có thể phục vụ một yêu cầu tại một thời điểm
  - vd: Printing
- Device reservation –cung cấp sự truy nhập riêng tới một thiết bị
  - Là các lời gọi hệ thống để phân phối và thu hồi các thiết bị cho các tiến trình
  - Là một cách đề phòng bế tắc

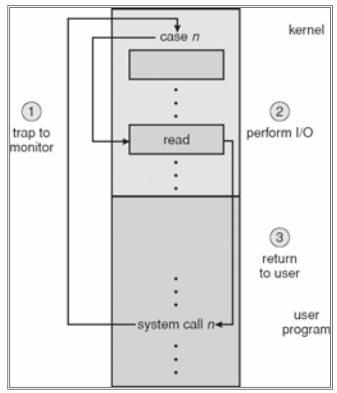
Kernel I/O Subsystem (tiếp)

- Error handling
  - HĐH sử dụng bộ nhớ được bảo vệ (protected memory) có thể ngăn ngừa cẩn thận nhiều loại lỗi phần cứng và lỗi ứng dụng.
  - HĐH có thể phục hồi sau các lỗi đọc đĩa, ghi ầu, thiết bị không khả dụng.
  - Hầu hết trả về một mã lỗi khi yêu cầu vào-ra lỗi

Kernel I/O Subsystem (tiếp)

- I/O Protection
  - Tiến trình của người sử dụng có thể vô tình hay cố ý cố gắng phá vỡ hoạt động bình thường thông qua các lệnh vào-ra bất hợp lệ
    - Tất cả các lệnh vào-ra được định nghĩa là đặc quyền
    - Vào-ra phải được thực hiện thông qua các system call.
      - Các vùng nhớ dùng cho ánh xạ (memory-mapped) và cổng vào-ra (I/O port) cũng phải được bảo vệ

Sử dụng system call để thực hiện vào-ra



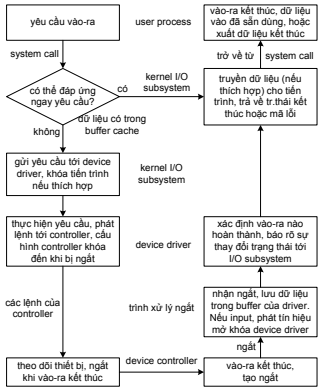
Kernel I/O Subsystem (tiếp)

- Các cấu trúc dữ liệu kernel
  - Kernel lưu thông tin trạng thái cho các thành phần vào-ra, bao gồm các bảng file mở, các kết nối mạng, trạng thái thiết bị vào-ra theo ký tự.
  - Rất nhiều cấu trúc dữ liệu phức tạp để theo dõi các buffer, phân phối bộ nhớ, các khối nhớ "dirty".
  - Một số phương pháp hướng đối tượng và truyền thông điệp được sử dụng để thực thi vào-ra.

12.4. Chuyển yêu cầu vào-ra tới các hoạt động phần cứng

- Các bước đọc một file từ đĩa cho một tiến trình:
  1. Xác định thiết bị chứa file
  2. Thông dịch tên tương ứng với thiết bị
  3. Đọc dữ liệu từ đĩa vào buffer
  4. Làm cho dữ liệu là khả dụng đối với tiến trình yêu cầu
  5. Trả điều khiển cho tiến trình

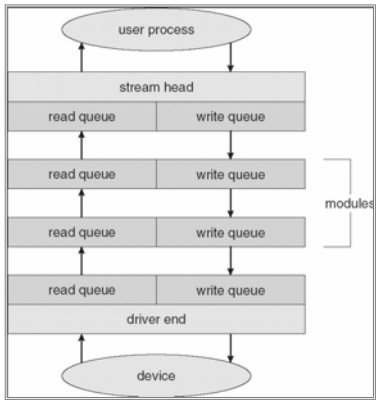
Vòng đời của một yêu cầu vào-ra



12.5. STREAMS (UNIX, Solaris)

- **STREAM** – một kênh giao tiếp có thể giữ và nhận các tín hiệu đồng thời cùng một lúc giữa một tiến trình mức người sử dụng với một thiết bị (truyền song công hoàn toàn)
- Một STREAM bao gồm:
  - STREAM head giao tiếp với tiến trình người sử dụng
  - driver end giao tiếp với thiết bị
  - các STREAM module nằm giữa 2 phần trên.
- Mỗi module có một **read queue** và một **write queue**
- Phương thức message passing được sử dụng để giao tiếp giữa các queue

Cấu trúc các STREAM



12.6. Performance

- Vào-ra là nhân tố chính trong hiệu năng hệ thống:
  - Luôn yêu cầu CPU thực hiện các mã của device driver, kernel I/O
  - Chuyển ngữ cảnh gây các ngắt
  - Copy dữ liệu luôn chiếm dụng bus
  - Lưu lượng mạng đặc biệt căng thẳng



Cải thiện hiệu năng

- Giảm số lần chuyển ngữ cảnh
- Giảm copy dữ liệu
- Giảm ngắt bằng cách sử dụng truyền lượng lớn dữ liệu, các controller thông minh, polling
- Sử dụng DMA
- Cân bằng CPU, memory, bus, và hiệu năng vào-ra để đạt thông lượng cao nhất.



End of Chapter 12

BÀI GIẢNG
NGUYÊN LÝ HỆ ĐIỀU HÀNH

Chương 13: Protection & Security

Phạm Quang Dũng
Bộ môn Khoa học máy tính
Khoa Công nghệ thông tin
Trường Đại học Nông nghiệp Hà Nội
Website: fita.hua.edu.vn/pqdung



Nội dung chương 13

- Protection Problem
  - Cấu trúc Protection Domain
  - Ma trận truy nhập - Access Matrix
- Security Problem
  - Thẩm định quyền — Authentication
  - Các hiểm họa chương trình — Program Threats
  - Các hiểm họa hệ thống — System Threats
  - Giám sát hiểm họa — Threat Monitoring
  - Tường lửa — Firewall
  - Mã hóa — Encryption





13.1. The Protection Problem

- Hệ thống máy tính gồm một tập hợp các tiến trình và các đối tượng (hardware và software).
- Mỗi đối tượng có một tên duy nhất và có thể được truy nhập qua một tập các thao tác xác định.
- Protection problem – đảm bảo rằng mỗi đối tượng được truy nhập đúng và chỉ bởi những tiến trình được phép.
- Chỉ quan tâm đến môi trường bên trong hệ thống (internal environment)

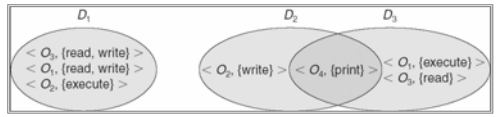




Cấu trúc Protection Domain

- Access-right = <*object-name*, *rights-set*>

  trong đó *rights-set* là một tập con của tất cả các thao tác hợp lệ có thể được thực hiện trên đối tượng.
- Domain = tập các access-right



- Một Domain có thể là: user, process, procedure



Ma trận truy nhập - Access Matrix

- Coi protection là một ma trận (*access matrix*)
  - Các hàng biểu diễn các domain
  - Các cột biểu diễn các đối tượng (objects)
- *Access(i, j)* là tập hợp các thao tác mà một tiến trình thực hiện trong $Domain_i$ có thể tác động trên $Object_j$

domain \ object	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Cách sử dụng Access Matrix

- Nếu một tiến trình trong Domain D_i cố gắng thực hiện thao tác "op" trên đối tượng O_j, thì "op" phải có trong access matrix, trên giao điểm hàng D_i và cột O_j .
- Có thể được mở rộng thành bảo vệ động (dynamic protection).
  - Các thao tác để thêm, xóa các access right.
  - Các access right đặc biệt:
    - *chủ của O_i (owner of O_i)*
    - *copy thao tác từ O_i đến O_j*
    - *control – D_i có thể thay đổi các access right của D_j*
    - *transfer – chuyển từ domain D_i đến D_j*

Sự thực hiện của Access Matrix

- Mỗi cột = Danh sách quyền truy nhập (access-control list) đối với một object.

  Xác định domain nào có thể thực hiện thao tác gì.

  Domain 1 = Read, Write

  Domain 2 = Read

  ⋮

- Mỗi hàng = Danh sách khả năng (capability list)

  Đối với mỗi domain, những thao tác gì được phép trên những object nào.

  Object 1 – Read

  Object 5 – Read, Write, Delete, Copy

13.2. Security Problem

- Security phải quan tâm đến môi trường bên ngoài hệ thống (external environment), và bảo vệ hệ thống khỏi:
  - truy nhập trái phép - unauthorized access
  - cố tình/vô tình thay đổi hoặc phá hoại
- Bảo vệ để chống lại sự sử dụng sai (misuse) vô tình dễ hơn cố tình.

Sự thẩm định quyền - Authentication

- Xác nhận người dùng (User identity) hầu hết thường được thiết lập thông qua mật khẩu (*password)*, có thể được coi là một trường hợp đặc biệt của chìa khóa hoặc khả năng.

Tìm Password

- Nếu bạn biết user, hãy thử với tên hoặc ngày sinh của chồng/vợ người đó.
- *Brute force*: thử tất cả các sự kết hợp của chữ và số.
- *Dictionary attack*: thử tất cả các từ trong từ điển, cả từ đơn và từ ghép.
- *Shoulder surfing*: nhìn bàn phím khi user gõ mật khẩu.
- *Keystroke recorder*: Các máy tính Internet cafe ghi tất cả lần bấm phím.
- Ăn cắp password file khi sử dụng hệ thống.
- …

Bảo vệ Password

- Thường xuyên thay đổi password
- Sử dụng password khó đoán: dài, không phải từ trong từ điển, các ký tự đầu tiên trong câu, hệ thống các dấu.
- Bắt đợi sau mỗi lần thử gõ password sai.
- Lưu password ở dạng mã hóa (Unix `crypt()`).
- Chỉ administrator có thể truy nhập file password và bằng một chương trình đặc biệt.
- Sử dụng password 1 lần: máy đưa ra một bài toán *c*, user tính rồi trả lời *f(c)*, máy cũng tính *f(c)* rồi so sánh. Chỉ user và computer biết hàm *f*.
- …

Các hiểm họa chương trình-Program Threats

- Trojan Horse – Chú ngựa thành Troa
  - Là đoạn mã mà dùng sai môi trường của nó.
  - Khai thác kỹ thuật cho phép chương trình viết bởi người này có thể được thực hiện bởi người khác.
  - **Spyware, pop-up browser windows, covert channels**
- VD1
  - Bob soạn một plugin mới hấp dẫn cho một chương trình soạn thảo văn bản.
  - Sue, nhân viên của Bob, sử dụng plugin đó, và nó làm việc tốt...
  - ... ngoại trừ một điều là nó copy mọi file Sue soạn thảo vào một thư mục của Bob.
  - Khi Sue soạn một bức thư mật cho Tổng giám đốc về sự quản lý yếu kém của Bob, Bob đọc nó trước khi Sue gửi…



Program Threats (tiếp)

- VD2
  - Bob viết một chương trình hiển thị một hộp thoại full-screen trông giống hệt màn hình đăng nhập Windows.
  - Bob chạy chương trình rồi rời Internet cafe mà không log out.
  - Sue gõ username và password trong chương trình của Bob.
  - Bob cất thông tin đó, rồi chương trình của anh ta thực hiện việc log out, vì vậy Sue sẽ thấy màn hình đăng nhập thật của Windows.
  - Sue nghĩ rằng mình gõ đã sai mật khẩu, thử lại và vào được…
- Internet Trojan horse:
  - Bob mua một domain tên là `www.citibanc.com`, và lập một trang web giống trang chủ của Citibank.
  - Sue gõ nhầm URL, vào trang web của Bob.
  - Bob ăn cắp được password của Sue, rồi gửi (redirect) Sue tới đúng trang web của Citibank.





Program Threats (tiếp)

- *Trap door*
  - User-id hoặc password riêng làm hỏng thủ tục bảo mật thông thường.
  - Có thể được gắn trong một trình biên dịch (compiler).
- Ví dụ:
  - Bob được Ngân hàng Vietnam thuê viết 1 phần mềm quản lý tài khoản.
  - Bob viết CT, và nó làm việc hoàn hảo, ngoại trừ nó có 2 trap door:
    1. Mã của Bob làm tròn xuống, vd $1.995 trở thành $1.99. $0.005 được gửi vào tài khoản cá nhân của Bob.
    2. Mã của Bob bao gồm một đoạn mà tại đó anh ta kiểm tra một username và password cố định riêng thay vì kiểm tra trong password file. Bob đăng nhập vào sử dụng tài khoản đó để kích hoạt đoạn mã trên chỉ vào ngày Ngân hàng tính tiền lãi phải trả.
  - Hãy tưởng tượng rằng Bob viết 1 compiler mà thêm một trap door vào mọi chượng trình!





Program Threats (tiếp)

- Stack and Buffer Overflow
  - Là cách tấn công phổ biến nhất từ một kẻ ngoài hệ thống trên một mạng hoặc kết nối dial-up để đạt được sự truy nhập trái phép vào hệ thống đích
  - Kiểu tấn công này lợi dụng một lỗi trong một chương trình (tràn stack hoặc bộ nhớ đệm buffer.)





Các hiểm họa hệ thống - System Threats

- Worms – là 1 chương trình sử dụng cơ chế đẻ trứng để tấn công liên tục hệ thống; tự động copy chính nó, dùng hết tài nguyên hệ thống và có thể khóa tất cả các tiến trình khác sử dụng hệ thống.
- Internet worm? Morris Internet worm – 1988?
- Denial of Service (DOS)
  - Làm quá tải máy tính đích để ngăn nó làm việc có ích.
  - SCO (1 công ty Mỹ) được tài trợ bởi Microsoft để sắp xếp việc kiện cáo chống lại các công ty sử dụng Linux.
  - SCO làm một số fan của Linux tức giận, họ thực hiện cuộc tấn công DOS vào web site của công ty.
  - Họ viết những đoạn script đơn giản giả là trình duyệt muốn kết nối tới web site của SCO...
  - nhưng dừng kết nối ngay sau khi họ gửi yêu cầu, và...
  - lập tức gửi 1 yêu cầu khác, rồi 1 nữa và cứ tiếp tục như vậy…
  - web site của SCO đã không thể truy cập trong vài tháng.



System Threats: Viruses

- Giống như trap door, virus là 1 đoạn mã được nhúng trong 1 chương trình có ích. Nhưng 1 virus đơn lẻ có thể tự nó tấn công nhiều chương trình khác nhau.
- Đơn giản là nó tự chèn vào giữa đoạn chương trình bình thường.
- Khi được thực hiện, nó copy chính nó đến các chương trình khác, và/hoặc gây ra sự hư hại (lựa chọn thường phụ thuộc vào thời gian đã thực hiện, hoặc vào ngày nhất định, vd virus Michelangelo).
- Các chương trình và dữ liệu không độc lập là cửa ngõ tốt cho virus tấn công:
  - Các macro của Word là chương trình VB được thực hiện khi tài liệu được nạp.
  - Các file đính kèm email (vd virus *love bug*).
  - Web pages (vd mã javascript thay *hosts* file, chuyển `www.yahoo.com` tới một web site quảng cáo).

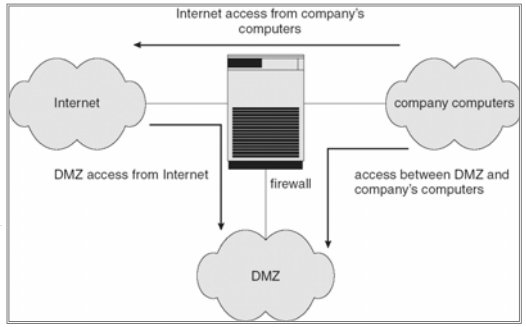
Giám sát hiểm họa (Threat Monitoring)

- Kiểm tra các hoạt động đáng nghi – vd một số lần thử password sai có thể báo hiệu đang thử password.
- Audit log (kiểm định truy nhập) – ghi thời gian, user, và kiểu của tất cả sự truy nhập một đối tượng; hữu dụng cho sự phục hồi từ một sự xâm phạm và cho việc phát triển những biện pháp bảo mật tốt hơn.
- Định kỳ quét (scan) hệ thống tìm các lỗi bảo mật rồi tự động sửa hoặc thông báo cho người quản trị hệ thống.
  - Password ngắn hoặc dễ đoán
  - Các chương trình có quyền trái phép
  - Các tiến trình chạy lâu không mong đợi
  - Trojan horse, …

FireWall

- Firewall tương tự như một màn ngăn lửa giữa động cơ ô tô với ghế ngồi của hành khách.
- LAN firewall: router kiểm tra kỹ và có thể loại bỏ các gói tin:
  - Một firewall được đặt giữa các host đáng tin T và không đáng tin U.
  - Nó cho phép các message đi từ T tới U không giới hạn.
  - Các message từ U tới T chỉ đi qua được nếu chúng là các hồi âm (reply) của các message trước đó được gửi từ T tới U.
- DMZ (Demilitarized Zone – Vùng phi quân sự):
  - Tất cả message từ U tới T mà không phải hồi âm thì đi đến các máy tính DMZ.
  - DMZ chứa web server, VPN server (truy nhập máy T an toàn với tư cách của các host U được phép), …

Dùng Firewall thực hiện an ninh mạng qua các Domain





Mã hóa - Encryption

- Mã hóa văn bản rõ ràng thành mật mã.
- Những đặc tính của kỹ thuật mã hóa tốt:
  - Tương đối đơn giản để người sử dụng có phép (authorized user) có thể mã hóa và giải mã.
  - Cách mã hóa không chỉ phụ thuộc vào sự bí mật của giải thuật mà còn vào một tham số của giải thuật được gọi là khóa mã hóa (encryption key).
  - Thực sự khó đối với kẻ xâm phạm có thể xác định được khóa mã hóa.
- *Data Encryption Standard* thay thế các ký tự và sắp xếp lại thứ tự của chúng trên nền tảng của khóa mã hóa được cung cấp bởi người sử dụng có phép thông qua một kỹ thuật mã hóa.





Ví dụ mã hóa - SSL

- SSL – Secure Socket Layer
- Giao thức bằng mật mã giới hạn 2 máy tính chỉ trao đổi dữ liệu với nhau.
- Được sử dụng giữa các web server và các browser nhằm giao tiếp an toàn (credit card numbers)
- Server được xác nhận bởi một **certificate** (≈ giấy chứng nhận).
- Giao tiếp giữa mỗi máy tính sử dụng mật mã khóa đối xứng.



End of Chapter 13